

**IMPLEMENTASI *GRAPH*
RETRIEVAL-AUGMENTED GENERATION
BERBASIS *KNOWLEDGE GRAPH* UNTUK
MENINGKATKAN KUALITAS *RETRIEVAL* DAN
VALIDITAS KONFIGURASI KUBERNETES**

Laporan Tugas Akhir

Oleh

Jihan Aurelia

18222001



**PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG**

Mei 2026

LEMBAR PENGESAHAN

IMPLEMENTASI *GRAPH RETRIEVAL AUGMENTED GENERATION* UNTUK MENINGKATKAN PRESISI RETRIEVAL DAN VALIDITAS SINTAKSIS PADA KONFIGURASI KUBERNETES

Laporan Tugas Akhir

Oleh

**Jihan Aurelia
18222001**

Program Studi Sistem dan Teknologi Informasi
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Laporan Tugas Akhir ini telah disetujui dan disahkan
di Bandung, pada tanggal 28 Mei 2026

Pembimbing

Dr. Ir. Dimitri Mahayana, M.Eng.

NIP. 196808091991021001

PERNYATAAN ORISINALITAS

Saya yang bertanda tangan di bawah ini menyatakan dengan sesungguhnya bahwa:

1. Tugas Akhir ini adalah hasil karya saya sendiri yang dibuat dengan sebenar-benarnya sesuai dengan kaidah ilmiah dan etika akademik.
2. Semua sumber rujukan dan data yang saya gunakan dalam penyusunan laporan tugas akhir ini, baik yang berupa kutipan langsung maupun tidak langsung, telah saya cantumkan sumbernya dengan baik dan benar sesuai dengan ketentuan penulisan laporan tugas akhir.
3. Isi laporan ini belum pernah diajukan pada program pendidikan di institusi mana pun.

Apabila di kemudian hari terbukti bahwa laporan ini melanggar hal-hal di atas, saya bersedia menerima sanksi sesuai dengan peraturan yang berlaku di Institut Teknologi Bandung.

Bandung, 28 Mei 2026

Jihan Aurelia

NIM: 18222001

PERNYATAAN PENGGUNAAN AI

Saya yang bertanda tangan di bawah ini menyatakan bahwa dalam penulisan dokumen Tugas Akhir ini, saya menggunakan alat bantu kecerdasan artifisial (AI) untuk beberapa aspek tertentu di dalam proses penulisan, seperti yang tercantum dalam tabel berikut.

No.	Jenis	Alat Bantu	Tingkat	Tempat
1	Memeriksa ejaan dan tata bahasa	Grammarly	Tinggi	Semua bab
2	Pembuatan teks	Microsoft Copilot	Sedang	Bab II hal. 15
3	Pembuatan grafik, gambar, atau ilustrasi	Tidak ada	Tidak ada	Tidak ada
4	Pencarian informasi atau referensi	...	...	...
5	Penyusunan bahan diskusi	...	...	...
6	Penyusunan kode program	...	...	...
7	Penyusunan formula atau perhitungan matematis	...	...	...
8	Penyusunan konsep desain atau algoritma	...	...	...
9	Penyusunan analisis data	...	...	...
10	Penyusunan kesimpulan atau rekomendasi	...	...	...

Bandung, 28 Mei 2026

Jihan Aurelia
NIM: 18222001

ABSTRAK

Implementasi *Graph Retrieval-Augmented Generation* Berbasis *Knowledge Graph* untuk Meningkatkan Kualitas *Retrieval* dan Validitas Konfigurasi Kubernetes

oleh

Jihan Aurelia

18222001

Transformasi menuju arsitektur *cloud-native* menempatkan Kubernetes sebagai standar *de facto* untuk orkestrasi kontainer, namun kompleksitas konfigurasi berbasis YAML menjadi tantangan operasional yang signifikan. Pemanfaatan *Large Language Models* (LLM) dalam menjawab pertanyaan teknis Kubernetes rentan menghasilkan konfigurasi yang keliru secara semantik atau tidak valid secara sintaksis—fenomena yang dikenal sebagai halusinasi. Pendekatan *Retrieval-Augmented Generation* (RAG) berbasis vektor yang umum digunakan juga terbukti tidak memadai karena gagal menangkap relasi hierarkis dan referensial yang diperlukan dalam domain Kubernetes, dengan akurasi yang hanya mencapai 63,4% *exact match* dan 69,8% *context precision*.

Penelitian ini membangun sistem *Graph Retrieval-Augmented Generation* (GraphRAG) untuk dokumentasi Kubernetes menggunakan metodologi CRISP-DM. *Knowledge graph* dibangun dari blok *definitions* pada spesifikasi OpenAPI Kubernetes v1.30 (*swagger.json*) dan menghasilkan 725 *node* definisi dengan 18 jenis *edge* yang merepresentasikan relasi struktural dan semantik antar-objek Kubernetes. Mekanisme *retrieval* bertingkat (*cascading retrieval*) memprioritaskan pencocokan nama eksak, dilengkapi *fallback* kemiripan vektor, serta penelusuran graf multi-lompatan (*multi-hop graph traversal*) hingga 3 lompatan. *Pipeline* respon dibangun menggunakan LangGraph dengan GPT-4o-mini untuk ekstraksi niat kueri dan Groq LLaMA untuk pembuatan jawaban.

Sistem dievaluasi menggunakan 97 *fixture* uji tervalidasi pakar dalam tiga dimensi: *Answer Quality* (AnsQ, bobot 40%), *Retrieval Quality* (RetQ, bobot 35%), dan *Reasoning Quality* (ReaQ, bobot 25%). Hasil evaluasi menunjukkan bahwa sistem GraphRAG mencapai skor AnsQ sebesar 0,58, RetQ sebesar 0,65, dan ReaQ sebesar 0,87 dengan total skor komposit sebesar 0,68. Sistem juga menghasilkan berkas konfigurasi YAML yang valid secara sintaksis dan dilengkapi jejak penalaran graf (*reasoning path*) sebagai mekanisme pendeteksi halusinasi LLM.

Kata kunci: *Graph Retrieval-Augmented Generation*, *knowledge graph*, Kubernetes, YAML, halusinasi LLM.

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, saya dapat menyelesaikan penulisan Laporan Tugas Akhir yang berjudul "Implementasi *Graph Retrieval-Augmented Generation* Berbasis *Knowledge Graph* untuk Meningkatkan Kualitas *Retrieval* dan Validitas Konfigurasi Kuberne-tes". Laporan ini disusun sebagai salah satu syarat untuk menyelesaikan program Sarjana di Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.

Bandung, 20 Mei 2026

Penulis

Jihan Aurelia

DAFTAR ISI

LEMBAR PENGESAHAN	iii
PERNYATAAN ORISINALITAS	v
PERNYATAAN PENGGUNAAN AI	vii
ABSTRAK	ix
KATA PENGANTAR	xi
DAFTAR ISI	xiii
DAFTAR GAMBAR	xvii
DAFTAR TABEL	xix
DAFTAR PERSAMAAN	xxi
DAFTAR ALGORITMA	xxiii
DAFTAR <i>LISTING</i>	xxvi
DAFTAR SIMBOL	xxvii
DAFTAR SINGKATAN	xxxix
I PENDAHULUAN	1
I.1 Latar Belakang	1
I.2 Rumusan Masalah	3
I.3 Tujuan	4
I.4 Batasan Masalah	4
I.5 Metodologi	5
II STUDI LITERATUR	9
II.1 Arsitektur <i>Cloud</i> Modern dan Tantangan Kompleksitas	9
II.2 Kubernetes: Platform Orkestrasi Kontainer untuk Aplikasi <i>Cloud-Native</i>	9
II.2.1 Gambaran Umum Arsitektur	10
II.2.2 <i>Resource Model</i> dan Hierarki	11
II.2.3 Manifes YAML dan <i>Infrastructure as Code</i>	12
II.3 <i>Large Language Models</i> (LLM)	13
II.3.1 Definisi dan Kapabilitas	13
II.3.2 <i>Prompt Engineering</i>	14
II.4 <i>Retrieval-Augmented Generation</i> (RAG)	14
II.4.1 Arsitektur <i>Retrieval-Augmented Generation</i>	15
II.4.2 Paradigma RAG	16
II.4.3 Relevansi RAG terhadap Dokumentasi Kubernetes API	17
II.5 <i>Knowledge Graph</i> dan GraphRAG	18
II.5.1 Definisi dan Struktur	18
II.5.2 Integrasi KG dalam RAG pada Tahap Perancangan Solusi	20
II.5.3 GraphRAG: Integrasi <i>Knowledge Graph</i> dan <i>Large Language Models</i>	20
II.6 Metrik Evaluasi Sistem Informasi Retrieval	21
II.6.1 <i>Answer Quality</i>	22
II.6.2 <i>Retrieval Quality</i>	22
II.6.3 <i>Syntactic Validity</i> dan <i>Schema Compliance</i>	23

II.6.4	<i>Reasoning Quality</i>	23
II.7	Penelitian Terkait	23
II.7.1	GraphRAG untuk Platform IoT Domain-Spesifik	24
II.7.2	RAG Hibrida KG-Vektor untuk Manufaktur Pintar	24
II.7.3	HSG-RAG: Konstruksi Basis Pengetahuan Hierarkis untuk Domain Teknis	25
III	ANALISIS MASALAH	27
III.1	Analisis Kondisi Saat Ini	27
III.1.1	Pemahaman Masalah Bisnis (<i>Business Understanding</i>)	27
III.1.2	Pemahaman Data (<i>Data Understanding</i>)	28
III.1.2.1	Sumber dan Spesifikasi Data (<i>Data Source & Specifications</i>)	28
III.1.2.2	Analisis Struktur Dokumen	28
III.1.2.3	Analisis Data Eksploratif Spesifikasi OpenAPI	29
III.1.2.4	Analisis Keterhubungan Data	31
III.1.2.5	Desain Taksonomi <i>Edge</i> untuk <i>Knowledge Graph</i> Kubernetes	32
III.1.2.6	Justifikasi Batas Kedalaman Penelusuran Graf (3 Lompatan)	34
III.1.2.7	Analisis Kompleksitas Pertanyaan dan Batasan Kemampuan Sistem	34
III.2	Analisis Kebutuhan	35
III.2.1	Kebutuhan Fungsional	37
III.2.2	Kebutuhan Non Fungsional	40
III.2.3	Pemetaan Kebutuhan Bisnis terhadap Kerangka Evaluasi	41
III.3	Alternatif Solusi	42
III.3.1	Hybrid KG–Vector RAG	42
III.3.2	<i>GNN-augmented GraphRAG</i> (G-Retriever)	43
III.3.3	<i>Hybrid Neural-Symbolic</i> dengan <i>Relational DB</i> (NeuSym-RAG)	43
III.4	Analisis Pemilihan Solusi	43
III.4.1	Rubrik Evaluasi Solusi	44
III.4.2	Matriks Keputusan & Justifikasi Penilaian	45
III.4.3	Analisis Pemilihan dan Penyesuaian (Selection & Adjustment)	46
IV	PERANCANGAN SISTEM <i>GRAPH</i>RAG BERBASIS <i>KNOWLEDGE GRAPH</i> KUBERNETES	49
IV.1	Rancangan Solusi Secara Garis Besar	49
IV.2	Pemetaan Metodologi terhadap Kebutuhan Sistem	50
IV.3	Perancangan <i>Pipeline</i> Ekstraksi dan <i>Knowledge Graph</i>	56
IV.3.1	<i>Pipeline Ingestion Data</i>	57
IV.4	Perancangan Mekanisme <i>GraphRAG</i>	58
IV.4.1	Perancangan <i>LangGraph Agent Pipeline</i>	58
IV.4.2	Perancangan Mekanisme <i>Retrieval</i> Bertingkat	61
IV.5	Perancangan Validasi YAML Tiga Lapis	62

V	IMPLEMENTASI SISTEM <i>GRAPH</i>RAG BERBASIS <i>KNOWLEDGE</i>	
	<i>GRAPH</i> KUBERNETES	65
V.1	Lingkungan Implementasi	65
V.2	Implementasi <i>Pipeline</i> Ekstraksi dan <i>Knowledge Graph</i>	67
V.2.1	Implementasi <i>Ingestion</i> Data dan <i>Knowledge Graph</i>	67
V.3	Implementasi Mekanisme <i>GraphRAG</i>	68
V.3.1	Implementasi <i>Pipeline</i> LangGraph	68
V.3.2	Implementasi Mekanisme <i>Retrieval</i> Bertingkat	69
V.4	Implementasi Validasi YAML Tiga Lapis	71
VI	EVALUASI	73
VI.1	Metode Evaluasi	73
VI.2	Validasi Dataset oleh Pakar	74
VI.3	Hasil Evaluasi Kuantitatif	76
VI.3.1	Skor Keseluruhan	76
VI.3.2	Skor per Kategori <i>Fixture</i>	76
VI.3.3	Analisis Metrik <i>Reasoning Quality</i>	77
VI.4	Pembahasan Hasil Evaluasi	78
VI.5	Perbandingan dengan Pendekatan <i>Baseline</i>	79
VI.6	<i>Ablation Study</i>	82
VI.6.1	Analisis Sensitivitas Kedalaman Traversal	83
VI.7	Uji Signifikansi Statistik	84
VI.8	Analisis Kondisi Batas Sistem <i>GraphRAG</i>	86
VI.8.1	Keunggulan <i>GraphRAG</i> per Tipe <i>Fixture</i>	86
VI.8.2	Faktor Kuantitatif: <i>Hops</i> dan Derajat Node	87
VI.8.3	Definisi Kondisi Batas	89
VI.8.4	Prediksi Generalisasi ke Domain Lain	89
VII	PENUTUP	91
VII.1	Kesimpulan	91
VII.2	Saran	92
Lampiran A	KODE PROGRAM	99
A.1	Manajemen Memori Percakapan (<i>ZepMemoryStore</i>)	99
A.2	<i>Pipeline Ingestion</i> Data (<i>SwaggerGraphBuilder</i>)	102
A.3	<i>LangGraph Agent Pipeline</i>	112
A.4	Mekanisme <i>Retrieval</i> Bertingkat (<i>StatefulK8sRetriever</i>)	117
A.5	Validasi YAML Tiga Lapis (<i>YAMLValidator</i>)	122
A.6	Query Cypher Neo4j	125
A.7	Fungsi Metrik Evaluasi	129
Lampiran B	FIXTURE EVALUASI PAKAR	141
B.1	Fixture B.1 — <i>deployment_basic</i> (Konseptual)	141
B.2	Fixture B.2 — <i>service_types</i> (Konseptual)	142
B.3	Fixture B.3 — <i>scale_existing_deployment</i> (Lanjutan)	143
B.4	Fixture B.4 — <i>ingress_service_pod</i> (Relasional)	144
B.5	Fixture B.5 — <i>hpa_deployment_pod</i> (Relasional)	146

B.6	Fixture B.6 — cronjob_backup (Generasi YAML)	148
B.7	Fixture B.7 — networkpolicy_deny_all (Generasi YAML) . . .	149
B.8	Fixture B.8 — statefulset_with_pvc (Generasi YAML)	150
B.9	Fixture B.9 — pi_want_to_reject_all_docker (Skenario Nyata)	152
B.10	Fixture B.10 — kubectl_find_pods_with_env (Perintah)	153
Lampiran C HASIL EVALUASI KUANTITATIF		155
C.1	Metrik <i>Answer Quality</i> (AnsQ)	155
C.2	Metrik <i>Retrieval Quality</i> (RetQ)	158
C.3	Metrik <i>Reasoning Quality</i> (ReaQ)	161

DAFTAR GAMBAR

I.1	Tahapan model referensi CRISP-DM (Niakšu 2015)	6
II.1	Arsitektur klaster Kubernetes yang terdiri atas <i>Control Plane</i> dan <i>Worker Nodes</i> , dihubungkan melalui <i>kube-apiserver</i> (The Kubernetes Authors 2024b)	10
II.2	Arsitektur <i>Knowledge Graph</i> (J. Yu dkk. 2021)	18
II.3	Model arsitektur integrasi KG dalam RAG (Knollmeyer, Caymazer, dan Grossmann 2025)	26
III.1	Proporsi objek Kubernetes <i>root</i> vs komponen pendukung dalam spesifikasi OpenAPI Kubernetes	30
IV.1	Rancangan solusi berdasarkan metodologi CRISP-DM	50
IV.2	<i>Architecture Diagram</i> Solusi <i>GraphRAG</i> untuk Mitigasi Halusinasi pada Sistem RAG-LLM	55
IV.3	Diagram Urutan (<i>Sequence Diagram</i>) Alur Kerja Sistem <i>GraphRAG</i>	56
IV.4	Diagram Interaksi Lima <i>Node LangGraph Agent Pipeline</i>	60
V.1	Analisis pemilihan batas karakter <i>graph context</i> . Panel (a) menunjukkan distribusi bimodal panjang konteks pada 97 <i>fixture</i> : 79 kueri (81,4%) menghasilkan 11.606 karakter dan 18 kueri (18,6%) menghasilkan 17.849 karakter, dengan garis hijau menandai batas yang dipilih (12.000 karakter). Panel (b) menunjukkan skor evaluasi empiris (<i>Total</i> tertimbang, AnsQ, RetQ, ReaQ) sebagai fungsi nilai batas untuk lima kandidat yang diuji. Batas 12.000 karakter mencapai <i>Total</i> tertinggi (0,6989) dan ditetapkan sebagai nilai operasional.	69
VI.1	RetQ per tipe <i>fixture</i> pada kedalaman traversal $d = 1$ hingga $d = 5$. Titik bertanda hitam menunjukkan kedalaman yang digunakan oleh sistem adaptif; garis putus-putus ($d = 2$) dan titik-titik ($d = 3$) menandai batas dua kelompok <i>intent</i>	84
VI.2	Precision@k, Recall@k, dan NDCG@k agregat pada kedalaman $d = 1$ hingga $d = 5$	85

VI.3 Rata-rata <i>RetQ-gain</i> (GraphRAG – <i>Vector RAG</i>) per tipe <i>fixture</i> ($n = 97$). Semua tipe menunjukkan gain positif; variasi mencerminkan tingkat ketergantungan kueri terhadap traversal relasi antar- <i>resource</i>	87
VI.4 <i>Scatter plot</i> jumlah hop yang dilalui vs. <i>RetQ-gain</i> per <i>fixture</i>	88
VI.5 <i>Scatter plot</i> derajat node primer vs. <i>RetQ-gain</i> . Garis menunjukkan rata-rata gain per nilai derajat.	88

DAFTAR TABEL

II.1	Ringkasan kategori relasi antar-objek Kubernetes	12
III.1	Taksonomi 18 jenis <i>edge</i> dalam <i>knowledge graph</i> Kubernetes	33
III.2	Distribusi karakteristik <i>node</i> berdasarkan kedalaman penelusuran graf	34
III.3	Klasifikasi Tingkat Kompleksitas Pertanyaan Pengguna	35
III.4	<i>Technical Gap Analysis</i> pada Domain Kubernetes	36
III.5	Pemetaan Kebutuhan Bisnis terhadap <i>Technical Gap</i> dan Kebutuhan Fungsional	38
III.6	Kebutuhan Fungsional Sistem	38
III.7	Kebutuhan Non-Fungsional Sistem	41
III.8	Pemetaan kebutuhan bisnis terhadap dimensi dan metrik evaluasi . .	42
III.9	Rubrik Penilaian Kriteria Evaluasi Solusi	44
III.10	Matriks Perbandingan Solusi Berdasarkan Rubrik Evaluasi	45
IV.1	Pemetaan Tahap Metodologi	51
IV.2	Lima Fase Pipeline Ingestion Data	57
IV.3	Lima Node LangGraph Agent Pipeline	59
IV.4	Pemetaan <i>Intent Type</i> terhadap Kedalaman Traversal Graf	62
IV.5	Rancangan Tiga Lapis Validasi YAML	63
V.1	Dependensi Utama Implementasi Sistem GraphRAG Kubernetes . .	66
V.2	Statistik <i>Knowledge Graph</i> Kubernetes Hasil Konstruksi	67
VI.1	Profil Narasumber Wawancara Validasi Dataset	74
VI.2	Penilaian Realisme Sampel <i>Fixture</i> oleh Pakar (Skala 1–5)	75
VI.3	Skor Evaluasi per Dimensi dan Sub-Metrik (97 <i>Fixture</i>)	76
VI.4	Skor Evaluasi per Kategori <i>Fixture</i>	77
VI.5	Perbandingan Skor Evaluasi: Vanilla LLM vs. Vector RAG vs. GraphRAG (97 <i>Fixture</i>)	80
VI.6	Hasil <i>ablation study</i> : selisih skor terhadap <i>baseline</i> GraphRAG (v12)	82
VI.7	Hasil uji signifikansi statistik: GraphRAG vs. <i>Vector RAG</i> ($n = 97$)	85
VI.8	Signifikansi statistik <i>ablation study</i> : <i>baseline</i> vs. varian ablasi . . .	86

VI.9 Rata-rata <i>RetQ-gain</i> per tipe <i>fixture</i>	87
VI.10 Korelasi Spearman antara faktor struktural dan <i>RetQ-gain</i>	89
B.1 Ringkasan 10 Fixture Evaluasi Pakar	141
C.1 Nilai Metrik AnsQ per Fixture (97 Fixture)	155
C.2 Nilai Metrik RetQ per Fixture (97 Fixture)	158
C.3 Nilai Metrik ReaQ per Fixture (97 Fixture)	161

DAFTAR PERSAMAAN

DAFTAR ALGORITMA

DAFTAR LISTING

III.1	Injeksi ConfigMap	31
III.2	Injeksi Secret	31
A.1	src/memory/zep_store.py — Manajemen Memori Percakapan . .	99
A.2	src/ingestion/parser.py — Pipeline Ingestion Data (Swagger- GraphBuilder)	102
A.3	src/chatbot/graph_agent.py — LangGraph Agent Pipeline . .	112
A.4	src/chatbot/custom_retriever.py — Mekanisme Retrieval Ber- tingkat	117
A.5	src/validation/yaml_validator.py — Validasi YAML Tiga Lapis	122
A.6	src/graph/queries.py — Query Cypher Neo4j	125
A.7	scripts/evaluate.py — Fungsi Metrik Evaluasi (AnsQ, RetQ, ReaQ)	129
B.1	fixture-deployment-basic.json — Konseptual: Pembaruan Ima- ge Deployment	141
B.2	fixture-service-types.json — Konseptual: Service LoadBa- lancer vs. Ingress	142
B.3	fixture-scale-existing-deployment.json — Lanjutan: Ska- lakan Replika Deployment	143
B.4	fixture-ingress-service-pod.json — Relasional: Alur Tra- ffic Ingress→Service→Pod	144
B.5	fixture-hpa-deployment-pod.json — Relasional: HPA dan Pens- kalaan Otomatis	146
B.6	fixture-cronjob-backup.json — Generasi YAML: CronJob Bac- kup Database	148
B.7	fixture-networkpolicy-deny-all.json — Generasi YAML: Ne- tworkPolicy Deny-All	149
B.8	fixture-statefulset-with-pvc.json — Generasi YAML: Sta- tefulSet dengan PVC	150
B.9	fixture-pi-reject-docker-registry.json — Skenario Nya- ta: Kebijakan Registry Docker	152

B.10 <code>fixture-kubectl-find-pods-with-env.json</code> — Perintah: Pencarian Environment Variable	153
--	-----

DAFTAR SIMBOL

Notasi	Deskripsi	Pemakaian pertama kali
α	Koefisien bobot untuk <i>loss function</i> , tingkat signifikansi statistik	75
β	Koefisien bobot untuk <i>Binary Cross-Entropy loss</i>	75
λ	Koefisien bobot untuk CTC <i>loss</i> , parameter regularisasi	75
θ	Parameter model <i>neural network</i>	75
σ	Deviasi standar	75
μ	Rata-rata (<i>mean</i>)	75
ϵ	<i>Token blank</i> /kosong dalam CTC, konstanta kecil untuk stabilitas numerik	75
Δ	Delta (selisih/perubahan nilai)	75
π	<i>Alignment path</i> dalam CTC	75
π_t	<i>Token</i> pada <i>timestep</i> t dalam <i>alignment path</i>	76
∇ atau ∂	Operator gradien atau turunan parsial	75
$\sum$	Operator penjumlahan	75
$\prod$	Operator perkalian	75
$\int$	Operator integral	75
argmax	Argumen yang memaksimalkan fungsi	76
$\sim$	Terdistribusi menurut	75
$\rightarrow$	Menuju atau konvergen ke	75
$\mathcal{L}$	Fungsi <i>loss</i> (kerugian)	75
$\mathcal{L}_{total}$	Total <i>loss function</i>	75
$\mathcal{L}_{adversarial}$ atau $\mathcal{L}_{adv}$	<i>Adversarial loss</i>	75
$\mathcal{L}_{reconstruction}$	<i>Reconstruction loss</i>	75
$\mathcal{L}_{L1}$	L1 <i>loss</i> (<i>Mean Absolute Error</i>)	75
$\mathcal{L}_{L2}$	L2 <i>loss</i> (<i>Mean Squared Error</i>)	75
$\mathcal{L}_{CTC}$	CTC (<i>Connectionist Temporal Classification</i>) <i>loss</i>	75
$\mathcal{L}_{perceptual}$	<i>Perceptual loss</i>	75

$\mathcal{L}_{pixel}$	<i>Pixel-wise loss</i>	75
$\mathcal{L}_{BCE}$	<i>Binary Cross-Entropy loss</i>	75
$\mathcal{L}_{GAN}$	<i>GAN loss</i>	75
$\mathcal{L}_{cycle}$	<i>Cycle consistency loss</i>	75
G	<i>Generator dalam GAN</i>	75
D	<i>Discriminator dalam GAN</i>	75
$G(z)$	<i>Output dari generator dengan input z</i>	75
$D(x)$	<i>Output dari discriminator dengan input x</i>	75
$G_{A \rightarrow B}$	<i>Generator dari domain A ke B</i>	75
$G_{B \rightarrow A}$	<i>Generator dari domain B ke A</i>	75
$V(D, G)$	<i>Fungsi nilai (value function) dalam GAN</i>	75
x	<i>Sampel data asli, input citra</i>	75
y	<i>Label atau ground truth, kondisi dalam conditional GAN</i>	75
z	<i>Noise vector atau representasi laten</i>	
$\mathbb{E}$	<i>Ekspektasi atau nilai harapan</i>	75
p_{data}	<i>Distribusi data asli</i>	75
p_z	<i>Distribusi prior (noise)</i>	75
$p(y x)$	<i>Probabilitas bersyarat output y given input x</i>	75
$p_t(\pi_t x)$	<i>Probabilitas token pada timestep t</i>	
$\mathcal{B}$	<i>Fungsi penciutan (collapse function) dalam CTC</i>	75
$\mathcal{B}^{-1}$	<i>Fungsi invers penciutan dalam CTC</i>	75
T	<i>Jumlah timestep atau panjang sequence</i>	
$ x - G(y) _1$	<i>L1 distance antara ground truth dan generated image</i>	75
$ x - G(y) _2$	<i>L2 distance antara ground truth dan generated image</i>	75
$N \times N$	<i>Ukuran patch dalam PatchGAN</i>	
d	<i>Cohen's d (effect size)</i>	75
n	<i>Jumlah sampel</i>	75
p	<i>p-value (nilai probabilitas statistik)</i>	75

r	Koefisien korelasi	75
R^2	Koefisien determinasi	
$O(n)$	Notasi Big-O untuk kompleksitas linear	75
$O(n^2)$	Notasi Big-O untuk kompleksitas kuadratik	75

DAFTAR SINGKATAN

Singkatan	Deskripsi	Pemakaian pertama kali
AI	<i>Artificial Intelligence</i>	1
CNN	<i>Convolutional Neural Network</i>	2
GPU	<i>Graphics Processing Unit</i>	14

BAB I

PENDAHULUAN

I.1 Latar Belakang

Transformasi industri teknologi informasi menuju arsitektur *cloud-native* kini menjadi kebutuhan fundamental, bukan sekadar tren. Gartner (2021) memproyeksikan bahwa pada tahun 2025, lebih dari 95% beban kerja digital akan di-*deploy* pada platform *cloud-native*. Angka ini meningkat tajam dari hanya 30% pada tahun 2021. Fondasi utama dari arsitektur *cloud-native* ini adalah penggunaan teknologi kontainer yang memungkinkan aplikasi dikemas dan dijalankan secara efisien. Namun, seiring dengan peningkatan skala sistem, pengelolaan ribuan kontainer secara manual tidak lagi memungkinkan sehingga dibutuhkan sebuah alat otomatisasi. Untuk menjembatani kebutuhan kompleks inilah, Kubernetes (K8s) hadir di pusat ekosistem dan mengukuhkan posisinya sebagai standar industri *de facto* untuk orkestrasi kontainer.

Berdasarkan laporan survei *Cloud Native Computing Foundation* (CNCF) tahun 2023, Kubernetes mendominasi pangsa pasar global karena ekosistemnya yang matang, dukungan komunitas yang luas, serta fleksibilitas dalam mengelola infrastruktur berskala besar secara deklaratif (Cloud Native Computing Foundation 2023). Pada tahun 2022, sekitar 58% pengguna telah menjalankan Kubernetes di lingkungan produksi dan 23% lainnya masih dalam tahap evaluasi (total 81%). Pada tahun 2023, angka tersebut meningkat menjadi 66% pengguna di produksi dengan 18% dalam evaluasi (total 84%). Peningkatan signifikan ini menunjukkan bahwa Kubernetes semakin menjadi fondasi utama dalam pengelolaan layanan *cloud-native* modern.

Meskipun Kubernetes mendominasi lanskap orkestrasi kontainer, platform ini menghadirkan tantangan operasional yang signifikan berupa kompleksitas konfigurasi berbasis berkas YAML. Paradigma deklaratif Kubernetes menuntut pengembang untuk menulis ratusan baris kode yang verbos dan hierarkis demi mendefinisikan *desired state* dari sebuah aplikasi yang mencakup berbagai spesifikasi parameter dari Kubernetes. Kompleksitas ini diperparah oleh interdependensi antar-objek yang

sangat ketat namun sering kali tidak terlihat secara eksplisit (*implicit dependencies*) yang mengakibatkan kesalahan kecil seperti ketidakcocokan *label selector* dapat memicu kegagalan sistem berantai. Akibat beban kognitif yang tinggi ini, kerumitan YAML tidak hanya menjadi hambatan skalabilitas utama bagi 65% penggunanya, tetapi juga menjadi sumber utama miskonfigurasi operasional dan keamanan; tercatat sekitar 80% insiden operasional berakar pada kompleksitas ini, dan 18% berkas konfigurasi *open-source* terbukti mengandung kerentanan keamanan yang serius (Rahman dkk. 2023).

Situasi tersebut berdampak langsung pada pemanfaatan *Large Language Models* (LLM) untuk menghasilkan kode maupun konfigurasi teknis Kubernetes. Walaupun model seperti GPT-4 menawarkan kemudahan dalam memahami dokumentasi API, kemampuan tersebut tidak selalu sejalan dengan ketepatan semantik. Studi evaluatif oleh Liu dkk. (2024) menunjukkan bahwa LLM dapat menghasilkan kode yang secara sintaksis tampak benar, tetapi keliru secara semantis. Dalam lingkungan *production-grade*, kesalahan semantik semacam ini tidak hanya bersifat minor, tetapi berpotensi memicu hilangnya layanan, gangguan orkestrasi, hingga *downtime* berskala besar.

Upaya mitigasi telah dilakukan melalui *Retrieval-Augmented Generation* (RAG) berbasis vektor untuk menambahkan konteks dokumentasi saat LLM menjawab pertanyaan teknis. Namun, penelitian oleh Wan dkk. (2025) mengungkapkan bahwa pendekatan vektor RAG hanya mengutamakan kesamaan *embedding* sehingga gagal menangkap relasi kausal, hierarkis, dan berbasis aturan teknis yang diperlukan untuk memahami konfigurasi domain Kubernetes. Pendekatan vektor RAG hanya mencapai 63,4% *exact match accuracy* dan 69,8% *context precision*, menunjukkan rendahnya ketepatan semantik dalam jawaban sistem. *Embedding* yang dilatih dari korpus umum juga mengabaikan terminologi teknis spesifik sehingga menghasilkan jawaban yang tampak relevan secara teks, tetapi tidak tepat secara domain.

Sebagai solusi, Wan dkk. (2025) mengusulkan penggabungan representasi pengetahuan terstruktur melalui *Knowledge Graph* (KG) untuk memberikan fondasi penalaran relasional yang eksplisit. Integrasi KG dalam sistem RAG menghasilkan peningkatan kinerja yang signifikan, mencapai 77,8% *exact match accuracy* dan 76,5% *context precision*, yaitu peningkatan lebih dari 14,4 poin persentase dibanding pendekatan vektor RAG. KG tidak hanya meningkatkan presisi pencarian informasi, tetapi juga memungkinkan *semantic alignment* melalui pembatasan ontologis sehingga *retrieval* tidak bergantung semata pada kemiripan *embedding*, melainkan

pada struktur relasional domain.

Dengan demikian, pendekatan Graph-RAG menawarkan fondasi penalaran teknis yang lebih dapat dipertanggungjawabkan dan mampu menurunkan risiko halusinasi semantik berbasis domain, khususnya pada sistem *cloud-native* yang sensitif terhadap kesalahan konfigurasi. Berdasarkan urgensi tersebut, penelitian ini bertujuan untuk membangun sistem *Retrieval-Augmented Generation* (RAG) berbasis *Knowledge Graph* yang dikhususkan untuk dokumentasi Kubernetes. Pendekatan ini diharapkan mampu meningkatkan akurasi *retrieval* terhadap pertanyaan yang membutuhkan penalaran struktural dengan mengekspresikan relasi ontologis antar objek konfigurasi Kubernetes. Selain menghasilkan jawaban teknis berupa penjelasan dan konfigurasi YAML yang valid, sistem juga dirancang untuk memberikan jejak penalaran graf sebagai mekanisme pendeteksi halusinasi. Hal ini memungkinkan setiap keluaran tidak hanya relevan secara tekstual, tetapi juga dapat diverifikasi kesesuaiannya terhadap skema objek Kubernetes.

Penelitian ini mengajukan tiga kontribusi teknis. **Pertama**, *pipeline* ekstraksi *knowledge graph* yang bersifat deterministik berbasis skema OpenAPI (*schema-derived deterministic KG construction*). Graf dibangun langsung dari `swagger.json` melalui aturan transformasi yang eksplisit: setiap tipe pada blok definitions dipetakan menjadi *node*, dan setiap referensi antar-tipe (`$ref`) dipetakan menjadi *edge*. Karena seluruh aturan transformasi bersifat tetap, hasil ekstraksi konsisten antar-eksekusi, berbeda dengan pendekatan ekstraksi berbasis LLM yang menghasilkan struktur graf bervariasi karena bersifat stokastik (Pan dkk. 2024; Wan dkk. 2025). **Kedua**, mekanisme *traversal* dengan kedalaman yang disesuaikan per tipe *intent* (*intent-adaptive depth traversal*): alih-alih menggunakan kedalaman tetap yang seragam, sistem menetapkan kedalaman berdasarkan karakteristik struktural tiap jenis kueri, yang terbukti secara empiris menghasilkan *retrieval quality* lebih baik dibandingkan kedalaman tetap. **Ketiga**, validasi struktural YAML tiga lapis berbasis *knowledge graph* (*KG-grounded structural validation*) yang memverifikasi *required fields* langsung dari representasi graf. Mekanisme ini berfungsi sebagai alternatif validasi sintaksis terhadap skema OpenAPI tanpa memerlukan *dry-run* pada kluster Kubernetes aktif.

1.2 Rumusan Masalah

Penelitian ini bertujuan untuk menjawab pertanyaan-pertanyaan berikut,

1. Bagaimana membangun *knowledge graph* dari spesifikasi Kubernetes guna merepresentasikan struktur dan relasi antar-objek konfigurasi secara kompre-

hensif?

2. Bagaimana merancang mekanisme GraphRAG yang memanfaatkan representasi struktural dalam *knowledge graph* untuk meningkatkan kualitas *retrieval* pada konfigurasi Kubernetes?
3. Bagaimana merancang mekanisme validasi struktural YAML berbasis *knowledge graph* yang mampu memverifikasi *required fields* secara langsung dari representasi graf tanpa memerlukan *dry-run* pada kluster Kubernetes aktif?

I.3 Tujuan

Berdasarkan rumusan masalah yang telah dijelaskan pada Bab I.2, tujuan dalam penulisan penelitian ini adalah sebagai berikut,

1. Membangun *knowledge graph* dari spesifikasi OpenAPI Kubernetes (swagger.json) melalui *pipeline* ekstraksi guna memetakan objek Kubernetes beserta relasi hierarkis dan referensial antar-objek Kubernetes.
2. Mengembangkan mekanisme GraphRAG dengan menelusuri jalur relasional (*graph traversal*) berdasarkan kueri pengguna, kemudian menginjeksikan *reasoning path* tersebut ke dalam *Large Language Model* (LLM) untuk menghasilkan jawaban konsisten sesuai logika domain Kubernetes.
3. Mengimplementasikan mekanisme validasi struktural YAML tiga lapis berbasis *knowledge graph* yang memverifikasi *required fields* langsung dari representasi graf, menyediakan alternatif validasi sintaksis terhadap skema OpenAPI tanpa memerlukan *dry-run* pada kluster Kubernetes aktif.

I.4 Batasan Masalah

Agar penelitian ini tetap fokus dan *feasible* dalam lingkup Tugas Akhir, batasan-batasan berikut akan diterapkan,

1. Data penelitian bersumber secara eksklusif dari spesifikasi resmi Kubernetes OpenAPI (swagger.json) versi v1.30 tanpa melakukan *web scraping* dokumentasi HTML. Ekstraksi data dibatasi murni pada blok *definitions* dengan mengeliminasi blok operasional protokol API (seperti *paths*, *parameters*, dan *responses*) guna mencegah *context noise* pada LLM.
2. *Knowledge graph* yang dibangun hanya merepresentasikan struktur skema (*schema structure*) dan tidak mencakup perilaku operasional (*runtime behavior*) maupun logika validasi dengan fokus utama penelitian untuk menguji validitas struktur sintaksis YAML.
3. Luaran penelitian berupa manifes Kubernetes YAML murni (*plain YAML*) berdasarkan *OpenAPI Specification*, dilengkapi jejak penalaran graf (*graph reasoning path*) guna mendeteksi halusinasi LLM.

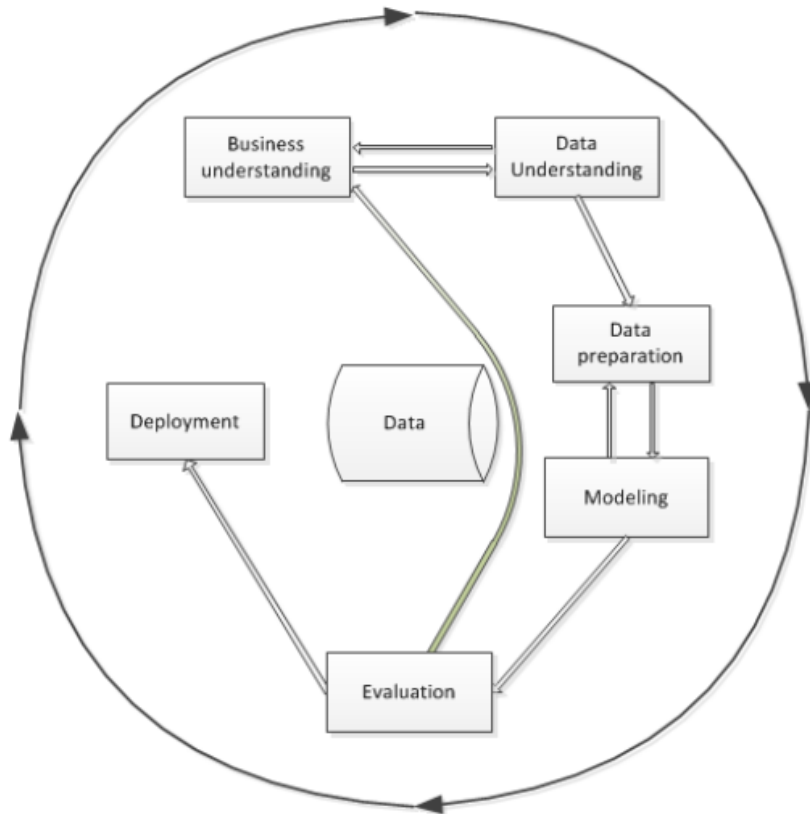
4. Luaran file YAML tidak diuji langsung pada kluster Kubernetes nyata (*actual cluster*). Pengujian sistem murni berfokus pada validitas sintaksis menggunakan pustaka *pyyaml* serta evaluasi kinerja berdasarkan dataset uji.
5. Model LLM yang digunakan berasal dari *pre-trained models* yang diakses melalui API, yaitu **GPT-4o-mini** sebagai *Thinker* dan **GPT-4o-mini** sebagai *Speaker*, tanpa proses *fine-tuning*.
6. Kedalaman *graph traversal* ditentukan secara adaptif berdasarkan tipe *intent* kueri: kedalaman 2 untuk *intent explain* dan *follow-up* (pertanyaan konseptual yang cukup diselesaikan pada lapisan spesifikasi langsung), serta kedalaman 3 untuk *intent* lainnya (*generate_yaml*, *trace_relationship*, *planning*). Batas atas kedalaman 3 ditetapkan berdasarkan analisis distribusi kedalaman skema OpenAPI Kubernetes: kedalaman 1–2 menghasilkan *node* spesifik yang hanya dibagikan oleh 2–3 *resource*; kedalaman 3 menjangkau *PodSpec*; sedangkan kedalaman 4 ke atas didominasi tipe utilitas generik (*Quantity*, *IntOrString*, *LocalObjectReference*) yang dibagikan oleh 19–136 *resource* sehingga menurunkan presisi konteks.
7. Proses pengolahan dan pra-pemrosesan data dibatasi pada lingkungan **Google Colaboratory** dan **Jupyter Notebook**.
8. Kinerja komputasi dan hasil eksperimen didasarkan pada perangkat keras spesifik (PC dengan CPU AMD Ryzen 5, GPU AMD Radeon RX 6750XT, RAM 16 GB) dan mungkin bervariasi pada konfigurasi lain.

I.5 Metodologi

Pelaksanaan Tugas Akhir ini mengadopsi kerangka kerja *Cross-Industry Standard Process for Data Mining* (CRISP-DM) yang dapat dilihat pada Gambar I.1 sebagai metodologi utama. CRISP-DM dipilih karena memberikan proses terstruktur, iteratif, dan dapat diadaptasi untuk pengembangan sistem berbasis analisis data dan representasi pengetahuan (Niakšu 2015). Tahapan metodologi yang digunakan terdiri dari enam fase berikut,

1. Pemahaman Bisnis (*Business Understanding*)

Tahap ini bertujuan untuk mengidentifikasi kebutuhan serta masalah inti yang ingin diselesaikan melalui pemanfaatan data. Dalam konteks penelitian ini, permasalahan yang diangkat adalah ketidaktepatan jawaban teknis yang dihasilkan *Large Language Models* (LLM) ketika merujuk dokumentasi Kubernetes, terutama pada konfigurasi YAML yang bersifat struktural dan rentan mengandung halusinasi. Oleh karena itu, kebutuhan penelitian diarahkan pada peningkatan akurasi penalaran struktural pada proses *retrieval* serta mitigasi halusinasi konfigurasi pada sistem *cloud-native*.



Gambar I.1 Tahapan model referensi CRISP-DM (Niakšu 2015)

2. Pemahaman Data (*Data Understanding*)

Fase ini berfokus pada pengumpulan dan eksplorasi sumber data untuk memahami struktur, kualitas, serta batasannya. Pada penelitian ini, data diperoleh dari spesifikasi resmi Kubernetes OpenAPI yang dianalisis untuk memahami jenis objek Kubernetes, atribut yang dimiliki, serta relasi hierarkis dan referensial antar-objek Kubernetes. Pemahaman ini menentukan objek Kubernetes yang akan dimodelkan dalam *knowledge graph*.

3. Persiapan Data (*Data Preparation*)

Tahapan ini mencakup pembersihan dan transformasi data agar siap digunakan dalam proses pemodelan. Dalam penelitian ini, dilakukan ekstraksi berbasis *script* dari spesifikasi OpenAPI Kubernetes (JSON) untuk membentuk representasi entitas, yaitu relasi yang dapat dimodelkan sebagai *Knowledge Graph*. Proses ini mencakup identifikasi objek, pemetaan hubungan antar-objek, serta penghubungan *cross-resource references*. Hasil akhirnya berupa struktur graf yang siap digunakan pada mekanisme *retrieval*.

4. Pemodelan (*Modeling*)

Fase ini bertujuan untuk menerapkan algoritma yang sesuai guna menyelesaikan masalah yang telah dirumuskan. Pada penelitian ini, dibangun mekanisme

graph-guided retrieval yang menelusuri graf menggunakan *graph traversal* berdasarkan kueri pengguna dan menghasilkan *reasoning path*. Jalur penalaran tersebut kemudian diinjeksikan sebagai konteks pada proses RAG untuk memandu LLM dalam menghasilkan jawaban serta konfigurasi YAML yang sesuai dengan logika sistem Kubernetes.

5. **Evaluasi (*Evaluation*)**

Tahap evaluasi bertujuan menilai kualitas model dan membandingkannya dengan pendekatan alternatif menggunakan metrik yang terukur. Dalam penelitian ini, kinerja sistem dievaluasi menggunakan metrik *answer quality*, *retrieval quality*, dan *reasoning quality*, kemudian dibandingkan dengan dua *baseline*, yaitu *Vanilla LLM* dan *Vector-based RAG*.

6. **Penyebaran (*Deployment*)**

Fase ini berkaitan dengan penyajian hasil implementasi dalam bentuk prototipe dan dokumentasi. Pada penelitian ini, penyebaran diwujudkan dalam bentuk antarmuka web interaktif berbasis Streamlit yang menampilkan jawaban, konfigurasi YAML, serta *reasoning path* sebagai bukti penalaran graf. Selain itu, dokumentasi lengkap disusun sebagai bentuk penyebaran akademik yang memuat rancangan sistem, hasil evaluasi, serta rekomendasi penelitian selanjutnya.

BAB II

STUDI LITERATUR

II.1 Arsitektur *Cloud* Modern dan Tantangan Kompleksitas

Pergeseran paradigma dari arsitektur monolitik menuju layanan mikro (*microservices*) kini telah menjadi fondasi utama dalam pengembangan aplikasi *cloud-native* modern. Berbeda dengan pendekatan arsitektur tradisional, gaya arsitektur layanan mikro mendistribusikan logika bisnis secara masif ke dalam puluhan hingga ratusan layanan kecil independen. Distribusi fungsionalitas ini memunculkan kebutuhan akan mekanisme penerapan (*deployment*) mandiri. Sebagai solusi, layanan mikro secara alami terintegrasi dengan platform berbasis kontainer karena setiap layanan dapat dikemas dan didistribusikan di dalam kontainernya masing-masing (Soldani, Tamburri, dan Van Den Heuvel 2018). Walaupun memberikan keuntungan operasional yang signifikan berupa portabilitas lintas *cloud*, skalabilitas tingkat tinggi, dan kebebasan pemilihan teknologi bagi pengembang, ekosistem kontainer ini memunculkan paradoks kompleksitas baru (Soldani, Tamburri, dan Van Den Heuvel 2018).

Dalam lingkungan terdistribusi ini, setiap layanan berkomunikasi melalui antarmuka API yang bertindak sebagai kontrak komunikasi esensial antar-layanan (Soldani, Tamburri, dan Van Den Heuvel 2018). Pemahaman kognitif yang keliru terhadap arsitektur dan kontrak API ini sering kali berujung pada kegagalan operasional; kegagalan satu layanan dapat memicu rentetan kegagalan layanan lain yang bergantung padanya (*cascading failures*) (Soldani, Tamburri, dan Van Den Heuvel 2018). Akibatnya, dokumentasi API dan manifes konfigurasi orkestrasi mengalami lonjakan kompleksitas yang menjadi tantangan utama pengembang *cloud-native*.

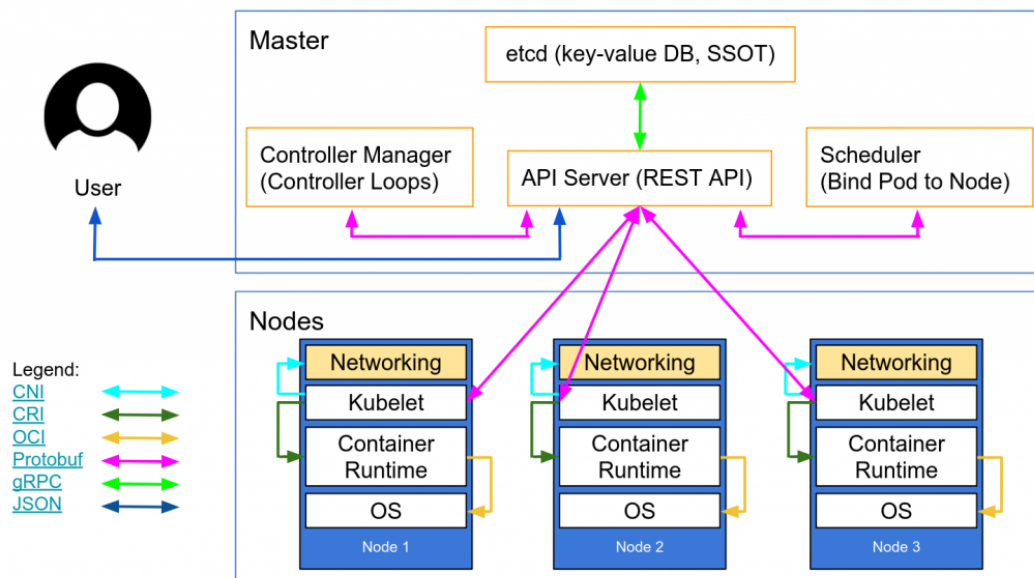
II.2 Kubernetes: Platform Orkestrasi Kontainer untuk Aplikasi *Cloud-Native*

Kubernetes merupakan platform orkestrasi kontainer yang dirancang untuk mengotomatisasi proses *deployment*, *scaling*, dan manajemen aplikasi berbasis kontainer dalam lingkungan *cloud-native* (The Kubernetes Authors 2024b). Platform ini menyediakan mekanisme deklaratif berbasis konfigurasi yang memungkinkan pengguna mendefinisikan keadaan sistem yang diinginkan (*desired state*) melalui berkas

konfigurasi, sementara Kubernetes menjaga agar keadaan tersebut tetap terpenuhi melalui rangkaian komponen pengendali (*controller loops*).

II.2.1 Gambaran Umum Arsitektur

Arsitektur Kubernetes dibangun atas konsep klaster yang terdiri dari dua kelompok komponen utama, yaitu *Control Plane* dan *Worker Nodes*. *Control Plane* berfungsi sebagai pengendali utama yang membuat keputusan bisnis aplikasi, misalnya penjadwalan, pemantauan, serta konsistensi status. *Worker Nodes* merupakan entitas komputasi fisik/virtual yang menjalankan kontainer sebagai satuan aplikasi. Kedua komponen ini berkomunikasi secara internal melalui API server yang menjadi pintu utama seluruh permintaan sistem.



Gambar II.1 Arsitektur klaster Kubernetes yang terdiri atas *Control Plane* dan *Worker Nodes*, dihubungkan melalui *kube-apiserver* (The Kubernetes Authors 2024b)

1. *Control Plane*

Control Plane bertanggung jawab mengelola keadaan (*state*) klaster secara keseluruhan. Komponen utamanya meliputi,

- kube-apiserver** sebagai gerbang akses seluruh operasi manipulasi objek konfigurasi melalui API,
- etcd** sebagai basis data *key-value* yang menyimpan informasi konfigurasi dan status,
- kube-scheduler** yang menentukan Node penempatan Pod berdasarkan ketersediaan sumber daya,

- d. **kube-controller-manager** yang berisi sekumpulan kontroler untuk menjaga kesesuaian antara kondisi aktual dan *desired state*. Melalui mekanisme deklaratif ini, Kubernetes secara otomatis memperbaiki status objek yang tidak sesuai dan memastikan setiap aplikasi berjalan sesuai aturan konfigurasi yang ditentukan.

2. *Worker Nodes*

Worker Nodes merupakan lapisan eksekusi tempat aplikasi berjalan dalam bentuk Pod. Node terdiri atas tiga komponen inti:

- a. **kubelet** yang bertugas mengeksekusi Pod sesuai *specification* yang diterima dari *Control Plane*,
- b. **kube-proxy** yang menyediakan layanan *network forwarding* dan *load balancing* sederhana antar Pod dalam klaster,
- c. **container runtime** yang bertanggung jawab menjalankan kontainer. Dengan demikian, Node menjalankan komputasi terdistribusi yang dikontrol oleh *Control Plane* melalui komunikasi berbasis API.

II.2.2 *Resource Model dan Hierarki*

Dalam *resource model* Kubernetes, terdapat beberapa objek inti pembangun aplikasi yang paling sering digunakan:

- a. **Pod**: Unit komputasi terkecil sebagai pembungkus kontainer aplikasi
- b. **Deployment**: Pengatur replika dan proses pembaruan versi untuk mencapai ketersediaan tinggi
- c. **Service**: Penyedia titik akses jaringan yang stabil untuk aplikasi di dalam *Pod*
- d. **Ingress**: Pengatur rute lalu lintas eksternal masuk ke dalam klaster
- e. **ConfigMap**: Penyimpan variabel lingkungan dan konfigurasi non-sensitif
- f. **Secret**: Pengelola data sensitif seperti kredensial dan token
- g. **PersistentVolumeClaim**: Permintaan penyimpanan persisten untuk data yang harus bertahan
- h. **ServiceAccount**: Identitas yang digunakan oleh *Pod* untuk berinteraksi dengan API server
- i. **Role** dan **RoleBinding**: Kontrol akses berbasis peran (RBAC)
- j. **HorizontalPodAutoscaler**: Pengatur skala otomatis berdasarkan metrik penggunaan

Antar-objek ini saling terikat melalui berbagai jenis relasi semantik yang dapat dikategorikan ke dalam 7 kelompok. Ringkasan kategori relasi ditampilkan pada Tabel II.1, sedangkan desain taksonomi *edge* secara rinci disajikan pada Bab III sebagai bagian dari kontribusi penelitian.

Tabel II.1 Ringkasan kategori relasi antar-objek Kubernetes

Kategori	Jumlah <i>Edge</i>	Contoh <i>Edge</i>
Struktural	4	<i>HAS_PROPERTY, EXTENDS</i>
<i>Workload</i>	3	<i>CONTAINS_POD_TEMPLATE</i>
Penyimpanan	3	<i>CLAIMS_VOLUME, MOUNTS_VOLUME</i>
Konfigurasi	2	<i>LOADS_CONFIGMAP, USES_SECRET</i>
Jaringan	2	<i>SELECTS_POD, ROUTES_TO_SERVICE</i>
RBAC	3	<i>BINDS_ROLE, USES_SERVICE_ACCOUNT</i>
<i>Autoscaling</i>	1	<i>SCALES_RESOURCE</i>
Total	18	

Pemetaan relasi kepemilikan (*ownership*) dan ketergantungan fungsional (*dependencies*) inilah yang akan secara langsung dikonversi menjadi skema graf pengetahuan (*knowledge graph*) pada penelitian ini. Sebagai contoh, sebuah *Deployment* secara hierarkis memiliki sub-sumber daya *PodSpec* melalui relasi *CONTAINS_POD_TEMPLATE*, dan *Pod* tersebut memiliki ketergantungan operasional terhadap *ConfigMap* tertentu melalui relasi *LOADS_CONFIGMAP*. Total terdapat 18 jenis *edge* yang terdistribusi dalam 7 kategori relasi sebagaimana dirangkum pada Tabel II.1. Desain taksonomi *edge* secara rinci dibahas pada Bab III.

II.2.3 Manifes YAML dan *Infrastructure as Code*

Pengembang berinteraksi dengan model sumber daya Kubernetes melalui pendekatan *Infrastructure as Code* (IaC) (The Kubernetes Authors 2024a). Pendekatan ini mewajibkan pengguna untuk mendeklarasikan wujud infrastruktur yang diinginkan (*desired state*) ke dalam bentuk dokumen teks berformat YAML. Setiap manifes YAML standar wajib memiliki empat atribut struktural utama agar dapat diterima oleh API Kubernetes:

1. *apiVersion*: Menentukan versi skema API pembungkus objek.
2. *kind*: Mendefinisikan jenis objek Kubernetes.
3. *metadata*: Memuat data identifikasi unik seperti nama, label pengelompokan,

dan *namespace*.

4. *spec*: Berisi spesifikasi teknis rinci penentu sifat dan bentuk hierarki objek tersebut.

Penyusunan manifes YAML menuntut pemahaman terkait batasan skema. Pengembang harus bisa membedakan secara presisi antara ruas wajib (*required*) penentu validitas dan ruas opsional (*optional*) penambah fitur khusus. Praktik terbaik (*best practices*) dalam penulisan IaC menyarankan modularitas dokumen, konsistensi penggunaan label, dan pemisahan logika aplikasi dari data konfigurasi. Sebuah struktur manifes YAML yang mematuhi kelengkapan ruas wajib (*field requirements*) dan valid secara sintaksis inilah yang menjadi representasi target luaran yang akan dibangkitkan oleh sistem generasi pada penelitian ini.

II.3 *Large Language Models (LLM)*

II.3.1 Definisi dan Kapabilitas

Large Language Model (LLM) merupakan model kecerdasan buatan berbasis jaringan saraf berskala besar yang dilatih pada korpus teks *multi-domain* untuk mempelajari representasi bahasa alami secara mendalam (Wan dkk. 2025). Melalui proses pelatihan tersebut, LLM mampu menginterpretasikan konteks linguistik yang kompleks serta menghasilkan keluaran berupa teks yang koheren, kontekstual, dan adaptif terhadap berbagai jenis tugas seperti *question answering*, penalaran, serta peringkasan teks.

Meskipun unggul dalam pemahaman bahasa alami, LLM memiliki keterbatasan faktual yang signifikan ketika diterapkan pada domain teknis, terutama terhadap pengetahuan yang presisi, bersifat prosedural, serta memerlukan landasan ontologis. Beberapa keterbatasan yang disorot oleh Wan dkk. (2025) meliputi,

- a. *Hallucination*, yaitu kemampuan menghasilkan jawaban yang secara linguistik meyakinkan tetapi tidak memiliki landasan fakta yang jelas sehingga berisiko menyesatkan proses pengambilan keputusan teknis.
- b. Keterbatasan cakupan pengetahuan, LLM hanya mengandalkan data pelatihan awal sehingga tidak selalu mencakup informasi domain yang mutakhir.
- c. Ketidadaan atribusi sumber, yaitu LLM tidak dapat memberikan justifikasi atau referensi eksplisit terhadap jawaban yang dihasilkan.

Keterbatasan tersebut menjadi alasan utama perlunya integrasi antara LLM dan arsitektur *retrieval-augmented generation (RAG)*.

II.3.2 *Prompt Engineering*

Prompt engineering merupakan teknik perancangan instruksi masukan bagi LLM untuk mengendalikan keluaran model berdasarkan tujuan tugas, cakupan konteks, serta batasan informasi yang diizinkan (Wan dkk. 2025). Dalam domain teknis, teknik ini tidak hanya berperan sebagai instruksi linguistik, tetapi juga sebagai representasi pengetahuan eksplisit yang mendefinisikan bagaimana LLM harus memanfaatkan sumber informasi domain.

Efektivitas keluaran LLM dipengaruhi secara signifikan oleh strategi *prompt engineering* berikut,

- a. Spesifikasi tugas yang eksplisit, termasuk definisi peran model dan batasan prosedural, misalnya instruksi untuk membatasi jawaban pada proses manufaktur tertentu.
- b. Penyediaan konteks terstruktur, melalui penyisipan entitas domain, hubungan semantik, serta cuplikan dokumen terkait yang relevan dengan pertanyaan.
- c. Pemberian batasan (*constraint injection*) yang secara eksplisit membatasi ruang solusi berdasarkan pengetahuan terbatas dalam dokumen yang valid.
- d. Penyusunan format jawaban, misalnya dalam bentuk tabel, daftar berhierarki, atau penjelasan berbasis pembuktian prosedural.

Strategi tersebut tidak hanya mengontrol keluaran, tetapi juga mencegah LLM menafsirkan konteks di luar pengetahuan domain sehingga berperan sebagai mekanisme mitigasi terhadap fenomena *hallucination*.

II.4 *Retrieval-Augmented Generation (RAG)*

Retrieval-Augmented Generation (RAG) merupakan arsitektur pemrosesan bahasa alami yang menggabungkan mekanisme *Information Retrieval (IR)* dengan kemampuan generatif *Large Language Models (LLM)*. Pendekatan ini dikembangkan untuk mengatasi keterbatasan memori parametris pada model bahasa, khususnya dalam menangani tugas yang bersifat *knowledge-intensive*, yaitu model rentan menghasilkan informasi yang keliru atau tidak berbasis fakta (*hallucination*) (Antonio Moreno-Cediel 2025).

Berbeda dengan metode generatif konvensional yang hanya mengandalkan pengetahuan tersimpan selama proses pelatihan, RAG memanfaatkan basis pengetahuan non-parametris melalui proses pencarian (*retrieval*) sehingga jawaban yang dihasilkan lebih akurat dan relevan terhadap konteks masukan.

II.4.1 Arsitektur *Retrieval-Augmented Generation*

Arsitektur *Retrieval-Augmented Generation* (RAG) merupakan pendekatan hibrida yang menggabungkan kemampuan penalaran *Large Language Models* (LLM) dengan basis pengetahuan eksternal melalui proses pencarian semantik. Struktur ini dirancang untuk mengurangi *hallucination*, meningkatkan akurasi fakta, serta mendukung pembaruan pengetahuan secara dinamis (Gao dkk. 2024). Secara umum, alur kerja RAG terdiri atas empat tahapan teknis utama yang berurutan dan saling bergantung, yaitu *Data Ingestion & Indexing*, *Retrieval*, *Augmentation*, dan *Generation*.

1. *Data Ingestion & Indexing*

Tahapan ini bersifat *offline* dan mencakup lima langkah berurutan: (a) *akuisisi data* dari sumber dokumentasi teknis atau spesifikasi API, (b) *preprocessing* berupa normalisasi format dan ekstraksi metadata, (c) *chunking* untuk memecah dokumen menjadi unit 200–500 token, (d) *embedding* menggunakan model seperti MiniLM atau BERT untuk mengubah setiap *chunk* menjadi representasi vektor, serta (e) *indexing* ke *vector store* (misalnya FAISS atau Chroma) agar pencarian berbasis kedekatan semantik dapat dilakukan secara efisien.

2. *Retrieval*

Proses *retrieval* dilakukan ketika pengguna mengajukan kueri. Sistem akan mencari konteks paling relevan melalui mekanisme berikut,

a. *Embedding Kueri*

Kueri pengguna diubah menjadi vektor menggunakan model *embedding* yang sama dengan proses *indexing*.

b. *Similarity Search*

Dilakukan pencocokan vektor menggunakan metrik jarak seperti *cosine similarity*, *dot-product*, atau *Euclidean distance*.

c. *Top-k Selection*

Sistem memilih sejumlah dokumen teratas (umumnya $k = 3$ atau $k = 5$) yang memiliki skor kemiripan tertinggi dan mengandung konteks yang berpotensi menjawab pertanyaan.

Kualitas *retrieval* yang baik menjadi faktor kunci dalam menurunkan risiko *hallucination* pada LLM karena jawaban dibatasi oleh fakta yang relevan.

3. *Augmentation*

Tahap *augmentation* bertugas menyisipkan konteks hasil *retrieval* ke dalam *prompt* secara sistematis sehingga dapat dipahami oleh LLM. Prosesnya men-

cakup tahapan berikut,

a. *Concatenation*

Sistem menggabungkan pertanyaan pengguna dengan konten dokumen yang ditemukan.

b. *Formatting*

Struktur informasi dapat diberikan dalam format teks baku, tabel, *JSON schema*, *instruction-style*, atau bentuk terstruktur lainnya.

c. *Constraint Injection*

Sistem memberikan batasan eksplisit, misalnya “*jawab hanya menggunakan konteks yang diberikan, jangan menambah informasi baru*”, untuk mencegah LLM berhalusinasi.

II.4.2 Paradigma RAG

Retrieval-Augmented Generation berkembang dalam tiga paradigma utama, yaitu *Naive RAG*, *Advanced RAG*, dan *Modular RAG*. Ketiga paradigma ini mencerminkan evolusi dari *pipeline* sederhana menuju arsitektur yang dapat diadaptasi untuk berbagai kebutuhan, mulai dari peningkatan akurasi sampai fleksibilitas sistem (Gao dkk. 2024).

1. *Naive RAG*

Paradigma ini merupakan bentuk paling dasar dari RAG. *Pipeline* umumnya terdiri dari tiga langkah, yaitu *indexing*, *retrieval*, dan *generation*. Dokumen melalui proses *chunking*, kemudian direpresentasikan menggunakan *embedding* dan disimpan pada *vector store*. Ketika pertanyaan diajukan, sistem mengambil k dokumen paling relevan menggunakan pencarian berbasis kesamaan semantik, lalu model generatif menyusun jawaban berdasarkan konteks tersebut. Keterbatasan utama pendekatan ini adalah ketergantungan penuh terhadap kualitas *chunk* dan hasil *retrieval* yang dapat mengarah pada jawaban yang tidak akurat.

2. *Advanced RAG*

Paradigma ini melakukan optimasi pada dua fase utama, yaitu *pre-retrieval* dan *post-retrieval*. Tahap *pre-retrieval* meliputi peningkatan kualitas indeks seperti pengayaan metadata, peningkatan granularitas *chunk*, serta teknik *query rewriting* dan *query expansion* untuk meningkatkan relevansi hasil pencarian. Sementara itu, *post-retrieval* dilakukan melalui *reranking* serta *context compression* agar konteks yang terkirim ke model lebih berfokus pada informasi penting saja. Paradigma ini masih mengikuti alur linear tetapi menghasilkan akurasi yang lebih tinggi dibanding *Naive RAG*.

3. *Modular RAG*

Paradigma ini memperluas arsitektur RAG menjadi sistem yang dapat dikustomisasi melalui penambahan atau penggantian modul seperti *Search*, *Memory*, *Routing*, dan *Task Adapter*. Modular RAG juga memungkinkan pola *iterative* dan *adaptive retrieval* sehingga menjadikannya paradigma paling fleksibel untuk aplikasi domain spesifik.

II.4.3 Relevansi RAG terhadap Dokumentasi Kubernetes API

Dokumentasi Web API, termasuk Kubernetes, memiliki karakteristik unik berupa kombinasi deskripsi sintaks terstruktur (misalnya skema JSON/YAML) dan penjelasan semantik dalam bentuk teks alami. Penelitian Kotstein dan Decker (2024) menunjukkan bahwa pemrosesan semantik terhadap dokumentasi API dapat dilakukan tanpa ontologi formal dengan memetakan makna berdasarkan nama parameter, struktur hierarkis, dan deskripsi bahasa alami.

Dalam konteks Kubernetes, komponen API seperti *Pod*, *Service*, dan *Deployment* direpresentasikan melalui skema bertingkat yang bersifat kompleks dan memiliki ketergantungan semantik. Oleh karena itu, implementasi RAG pada domain ini membutuhkan,

1. ***Schema-aware semantic chunking***

Teknik ini memastikan bahwa pemotongan dokumen (*chunking*) dilakukan mengikuti struktur skema objek API. Setiap potongan teks harus mewakili satu entitas atau blok atribut yang utuh sehingga relasi antar-*field* tidak terpisah atau kehilangan konteks. Dengan demikian, representasi vektor yang dihasilkan tetap mempertahankan hierarki objek.

2. ***Path-based serialization***

Teknik ini menyimpan setiap elemen API beserta jalur lengkapnya dalam struktur objek sehingga posisi semantiknya tetap terpelihara.

3. ***Hybrid index retrieval***

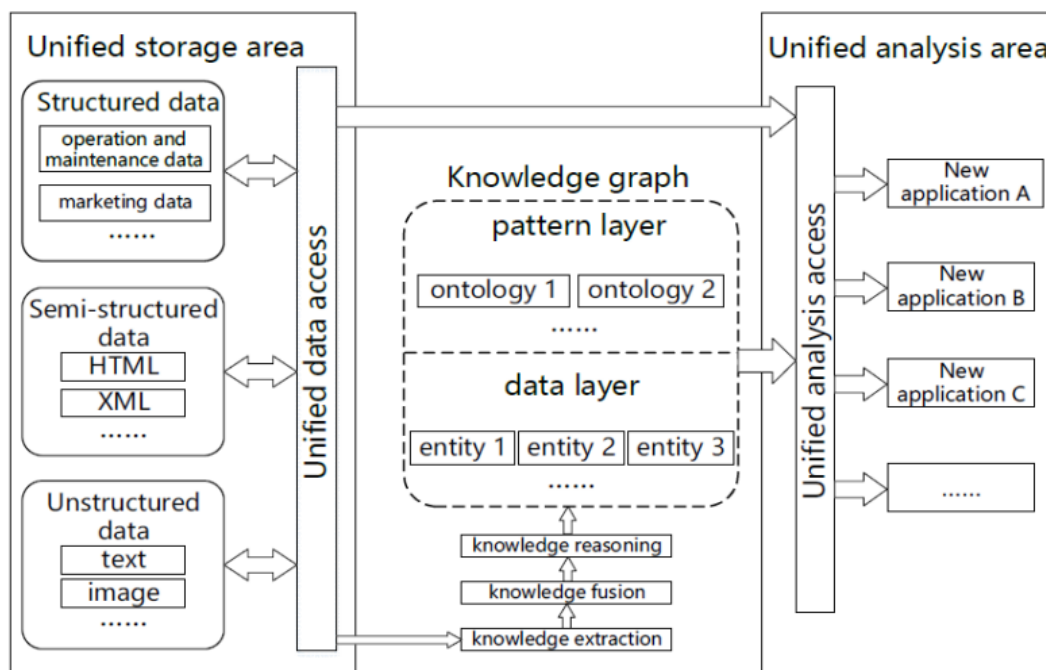
Pendekatan ini mengombinasikan pencarian berbasis kesamaan semantik teks dengan pembobotan struktur sintaksis dari objek API. Selain memanfaatkan *embedding* untuk menemukan kemiripan makna antar-teks, sistem juga memberikan bobot lebih tinggi pada elemen struktural kunci (misalnya *field* wajib atau relasi antar-objek). Hasilnya, mekanisme *retrieval* menghasilkan konteks yang tidak hanya relevan secara linguistik, tetapi juga benar secara hierarki dan fungsi API.

Dengan demikian, penerapan RAG pada dokumentasi Kubernetes tidak hanya berfungsi sebagai mesin pencarian semantik, tetapi juga menjadi pendekatan mitigasi *hallucination*.

II.5 Knowledge Graph dan GraphRAG

II.5.1 Definisi dan Struktur

Knowledge Graph (KG) merupakan representasi pengetahuan dalam bentuk graf yang tersusun dari entitas (*node*) dan relasi (*edge*) yang memiliki makna semantik (Hofer dkk. 2024). Berbeda dengan basis data konvensional yang mengandalkan skema statis, KG bersifat *schema-flexible* sehingga mampu mengintegrasikan data heterogen dari sumber terstruktur, semi-terstruktur, maupun tidak terstruktur seperti yang ditunjukkan pada Gambar II.2. Struktur KG umumnya dinyatakan melalui *triples* berbentuk (subjek–predikat–objek).



Gambar II.2 Arsitektur *Knowledge Graph* (J. Yu dkk. 2021)

Menurut J. Yu dkk. (2021), terdapat dua pendekatan utama dalam pembangunan KG, yaitu *top-down* dan *bottom-up* yang dibedakan berdasarkan urutan perancangan model dan pengisian fakta.

1. *Top-down Construction*

Pada pendekatan ini, proses pembangunan KG dimulai dari definisi model semantik terlebih dahulu. Langkah pertama mencakup perancangan *ontology library* yang memuat kelas entitas, relasi, hierarki, dan aturan semantik. Setelah struktur konseptual ditetapkan, barulah fakta atau data dimasukkan ke dalam basis pengetahuan. Dengan demikian, pendekatan *top-down* mengikuti urutan *model layer* kemudian *data layer*. Pendekatan ini umumnya digunakan pada domain yang membutuhkan konsistensi semantik tinggi seperti medis,

keuangan, atau layanan publik.

2. ***Bottom-up Construction***

Berbeda dari pendekatan sebelumnya, *bottom-up* mengawali proses konstruksi melalui ekstraksi data langsung dari korpus teks atau sumber data lain. Dengan menganalisis frekuensi entitas, dan pola semantik kemudian dibentuk secara bertahap. Artinya, *data layer* dibangun terlebih dahulu, kemudian dilanjutkan dengan penyusunan *pattern layer*. Pendekatan ini didorong oleh ketersediaan data dan sesuai untuk domain yang cepat berubah atau belum memiliki referensi ontologi.

Menurut Hofer dkk. (2024), konstruksi *knowledge graph* (KG) modern umumnya dilakukan secara semi-otomatis dan bersifat inkremental. Proses pembangunan KG mencakup sejumlah tugas inti yang dapat dijalankan secara paralel, asinkron, maupun selektif sesuai jenis sumber data dan kebutuhan penggunaan. Tugas-tugas utama tersebut adalah sebagai berikut,

1. **Manajemen Metadata**

Pengelolaan metadata terkait, seperti *provenance* entitas, informasi temporal, struktur data, kualitas data, hingga log proses. Tugas ini bersifat menyeluruh karena diperlukan pada setiap tahapan konstruksi KG.

2. ***Data Acquisition dan Preprocessing***

Pemilihan sumber data yang relevan, akuisisi data, transformasi format, serta pembersihan awal. Tingkat detail proses ini bergantung pada keragaman sumber yang digunakan.

3. **Manajemen Ontologi**

Pembuatan dan pengembangan ontologi secara bertahap untuk mendefinisikan kelas, properti, dan relasi yang membentuk struktur semantik KG.

4. ***Knowledge Extraction (KE)***

Ekstraksi informasi terstruktur dari data tidak terstruktur atau semi-terstruktur menggunakan teknik seperti *Named Entity Recognition*, *entity linking*, dan *relation extraction*.

5. ***Entity Resolution (ER) dan Fusion***

Penyatuan entitas yang merepresentasikan objek yang sama untuk menghindari duplikasi.

6. ***Knowledge Completion***

Pelengkapan KG dengan pengetahuan baru melalui inferensi, prediksi relasi hilang, atau penambahan tipe entitas untuk meningkatkan kelengkapan semantik.

7. ***Quality Assurance (QA)***

Identifikasi masalah kualitas data pada KG serta penerapan strategi perbaikan yang dapat mencakup *validation checks*, konsistensi semantik, atau pemberian lanjutan.

Proses ini tidak selalu bersifat linear karena urutan eksekusi bergantung pada jenis sumber data dan beberapa langkah dapat menghilangkan kebutuhan langkah lain, seperti *entity linking* yang dapat menggantikan *entity resolution*. Selain itu, *metadata management* bersifat *cross-cutting* karena berlangsung pada seluruh tahap konstruksi KG.

II.5.2 Integrasi KG dalam RAG pada Tahap Perancangan Solusi

Sistem menggabungkan *Knowledge Graph* (KG) dengan alur RAG yang bertujuan untuk meningkatkan akurasi pencarian, kemampuan *multi-hop reasoning*, serta menjaga keterkaitan struktur dokumen. Arsitektur keseluruhan mengikuti tiga fase utama, yaitu *Indexing*, *Retrieval*, dan *Generation* (Knollmeyer, Caymazer, dan Grossmann 2025). Model arsitektur ditunjukkan pada Gambar II.3.

Arsitektur ini bekerja dalam tiga fase utama. Pada tahap *Indexing*, dokumen diekstrak dan dibentuk menjadi dua lapisan graf: *Document KG* (DKG) yang memetakan hierarki dokumen (bab–subbab–*chunk*) dan *Information KG* (IKG) yang menghubungkan antar-*chunk* melalui *keywords*; seluruh URI *chunk* juga disimpan pada basis data vektor untuk pencarian hibrid. Pada tahap *Retrieval*, sistem memperluas hasil *dense embedding* awal melalui tiga strategi graf: *Informed Chapter Search* (ICS, konteks satu bab), *Informed Keyword Search* (IKS, konteks berbasis IKG), dan *Uninformed Keyword Search* (UKS, berbasis kata kunci kueri). Pada tahap *Generation*, sistem menerapkan *pass condition* untuk membatasi konteks yang dikirim ke LLM, memastikan jawaban berbasis bukti dari konteks yang tersedia dengan tingkat *faithfulness* yang tinggi dan risiko *hallucination* yang minimal (Knollmeyer, Caymazer, dan Grossmann 2025).

II.5.3 GraphRAG: Integrasi *Knowledge Graph* dan *Large Language Models*

Large Language Models (LLM) dan *Knowledge Graphs* (KG) merupakan dua paradigma representasi pengetahuan yang saling melengkapi. LLM unggul dalam pemrosesan bahasa alami serta generalisasi lintas tugas tetapi rentan terhadap halusinasi fakta. Sebaliknya, KG menawarkan representasi pengetahuan faktual dalam bentuk struktur graf eksplisit yang kuat untuk penalaran simbolik namun kurang mampu menangani pemahaman bahasa alami yang kompleks (Pan dkk. 2024).

Untuk mengatasi kelemahan LLM tersebut, pendekatan *Retrieval-Augmented Generation* (RAG) konvensional sering digunakan. Namun, RAG tradisional bekerja

dengan cara memecah dokumen menjadi potongan teks (*chunks*) independen sehingga sering kali menghilangkan informasi struktural dan relasi esensial antar-entitas (Ma dkk. 2025). Sebagai solusi komprehensif atas keterbatasan tersebut, muncullah konsep *Graph Retrieval-Augmented Generation* (GraphRAG).

Berbeda dengan pencarian semantik biasa berbasis vektor teks, GraphRAG memanfaatkan mekanisme penelusuran graf (*graph traversal*) untuk mengambil konteks terstruktur secara utuh sebelum diberikan ke LLM (Han dkk. 2024). Mekanisme ini secara langsung mengekstraksi pengetahuan relevan dari data graf dengan merangkai subgraf atau jalur relasional berdasarkan kueri pengguna. Proses ini mencakup ekstraksi subgraf, penyaringan jalur (*path-filtering*), dan penyempurnaan jalur (*path-refinement*) guna menyajikan konteks penalaran yang akurat (Ma dkk. 2025).

Secara formal, integrasi generatif ini dimodelkan sebagai fungsi yang bersyarat terhadap pengetahuan graf:

$$p_{\theta}(y \mid x, \mathcal{K}) = \sum_{z \in \mathcal{Z}(x, \mathcal{K})} p_{\theta}(y \mid x, z) p(z \mid x, \mathcal{K}), \quad (\text{II.1})$$

dengan $\mathcal{K} = \{(h, r, t) \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}\}$ merepresentasikan KG sebagai himpunan *triples*, dan $\mathcal{Z}(x, \mathcal{K})$ merupakan himpunan subgraf atau fakta terstruktur yang ditarik dari graf berdasarkan kueri x . Dalam skema ini, KG bertindak sebagai ruang pengetahuan non-parametrik penyedia panduan penalaran (*reasoning guidelines*), sedangkan LLM bertindak sebagai generator parametrik (Ma dkk. 2025).

Pendekatan GraphRAG ini menjadi sangat krusial dalam domain berskala kompleks seperti manajemen konfigurasi Kubernetes. Dengan memetakan hierarki objek dan referensi silang ke dalam graf, algoritma *traversal* dapat merangkai jejak penalaran (*reasoning path*) yang kohesif. Injeksi konteks terstruktur ini ke dalam perintah LLM terbukti secara efektif menekan tingkat halusinasi, memastikan akurasi faktual, dan mempertahankan validitas sintaksis pada hasil akhir manifes YAML yang dibangkitkan.

II.6 Metrik Evaluasi Sistem Informasi Retrieval

Evaluasi sistem GraphRAG membutuhkan landasan metrik yang bersumber dari literatur *Information Retrieval* (IR) dan NLP. Bagian ini membahas konsep metrik standar yang menjadi acuan, sedangkan metrik khusus domain Kubernetes (seperti *Graph Coverage*, *Edge Coverage*, *Hop Accuracy*, *Multi-Hop Success*, *Scope Accuracy*, *Grounding Score*, dan *Hallucination Rate*) dirancang sebagai kontribusi penelitian dan diperkenalkan pada Bab III.

II.6.1 Answer Quality

Evaluasi kualitas jawaban sistem RAG dalam literatur umumnya menggunakan dua metrik berikut:

1. Faithfulness

Mengukur sejauh mana jawaban yang dihasilkan oleh LLM bersumber dari konteks yang di-*retrieve*, bukan dari pengetahuan parametrisnya sendiri (Es dkk. 2023). Metrik ini secara langsung mengukur risiko halusinasi: jawaban yang tinggi *faithfulness*-nya berarti setiap klaim dapat ditelusuri ke konteks yang disediakan.

$$\text{Faithfulness} = \frac{|\text{klaim berbasis konteks}|}{|\text{total klaim}|} \quad (\text{II.2})$$

2. Answer Relevance

Mengukur kesesuaian semantik antara jawaban yang dihasilkan dan maksud asli kueri pengguna (Es dkk. 2023). Implementasinya umumnya berbasis kemiripan kosinus antara representasi vektor jawaban dan pertanyaan.

II.6.2 Retrieval Quality

Evaluasi komponen *retrieval* menggunakan metrik standar IR yang mengukur ketepatan dan kelengkapan dokumen yang diambil:

1. Precision@k dan Recall@k

Precision@k mengukur proporsi dokumen relevan di antara k dokumen teratas yang diambil, sedangkan *Recall@k* mengukur proporsi dokumen relevan yang berhasil ditemukan dari seluruh dokumen relevan yang ada (Zhao dkk. 2024).

2. F1-Score@k

Merupakan rata-rata harmonik antara *Precision@k* dan *Recall@k*, memberikan gambaran seimbang tentang trade-off antara ketepatan dan kelengkapan (Zhao dkk. 2024):

$$\text{F1@k} = 2 \times \frac{\text{Precision@k} \times \text{Recall@k}}{\text{Precision@k} + \text{Recall@k}} \quad (\text{II.3})$$

3. NDCG@k (Normalized Discounted Cumulative Gain)

Mengevaluasi kualitas peringkat hasil *retrieval* dengan memberikan bobot lebih tinggi pada dokumen relevan yang muncul di posisi atas (Es dkk. 2023):

$$\text{NDCG@k} = \frac{\text{DCG@k}}{\text{IDCG@k}} \quad (\text{II.4})$$

Metrik ini penting dalam konteks RAG karena dokumen relevan yang terpotong akibat batasan token LLM tidak memberikan manfaat apapun.

II.6.3 *Syntactic Validity* dan *Schema Compliance*

Untuk evaluasi keluaran konfigurasi YAML, dua metrik standar yang relevan adalah:

1. *Syntactic Validity*

Mengukur persentase keluaran YAML yang dapat diurai tanpa galat menggunakan *parser* standar seperti `pyyaml`. Metrik ini bersifat biner: dokumen YAML valid atau tidak valid.

2. *Schema Compliance*

Mengukur kelengkapan *required fields* berdasarkan spesifikasi skema yang berlaku, dalam konteks ini spesifikasi OpenAPI Kubernetes v1.30 (The Kubernetes Authors 2024b). Catatan: kedua metrik ini hanya relevan untuk *fixture* bertipe `yaml_gen`.

II.6.4 *Reasoning Quality*

Evaluasi dimensi penalaran mengukur sejauh mana jawaban sistem dapat diverifikasi melalui konteks yang di-*retrieve*, bukan semata dari pengetahuan parametrik LLM:

1. *Grounding Score*

Mengukur proporsi istilah teknis Kubernetes dalam jawaban yang dapat ditelusuri secara langsung ke konteks yang di-*retrieve*, bukan berasal dari pengetahuan bawaan model (Es dkk. 2023). Nilai tinggi menunjukkan bahwa jawaban berbasis konteks graf, bukan halusinasi.

2. *Hallucination Rate*

Komplemen dari *Grounding Score*: mengukur proporsi istilah teknis dalam jawaban yang tidak ditemukan dalam konteks yang disediakan. Secara formal, $\text{Hallucination Rate} = 1 - \text{Grounding Score}$.

Metrik khusus *graph traversal* (*Hop Accuracy*, *Multi-Hop Success*, *Scope Accuracy*) dirancang sebagai kontribusi penelitian dan diperkenalkan pada Bab III.

Kerangka evaluasi tiga dimensi (*Answer Quality*, *Retrieval Quality*, *Reasoning Quality*) beserta metrik khusus yang dikembangkan untuk domain Kubernetes diuraikan secara rinci pada Bab III.

II.7 Penelitian Terkait

Bagian ini merangkum tiga penelitian terdahulu yang paling relevan terhadap pendekatan yang dikembangkan dalam penelitian ini. Ketiga penelitian tersebut dipilih

karena secara bersama-sama merepresentasikan perkembangan mutakhir penerapan *Knowledge Graph* dan RAG pada domain teknis spesifik, sehingga membentuk pijakan perbandingan langsung terhadap kontribusi yang diajukan.

II.7.1 GraphRAG untuk Platform IoT Domain-Spesifik

H.-H. Yu dkk. (2025) mengembangkan mesin dialog tanya jawab berbasis GraphRAG untuk membantu pengguna mengambil informasi secara presisi dari *Civil IoT Taiwan Data Service Platform*. Penelitian ini membangun *Knowledge Graph* secara semi-otomatis menggunakan GPT-4.1 untuk mengekstraksi entitas dan relasi dari dokumen platform, kemudian menggunakan model Llama-3-TAIDE-LX-8B untuk menerjemahkan pertanyaan bahasa alami pengguna ke jalur kueri graf. Sistem berhasil melampaui nilai F1 sebesar 0.7 pada kueri multi-entitas dan terbukti efektif menghindari halusinasi pada kasus data yang tidak tersedia.

Keterbatasan utamanya adalah *Knowledge Graph* yang bersifat statis tanpa mekanisme pembaruan dinamis, serta biaya adaptasi yang tinggi untuk platform lain. Perbedaan kunci dengan penelitian ini terletak pada sumber data dan topologi graf: penelitian H.-H. Yu dkk. (2025) membangun graf dari narasi dokumen tidak terstruktur dengan entitas IoT yang beragam, sedangkan penelitian ini membangun graf dari skema OpenAPI Kubernetes yang bersifat deterministik dengan taksonomi relasi yang didefinisikan secara eksplisit.

II.7.2 RAG Hibrida KG-Vektor untuk Manufaktur Pintar

Wan dkk. (2025) mengusulkan kerangka kerja Q&A berbasis RAG hibrida yang menggabungkan *Knowledge Graph* Neo4j dengan pencarian vektor untuk domain *Design for Additive Manufacturing* (DfAM). Konstruksi KG dilakukan berdasarkan metadata dari 69 publikasi teknis, diperkuat dengan *Semantic Alignment* dan *Prompt Enhancement* (PE) berbasis konstrain domain. Kombinasi pendekatan hibrida tersebut dengan GPT-4o menghasilkan akurasi 77.8% *Exact Match* dan 76.5% *Context Precision*, melampaui pendekatan RAG vektor atau graf secara mandiri.

Keterbatasan utamanya adalah latensi tinggi akibat proses pencarian hibrida yang kompleks, serta KG yang belum mendukung pembaruan data secara dinamis. Penelitian ini mengadopsi konsep hibrida KG-vektor yang sama, namun memperluas mekanisme *retrieval* dengan pencocokan eksak berbasis nama *resource* dan penelusuran multi-hop mengikuti taksonomi 18 jenis relasi Kubernetes yang dirancang secara domain-spesifik.

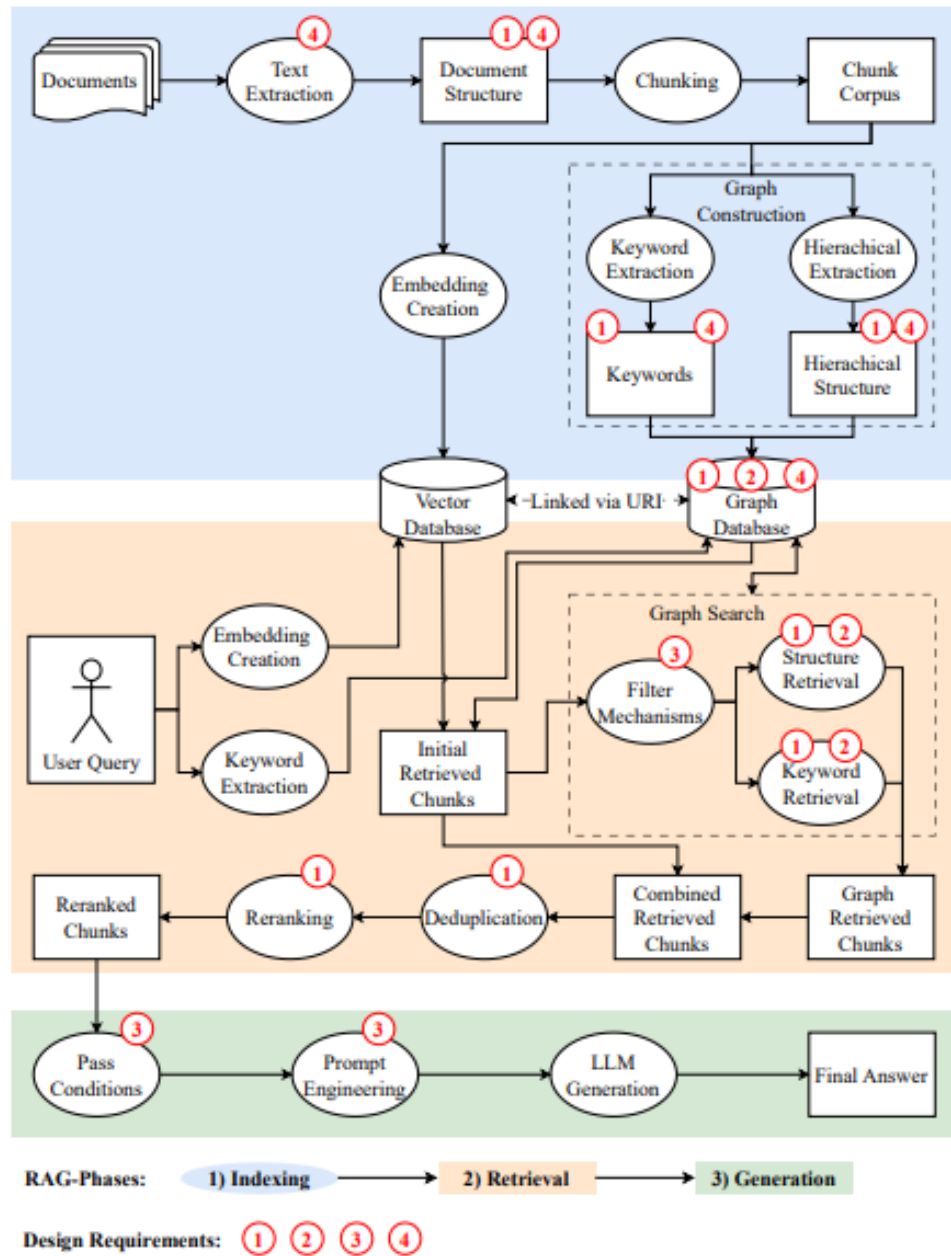
II.7.3 HSG-RAG: Konstruksi Basis Pengetahuan Hierarkis untuk Domain Teknis

Lu dkk. (October 2025) mengembangkan HSG-RAG (*Hierarchical Semantic Graph Retrieval Augmented Generation*), sebuah metode konstruksi dan pencarian basis pengetahuan bertingkat untuk mendukung pengembangan sistem *embedded*. Graf dibangun dari korpus dokumen teknis SDK dan *datasheet* tiga platform perangkat keras (ESP32S3, STM32F103, JL701) menggunakan LLM, mencakup dua lapisan relasi: jarak dekat antar-paragraf dan jarak jauh berbasis kemiripan semantik. Strategi pencarian *top-down* memecah kueri pengguna menjadi sub-kueri yang ditelusuri secara bertahap di dalam graf, menghasilkan jawaban yang lebih spesifik, ringkas, dan lengkap dibandingkan VanillaRAG maupun GraphRAG konvensional.

Keterbatasan utamanya adalah belum adanya mekanisme verifikasi otomatis kebenaran kode yang dihasilkan, serta cakupan evaluasi yang masih terbatas pada beberapa platform perangkat keras. Penelitian ini mengadopsi konsep dekomposisi kueri dan penelusuran bertingkat yang serupa, namun menerapkannya pada skema Kubernetes yang memiliki hierarki objek lebih terstruktur dan jenis relasi antar-*resource* yang terdefinisi secara formal dalam spesifikasi OpenAPI.

Ketiga penelitian terdahulu secara konsisten memvalidasi efektivitas pendekatan GraphRAG dalam domain teknis spesifik, sekaligus mengidentifikasi dua pola keterbatasan yang berulang: (1) KG bersifat statis tanpa mekanisme pembaruan dinamis, dan (2) taksonomi relasi yang belum disesuaikan dengan karakteristik skema target secara mendalam. Penelitian ini merespons kedua pola keterbatasan tersebut secara domain-spesifik untuk Kubernetes.

Berdasarkan kajian literatur yang telah dipaparkan, teridentifikasi dua kesenjangan utama yang menjadi landasan penelitian ini. Pertama, pendekatan RAG berbasis vektor konvensional terbukti tidak memadai untuk domain Kubernetes karena gagal menangkap relasi hierarkis dan referensial yang menentukan validitas konfigurasi (Wan dkk. 2025). Kedua, meskipun integrasi *Knowledge Graph* dalam RAG telah menunjukkan peningkatan signifikan pada domain umum, belum ada rancangan taksonomi relasi dan mekanisme *traversal* yang secara spesifik disesuaikan untuk skema OpenAPI Kubernetes. Bab selanjutnya menganalisis karakteristik data, kebutuhan sistem, dan alternatif pendekatan solusi untuk menjawab kedua kesenjangan tersebut.



Gambar II.3 Model arsitektur integrasi KG dalam RAG (Knollmeyer, Caymazer, dan Grossmann 2025)

BAB III

ANALISIS MASALAH

III.1 Analisis Kondisi Saat Ini

Bagian ini menganalisis permasalahan yang muncul dalam proses pengembangan sistem *cloud-native* berbasis Kubernetes, ditinjau dari dua sudut utama yaitu pemahaman masalah bisnis (*Business Understanding*) dan kondisi data yang menjadi sumber permasalahan (*Data Understanding*).

III.1.1 Pemahaman Masalah Bisnis (*Business Understanding*)

Transformasi digital di berbagai industri mendorong adopsi arsitektur *cloud-native* berbasis *microservices* yang dikelola melalui Kubernetes. Dalam praktiknya, Kubernetes tidak hanya berfungsi sebagai alat operasional, tetapi juga sebagai aset strategis yang menentukan kecepatan dan stabilitas siklus pengembangan perangkat lunak (*software development lifecycle*). Kompleksitas konfigurasi yang tinggi, kurva pembelajaran yang curam, serta kebutuhan untuk melakukan *onboarding* bagi *engineer* baru menjadikan Kubernetes sebuah titik krusial dalam kapasitas operasional.

Permasalahan terkait pengelolaan konfigurasi Kubernetes tidak hanya bersifat teknis, tetapi menimbulkan dampak bisnis berupa biaya operasional dan risiko produksi. Beberapa permasalahan utama yang diidentifikasi adalah sebagai berikut:

- 1. B01 - Inefisiensi Waktu (*Operational Cost*)**

Dokumentasi Kubernetes bersifat luas dan terfragmentasi sehingga *developer/DevOps* membutuhkan waktu yang cukup lama untuk menelusuri relasi antar-objek Kubernetes. Waktu pencarian informasi yang berlebih menjadi pemborosan sumber daya, terutama pada posisi *engineer* dengan biaya tenaga kerja tinggi.

- 2. B02 - Kesenjangan Pengetahuan (*Knowledge Gap*)**

Dokumentasi statis tidak sepenuhnya mendukung cara berpikir asosiatif manusia ketika memahami struktur spesifikasi. Sistem generatif berbasis LLM pun masih sering menghasilkan *hallucinated fields* karena tidak memiliki pemahaman skematik. Hal ini bisa memperburuk risiko kesalahan konfigurasi.

3. B03 - Risiko Kesalahan Konfigurasi (*Operational Risk*)

Kesalahan sintaksis atau penempatan atribut berkas konfigurasi YAML yang tidak sesuai spesifikasi dapat menyebabkan *deployment failure* hingga *downtime*. *Downtime* produksi berimplikasi langsung pada kerugian finansial, reputasi layanan, dan penundaan proses bisnis.

Ketiga butir *business understanding* tersebut menjadi dasar dalam penyusunan rumusan masalah penelitian. B01 mengenai inefisiensi waktu akibat dokumentasi yang terfragmentasi mengarah pada kebutuhan representasi pengetahuan yang lebih terstruktur sehingga dirumuskan sebagai Rumusan Masalah 1 tentang rekonstruksi spesifikasi OpenAPI menjadi *knowledge graph*. B02 yang menyoroti kesenjangan pemahaman skematik dan risiko *hallucination* pada LLM memunculkan Rumusan Masalah 2 terkait perancangan mekanisme *graph-guided retrieval* untuk menghadirkan konteks struktural yang akurat. Sementara itu, B03 terkait risiko kesalahan konfigurasi dan dampak operasionalnya diterjemahkan menjadi Rumusan Masalah 3 yang berfokus pada evaluasi efektivitas GraphRAG dalam meningkatkan relevansi konteks, *faithfulness*, serta validitas sintaksis dibandingkan pendekatan *baseline*.

III.1.2 Pemahaman Data (*Data Understanding*)

Bagian ini menjelaskan karakteristik data dokumentasi API Kubernetes yang menjadi sumber utama dalam proses konstruksi *Graph-based Retrieval Augmentation*. Analisis mencakup identitas data, struktur, pola hubungan antar-objek Kubernetes, keterbatasan kualitas, serta seleksi data relevan yang digunakan dalam penelitian.

III.1.2.1 Sumber dan Spesifikasi Data (*Data Source & Specifications*)

Data penelitian diperoleh dari repositori resmi Kubernetes pada GitHub dengan alamat `kubernetes/kubernetes`. Data diambil dalam bentuk berkas `swagger.json` yang mengikuti standar *OpenAPI Specification* (OAS). Penelitian ini menggunakan versi Kubernetes **v1.30** sebagai acuan. Berkas `swagger.json` memiliki ukuran rata-rata 3,757 MB dan memuat 2.500–3.500 definisi API. Kompleksitas tersebut menjustifikasi penggunaan metode ekstraksi otomatis, karena pendekatan manual berpotensi menghasilkan kesalahan interpretasi struktur dan kehilangan relasi semantik antar-objek Kubernetes.

III.1.2.2 Analisis Struktur Dokumen

Berdasarkan analisis terhadap struktur `swagger.json`, terdapat dua komponen utama beserta karakteristik hierarkisnya:

1. **Definitions (atau Schema Resources):** Mendeskripsikan spesifikasi objek Kubernetes seperti *Pod*, *Service*, dan *Deployment*. Hampir seluruh objek Ku-

bernetes mematuhi pola struktural wajib yang terdiri atas *apiVersion*, *kind*, *metadata*, *spec*, dan *status*. Namun, isi dari atribut *spec* sangat bervariasi antar-komponen di dalam teks YAML. Akibatnya, waktu pembuatan konfigurasi meningkat dan pengembang sangat bergantung pada panduan eksternal. Secara spesifik, setiap definisi skema di bagian ini memuat:

- a. *properties*: Merepresentasikan ruas (*field*) aktual pada berkas konfigurasi YAML.
 - b. *type*: Menentukan format tipe data (seperti *object*, *array*, *string*, *integer*, *boolean*).
 - c. *required*: Mendefinisikan daftar ruas konfigurasi wajib isi.
 - d. *description*: Menyediakan dokumentasi tekstual mengenai fungsionalitas ruas.
 - e. *\$ref*: Bertindak sebagai referensi silang ke definisi skema bersarang (*nested schema*). Ciri utama dari struktur dokumen ini adalah ketergantungan referensial bersarang antar-definisi skema. Akibatnya, satu ruas YAML dapat mengarah ke berbagai kemungkinan bentuk, meliputi:
 - i. Definisi objek skema lain.
 - ii. Struktur daftar (*list*).
 - iii. Struktur pemetaan (*map*) dengan batasan (*constraint*) khusus.
2. ***Paths***: Mendeskripsikan *endpoint* API, misalnya `GET /api/v1/ pods`, sebagai antarmuka interaksi objek Kubernetes dalam ekosistem kluster. Setiap titik akhir memiliki atribut penentu interaksi komunikasi yang memuat:
- a. Metode HTTP fungsional (*GET*, *POST*, *PUT*, *PATCH*, *DELETE*).
 - b. Parameter injeksi pada jalur (*path*) dan kueri (*query*).
 - c. Skema balasan dari peladen (*response schema*).
 - d. Relasi struktural pembentuk fungsionalitas yang mengarah kembali ke blok *definitions*.

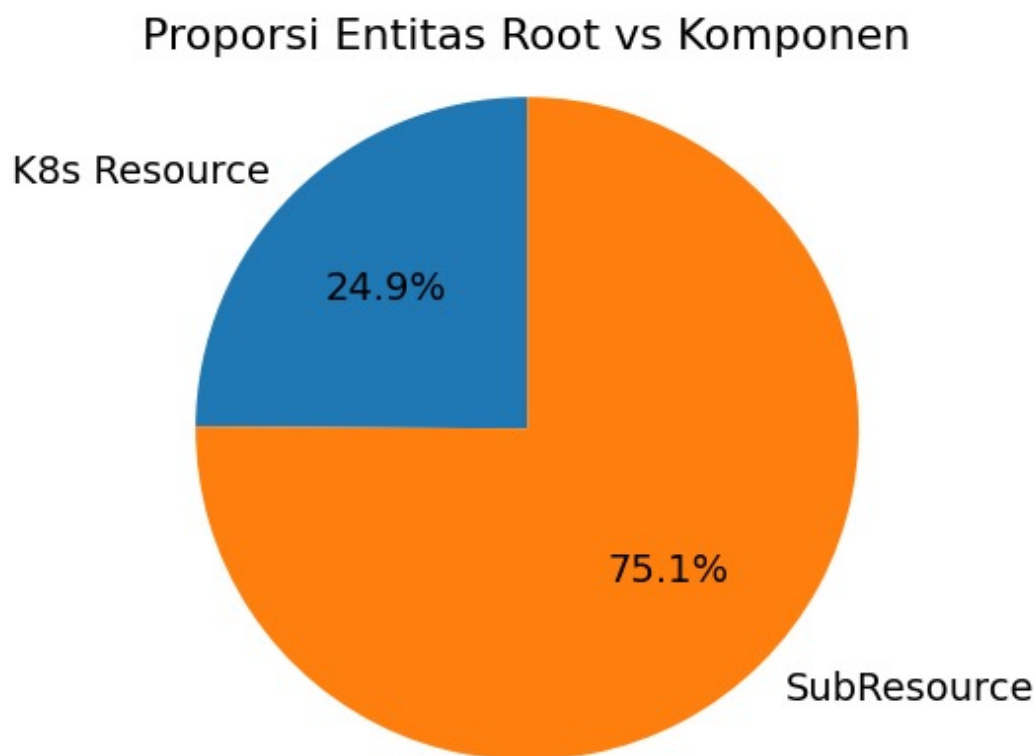
III.1.2.3 Analisis Data Eksploratif Spesifikasi OpenAPI

Sebelum merancang skema *knowledge graph*, analisis data eksploratif (*Exploratory Data Analysis* / EDA) dilakukan terhadap berkas `kubernetes_swagger.json` berukuran 3,67 MB guna memetakan skala, kompleksitas struktural, dan sebaran definisi skema. Proses ekstraksi awal berhasil mengidentifikasi 735 definisi skema. Sebanyak 5 entri diabaikan karena teridentifikasi sebagai data bising (*noise*), menyisakan 730 definisi skema valid untuk dievaluasi lebih lanjut.

Temuan utama dari proses EDA ini dikelompokkan ke dalam tiga metrik utama:

1. **Distribusi objek Kubernetes *root* vs komponen pendukung**

Klasifikasi definisi skema menunjukkan ketimpangan proporsi yang sangat signifikan antara objek utama dan objek pendukung. Dari total 730 definisi, hanya 183 (24,9%) terklasifikasi sebagai objek Kubernetes akar (*root resources*). Objek akar ini merupakan objek mandiri penyangga atribut *GroupVersionKind* (GVK) seperti *Deployment* atau *Pod* sehingga dapat diterapkan secara langsung ke dalam kluster. Sementara itu, mayoritas mutlak sebanyak 547 definisi (75,1%) hanyalah skema pendukung (*sub-resources*) pembangun komponen akar tersebut. Ketimpangan ini membuktikan betapa masifnya dependensi struktural di dalam Kubernetes.



Gambar III.1 Proporsi objek Kubernetes *root* vs komponen pendukung dalam spesifikasi OpenAPI Kubernetes

2. Kepadatan Properti dan Intensitas Referensi Silang

Inspeksi terhadap struktur properti mengungkap bahwa setiap definisi skema rata-rata memuat 4,1 ruas (*field*), dengan objek Kubernetes paling kompleks menampung hingga 44 ruas parameter. Temuan paling krusial yang memvalidasi tingginya kompleksitas hierarki adalah keberadaan referensi silang (\$ref). Sebanyak 437 definisi (hampir 60% dari total data) terbukti mengandalkan referensi ke skema lain dengan rata-rata 1,11 referensi per definisi. Tingginya intensitas referensi ini secara kuantitatif memvalidasi argumen

mengenai sulitnya melacak skema bersarang (*nested schema*) secara manual.

Keseluruhan temuan kuantitatif EDA ini secara langsung menjadi landasan arsitektural dalam perancangan skema graf. Pemetaan atas 149 jenis (*kind*) objek Kubernetes unik hasil analisis ini secara presisi diproyeksikan akan membentuk sebuah *knowledge graph* Kubernetes yang memuat 730 *node* pendefinisi objek beserta estimasi 1.486 *edge* pembentuk topologi infrastrukturnya.

III.1.2.4 Analisis Keterhubungan Data

1. Relasi Tingkat Skema (*Schema-Level*)

Kategori ini terpetakan dan direpresentasikan secara utuh di dalam graf karena bersumber langsung dari struktur statis spesifikasi OpenAPI. Relasi pada tingkat ini mencakup:

- a. **Sifat Atomik *Pod* (*Passive Relationships*):** *Pod* bertindak sebagai unit eksekusi atomik pasif tanpa relasi skema eksplisit ke luar. Relasinya terbentuk secara pasif akibat interaksi struktural dari objek Kubernetes eksternal.
- b. **Komposisi Hierarkis (*Hierarchical Composition*):** Terjadi ketika satu objek Kubernetes mengandung definisi objek lain secara utuh. Contoh utamanya adalah *Deployment* mereferensikan *Pod* melalui blok *spec.template* menyimpan keseluruhan spesifikasi *PodSpec*.
- c. **Referensi Langsung (*Direct Reference & Injection*):** Terjadi ketika sebuah objek Kubernetes memanggil objek lain secara eksplisit pada parameternya. Contohnya adalah injeksi konfigurasi satu arah dari *ConfigMap* atau *Secret* ke lingkungan eksekusi *Pod* seperti pada Kode III.1

1	<code>env:</code>	1	<code>env:</code>
2	<code>valueFrom:</code>	2	<code>valueFrom:</code>
3	<code>configMapKeyRef:</code>	3	<code>secretKeyRef:</code>
4	<code>name: "app-config"</code>	4	<code>name: "db-secret"</code>
5		5	

Listing III.1 Injeksi ConfigMap

Listing III.2 Injeksi Secret

2. Relasi Semi-Statis (*Semi-Static*)

Kategori ini terpetakan ke dalam graf secara konseptual atau sebagai properti *node*, namun tidak membentuk jalur semantik fungsional secara utuh.

- a. **Asosiasi Label/Selektor (*Label/Selector Association*):** Direpresentasikan sekadar sebagai *edge* konseptual pembentuk topologi graf.
- b. **Deteksi Cakupan Klaster (*Cluster-Scope Detection*):** Pembatasan area operasional tidak dipetakan sebagai ruang graf terpisah melainkan dia-

komodasi melalui atribut properti cakupan (*scope property*) pada setiap *node*.

- c. **Anotasi dan Toleransi (*Annotations/Tolerations*):** Nilai konfigurasi tambahan ini hanya disimpan sebagai properti pada *node* tanpa memicu pembentukan *edge* semantik dengan objek Kubernetes lain.

3. Relasi Tingkat Operasional (*Runtime-Level*)

Kategori ini secara mutlak berada di luar cakupan fungsional sistem GraphRAG.

Relasi pada tingkat ini mencakup:

- a. **Hierarki Kepemilikan (*OwnerReferences*):** Merupakan atribut meta-data operasional penentu pengumpulan sampah sumber daya (*garbage collection*) saat infrastruktur berjalan.
- b. **Interaksi Lintas Ruang Nama (*Cross-Namespace Interactions*):** Melibatkan perilaku pembatas akses operasional jaringan secara langsung (*runtime behavior*).
- c. **Logika Pencocokan Label (*Label Matching Logic*):** Merupakan proses komputasi evaluasi dinamis oleh layanan kluster untuk menentukan target pembagian beban jaringan sesungguhnya.

4. Keterbatasan Inheren dan Pengembangan Lanjutan

Ketiadaan pemetaan pada relasi tingkat operasional merupakan batasan inheren dalam penelitian ini. Hal ini terjadi karena arsitektur GraphRAG dibangun secara eksklusif bermodalkan skema OpenAPI (*swagger.json*) penyedia representasi struktur statis dan bukan pembaca status kluster yang sedang beroperasi (*runtime state*). Sebagai langkah pengembangan lanjutan (*future work*) guna mengakomodasi kasus penggunaan pelacak relasi dinamis, sistem pemetaan graf ini dapat diintegrasikan secara langsung dengan mekanisme pemantauan kontroler Kubernetes (*controller watch mechanism*) atau penerapan pola operator (*operator pattern*).

III.1.2.5 Desain Taksonomi *Edge* untuk *Knowledge Graph* Kubernetes

Berdasarkan analisis pola keterhubungan data di atas, penelitian ini merancang 18 jenis *edge* yang terdistribusi dalam 7 kategori relasi. Taksonomi ini adalah kontribusi penelitian yang dirancang khusus berdasarkan semantik domain Kubernetes. Seluruh 18 *edge type* diimplementasikan dalam kode pada variabel `_ALL_EDGE_TYPES` di `src/graph/queries.py`.

Tabel III.1 Taksonomi 18 jenis *edge* dalam *knowledge graph* Kubernetes

No	<i>Edge Type</i>	Kategori	Contoh Relasi
1	<i>HAS_PROPERTY</i>	Struktural	<i>Deployment</i> → <i>spec</i>
2	<i>EXTENDS</i>	Struktural	<i>Deployment</i> → <i>ObjectMeta</i>
3	<i>ONE_OF</i>	Struktural	<i>Volume</i> → satu sumber penyimpanan
4	<i>ANY_OF</i>	Struktural	<i>EnvFromSource</i> → <i>ConfigMap/Secret</i>
5	<i>CONTAINS_POD_TEMPLATE</i>	Workload	<i>Deployment</i> → <i>PodTemplateSpec</i>
6	<i>CONTAINS_JOB_TEMPLATE</i>	Workload	<i>CronJob</i> → <i>JobTemplateSpec</i>
7	<i>HAS_CONTAINER</i>	Workload	<i>PodSpec</i> → <i>Container</i>
8	<i>CLAIMS_VOLUME</i>	Penyimpanan	<i>StatefulSet</i> → <i>PersistentVolumeClaim</i>
9	<i>MOUNTS_VOLUME</i>	Penyimpanan	<i>PodSpec</i> → <i>Volume</i>
10	<i>USES_STORAGE_CLASS</i>	Penyimpanan	<i>PVC</i> → <i>StorageClass</i>
11	<i>LOADS_CONFIGMAP</i>	Konfigurasi	<i>Container</i> → <i>ConfigMap</i>
12	<i>USES_SECRET</i>	Konfigurasi	<i>PodSpec</i> → <i>Secret</i>
13	<i>SELECTS_POD</i>	Jaringan	<i>Service</i> → <i>Pod</i> via <i>selector</i>
14	<i>ROUTES_TO_SERVICE</i>	Jaringan	<i>Ingress</i> → <i>Service</i>
15	<i>BINDS_ROLE</i>	RBAC	<i>RoleBinding</i> → <i>Role</i>
16	<i>BINDS_SERVICE_ACCOUNT</i>	RBAC	<i>RoleBinding</i> → <i>ServiceAccount</i>
17	<i>USES_SERVICE_ACCOUNT</i>	RBAC	<i>PodSpec</i> → <i>ServiceAccount</i>
18	<i>SCALES_RESOURCE</i>	Autoscaling	<i>HPA</i> → <i>Deployment/StatefulSet</i>

Graf yang dibangun bersifat *directed graph*: setiap *edge* A→B mencerminkan referensi tipe dalam skema OpenAPI (A memiliki *field* bertipe B). Arah ini bukan arbitrer—ia merepresentasikan dependensi struktural yang deterministik dan dapat

diverifikasi langsung dari `swagger.json`. Akibatnya, traversal yang dimulai dari *node* A tidak dapat menjangkau *node* yang mereferensikan A kecuali terdapat *edge* keluar eksplisit dari A ke *node* tersebut. Misalnya, *edge* `RoleBinding`→`ServiceAccount` ada dalam graf, sedangkan *edge* `ServiceAccount`→`RoleBinding` tidak ada—sehingga traversal yang dimulai dari *ServiceAccount* tidak dapat mencapai *RoleBinding* melalui penelusuran *edge* keluar.

III.1.2.6 Justifikasi Batas Kedalaman Penelusuran Graf (3 Lompatan)

Batas maksimum 3 lompatan (*hops*) dalam penelusuran graf ditetapkan berdasarkan analisis distribusi kedalaman skema OpenAPI Kubernetes. Tabel III.2 menunjukkan bahwa karakteristik *node* berubah secara signifikan seiring bertambahnya kedalaman:

Tabel III.2 Distribusi karakteristik *node* berdasarkan kedalaman penelusuran graf

Kedalaman	Karakteristik <i>Node</i>	Contoh	Penggunaan
1–2	Spesifik <i>resource</i> (dibagikan 2–3 <i>resource</i>)	<i>DeploymentSpec</i> , <i>ServiceSpec</i>	Tipe <i>explain</i> , <i>followup</i>
3	<i>Field</i> tingkat kontainer	<i>image</i> , <i>ports</i> , <i>env</i> , <i>resources</i>	Tipe <i>generate_yaml</i> , <i>trace_relationship</i>
4+	Tipe generik (dibagikan 19–136 <i>resource</i>)	<i>Quantity</i> , <i>IntOrString</i> , <i>LocalObjectReference</i>	Tidak digunakan (menurunkan presisi)

Dengan menggunakan kedalaman yang disesuaikan per tipe *intent* (kedalaman 2 untuk *explain/followup*, kedalaman 3 untuk *generate_yaml/trace_relationship*), sistem mengoptimalkan *Context Precision* dengan menghindari injeksi tipe generik yang tidak informatif ke dalam konteks LLM.

III.1.2.7 Analisis Kompleksitas Pertanyaan dan Batasan Kemampuan Sistem

Batasan kedalaman traversal memiliki implikasi langsung terhadap jenis pertanyaan yang dapat dijawab secara optimal. Tabel III.3 mengklasifikasikan pertanyaan pengguna ke dalam empat tingkat berdasarkan kedalaman informasi yang dibutuhkan dan cakupan pengetahuan graf.

Tabel III.3 Klasifikasi Tingkat Kompleksitas Pertanyaan Pengguna

Tier	Kategori	Karakteristik	Kualitas
1	Optimal	Satu <i>resource</i> , informasi pada kedalaman ≤ 3 hop. Contoh: “ <i>Apa itu Deployment?</i> ”, “ <i>Buat YAML StatefulSet dengan PVC.</i> ”	Tinggi
2	Terdegradasi	Informasi pada kedalaman 4–5 hop, atau melibatkan dua <i>resource</i> utama. Contoh: “ <i>Resource limits container di dalam Deployment.</i> ”	Sedang
3	Di Luar Graf	Membutuhkan data <i>runtime</i> atau perintah operasional. Contoh: “ <i>Kenapa Pod saya CrashLoopBackOff?</i> ”	Rendah
4	Di Luar Domain	Tidak berkaitan dengan Kubernetes dan ekosistemnya.	Ditolak

Ketika pertanyaan melampaui batas Tier 1, sistem tidak mengembalikan pesan kesalahan, melainkan mengisi kesenjangan konteks menggunakan pengetahuan parametrik LLM (*silent degradation*). Kondisi ini menjadi salah satu faktor yang menjelaskan perbedaan nilai *Retrieval Quality* dan *Answer Quality* pada hasil evaluasi.

III.2 Analisis Kebutuhan

Tahap analisis kebutuhan berfokus pada identifikasi kapabilitas yang harus dimiliki sistem guna menjawab tujuan penelitian dan kebutuhan bisnis. Untuk memastikan bahwa kebutuhan yang dirumuskan bersifat relevan dengan masalah yang dihadapi, dilakukan analisis kesenjangan teknis (*technical gap*) pada pendekatan yang ada saat ini pada sistem *Retrieval-Augmented Generation*, termasuk pendekatan GraphRAG terkini pada domain teknis serupa (H.-H. Yu dkk. 2025; Lu dkk. October 2025), pada Tabel III.4.

Tabel III.4 *Technical Gap Analysis* pada Domain Kubernetes

Aspek Evaluasi	Kondisi Saat Ini (<i>Existing Approaches</i>)	Kesenjangan di Domain Kubernetes
T01 - Representasi Struktur Data	Menggunakan pemecahan teks secara <i>linear chunking</i> ataupun konstruksi <i>knowledge graph</i> berbasis ekstraksi entitas generik dari teks, tanpa mendefinisikan taksonomi jenis relasi yang spesifik terhadap semantik domain target (Cao dkk. 2025; H.-H. Yu dkk. 2025; Lu dkk. October 2025).	Terjadi <i>loss of deep context</i> dan ambiguitas semantik relasi. Sistem gagal memahami kedalaman hierarki bersarang (misalnya properti <i>limits</i> sebagai turunan dari Deployment) sekaligus tidak dapat membedakan jenis relasi antar-objek Kubernetes—kepemilikan, ketergantungan konfigurasi, akses jaringan, dan kebijakan RBAC memiliki perilaku <i>traversal</i> yang berbeda (Kotstein dan Decker 2024).
T02 - Mekanisme Validasi	Validasi bergantung pada model LLM atau aturan manual berbasis <i>hard constraints</i> yang harus ditulis satu per satu (Wan dkk. 2025).	Risiko terjadinya <i>syntactic hallucination</i> tinggi (LLM mengarang <i>field</i>), atau beban pemeliharaan menjadi besar saat aturan manual harus diperbarui (Gao dkk. 2024).
T03 - Ambiguitas Entitas	Mengandalkan frekuensi kemunculan kata kunci (<i>keyword frequency</i>) atau kedekatan vektor semata.	Terdapat ambiguitas pada pola <i>loose coupling</i> karena <i>keywords</i> (misal: <i>labels</i> , <i>ports</i>) muncul berulang di banyak objek berbeda. Hal ini menyebabkan tercampurnya konteks antar-objek (<i>contextual mixing</i>).

bersambung ke halaman berikutnya

Tabel III.4 *Technical Gap Analysis* pada Domain Kubernetes (lanjutan)

Aspek Evaluasi	Kondisi Saat Ini (<i>Existing Approaches</i>)	Kesenjangan di Domain Kubernetes
T04 - Pemeliharaan Pengetahuan	Membutuhkan <i>fine-tuning</i> ulang model atau pembaruan aturan manual ketika terjadi perubahan versi API (Gao dkk. 2024). Penelitian GraphRAG terkini pada domain teknis serupa pun masih menghadapi keterbatasan yang sama berupa <i>knowledge graph</i> yang bersifat statis tanpa mekanisme pembaruan dinamis (H.-H. Yu dkk. 2025; Lu dkk. October 2025).	Tidak efisien, mudah menjadi <i>obsolete</i> , dan sensitif terhadap perubahan versi Kubernetes yang dirilis secara berkala (± 4 bulan).
T05 - Akurasi Jawaban	Sistem cenderung menghasilkan jawaban naratif umum dan tidak memberikan jaminan presisi sintaksis konfigurasi YAML.	Tidak mampu menghasilkan artefak konfigurasi yang sesuai karena sering ditemukan kesalahan format atau struktur.

III.2.1 Kebutuhan Fungsional

Berdasarkan *Business Understanding* (B01–B03), diidentifikasi sejumlah urgensi bisnis yang dianalisis lebih lanjut hingga menghasilkan beberapa celah teknis (*technical gaps*) yang menjadi dasar perumusan kebutuhan sistem. Setiap *technical gap* kemudian dijawab melalui perancangan kebutuhan fungsional yang akan diimplementasikan untuk menyelesaikan permasalahan bisnis yang ada. Tabel III.5 berikut memetakan hubungan antara kebutuhan bisnis, *technical gap*, serta kebutuhan fungsional yang dirumuskan untuk mengatasinya.

Tabel III.5 Pemetaan Kebutuhan Bisnis terhadap *Technical Gap* dan Kebutuhan Fungsional

Business Requirement (B)	Technical GAP (T)	Functional Requirement (F)
B01 – Inefisiensi waktu	T01 – Representasi struktur data	F-01 <i>Structured Object Representation</i>
		F-02 <i>Structured Relation Retrieval</i>
	T03 – Ambiguitas entitas	F-03 <i>Intent Classification</i>
		F-04 <i>Context Construction & Injection</i>
B02 – Kesenjangan pengetahuan	T02 – Mekanisme validasi	F-05 <i>Attribute Enrichment</i>
		F-06 <i>Web Interaction Interface</i>
	T04 – Pemeliharaan pengetahuan	F-01 <i>Structured Object Representation</i>
B03 – Risiko kesalahan konfigurasi	T05 – Akurasi jawaban	F-07 <i>Conditional YAML Generation Constraint</i>

Berdasarkan hasil pemetaan pada Tabel III.5, berikut adalah kebutuhan fungsional yang dibutuhkan oleh sistem,

Tabel III.6 Kebutuhan Fungsional Sistem

Kode	Parameter	Deskripsi Spesifikasi
F-01	<i>Structured Object Representation</i>	Sistem harus mengubah skema objek ke dalam suatu representasi terstruktur yang membedakan setiap komponen, <i>field</i> , dan relasi antar-objek sehingga dapat diakses kembali secara terprogram pada tahap <i>retrieval</i> .

bersambung ke halaman berikutnya

Tabel III.6 Kebutuhan Fungsional Sistem (lanjutan)

Kode	Parameter	Deskripsi Spesifikasi
F-02	<i>Structured Relation Retrieval</i>	Sistem harus menyediakan mekanisme <i>retrieval</i> yang mampu mengambil himpunan objek dan <i>field</i> yang saling berkaitan (misalnya hubungan antara <i>Deployment</i> , <i>Pod</i> , dan <i>Service</i>) berdasarkan entitas awal dan <i>intent</i> pengguna sebagaimana didefinisikan pada F-03.
F-03	<i>Intent Classification</i>	Sistem harus mampu (1) mendeteksi entitas Kubernetes yang dirujuk pengguna (misalnya <i>Service</i> , <i>Pod</i>), dan (2) mengklasifikasikan maksud pengguna ke dalam lima tipe <i>intent</i> : dua tipe utama, yaitu explain (pertanyaan definisi, cara kerja, atau tujuan suatu <i>resource</i>) dan generate_yaml (permintaan pembuatan konfigurasi YAML), serta tiga mode pendukung, yaitu trace_relationship (penelusuran relasi antar- <i>resource</i>), followup (modifikasi atau perluasan jawaban sebelumnya), dan planning (perencanaan arsitektur sistem multi-komponen). Setiap tipe <i>intent</i> mengontrol kedalaman penelusuran graf: <i>explain/followup</i> menggunakan kedalaman 2, sedangkan <i>generate_yaml/trace_relationship/planning</i> menggunakan kedalaman 3.
F-04	<i>Context Construction & Injection</i>	Sistem harus mengubah hasil F-02 menjadi sebuah konteks dan menyertakannya sebagai bagian dari <i>prompt</i> yang dikirimkan ke LLM.

bersambung ke halaman berikutnya

Tabel III.6 Kebutuhan Fungsional Sistem (lanjutan)

Kode	Parameter	Deskripsi Spesifikasi
F-05	<i>Attribute Enrichment</i>	Sistem harus menyimpan metadata untuk setiap <i>field</i> konfigurasi di dalam representasi terstruktur sehingga informasi tersebut tersedia saat konteks disiapkan untuk menjawab pertanyaan pengguna.
F-06	<i>Web Interaction Interface</i>	Sistem harus menyediakan antarmuka web interaktif berbasis Streamlit yang memungkinkan pengguna memasukkan pertanyaan dalam bahasa alami, menampilkan jawaban beserta konfigurasi YAML (jika relevan), serta menyajikan jejak penalaran graf (<i>reasoning path</i>) secara visual dengan pesan kesalahan yang jelas.
F-07	<i>Conditional YAML Generation Constraint</i>	Sistem harus menginstruksikan LLM untuk menghasilkan keluaran berupa blok kode YAML yang valid secara sintaksis apabila <i>intent</i> pengguna diklasifikasikan sebagai generate_yaml . Untuk <i>intent explain</i> , trace_relationship , dan followup , sistem tidak boleh memaksakan format YAML dan harus menghasilkan jawaban naratif.

III.2.2 Kebutuhan Non Fungsional

Kebutuhan non-fungsional mendefinisikan karakteristik kualitas yang harus dimiliki oleh sistem untuk memastikan kinerja, keandalan, dan keamanan operasional. Kebutuhan ini penting untuk menjamin bahwa sistem tidak hanya berfungsi dengan benar, tetapi juga memenuhi standar kualitas yang diharapkan oleh pengguna. Berikut adalah kebutuhan non-fungsional utama yang diidentifikasi:

Tabel III.7 Kebutuhan Non-Fungsional Sistem

Kode	Parameter	Deskripsi Spesifikasi
NF-01 (Reliability)	<i>Fail-Safe Mechanism</i>	Sistem harus mencegah keluaran halusinasi melalui mekanisme <i>error handling</i> , yaitu (1) Jika entitas tidak ditemukan, sistem harus memberi respons “Entity not found”. (2) Jika YAML terdeteksi tidak valid oleh validator internal, sistem harus menampilkan peringatan “Warning: Potential Syntax Error” pada antarmuka pengguna.
NF-02 (Performance)	<i>Retrieval–Generation Latency</i>	Waktu total yang dibutuhkan untuk memproses satu permintaan pengguna (<i>end-to-end latency</i>) tidak boleh melebihi 60 detik pada lingkungan pengujian standar.
NF-03 (Usability)	<i>Web UI Usability Standard</i>	Antarmuka web interaktif berbasis Streamlit harus menyediakan pengalaman penggunaan yang konsisten, mudah dipahami, dan informatif melalui komponen <i>chat input</i> , panel jawaban, serta visualisasi jejak penalaran graf.
NF-04 (Portability)	<i>Resource Constraints</i>	Seluruh komponen sistem harus dapat dijalankan pada lingkungan lokal (<i>Local Environment</i>) dengan spesifikasi perangkat keras terbatas (minimal RAM 8GB), tanpa ketergantungan pada infrastruktur <i>cloud</i> terdistribusi.

III.2.3 Pemetaan Kebutuhan Bisnis terhadap Kerangka Evaluasi

Untuk memastikan bahwa setiap dimensi evaluasi sistem memiliki dasar bisnis yang jelas, Tabel III.8 memetakan hubungan antara kebutuhan bisnis (B01–B03), dimensi evaluasi, dan metrik yang digunakan.

Tabel III.8 Pemetaan kebutuhan bisnis terhadap dimensi dan metrik evaluasi

Kebutuhan Bisnis	Pertanyaan Evaluasi	Dimensi	Metrik Utama
B01 – Inefisiensi Waktu	Apakah <i>retrieval</i> efisien dan tepat sasaran?	RetQ (35%)	F1@k, NDCG@k, Graph Coverage
B02 – Kesenjangan Pengetahuan	Apakah penalaran berbasis graf menghasilkan konteks yang benar?	ReaQ (25%)	Hop-Accuracy, Multi-Hop Success Rate, Hallucination Rate
B03 – Risiko Konfigurasi	Apakah keluaran YAML valid dan relevan terhadap kueri?	AnsQ (40%)	Syntactic Validity, Schema Compliance, Answer Relevance, Faithfulness

Pemetaan ini mempertegas bahwa dimensi evaluasi bukan sekadar metrik teknis, melainkan representasi langsung dari dampak bisnis yang ingin dimitigasi oleh sistem GraphRAG.

III.3 Alternatif Solusi

Bagian ini membahas berbagai alternatif solusi yang dapat diimplementasikan untuk memenuhi kebutuhan sistem yang telah diidentifikasi sebelumnya.

III.3.1 Hybrid KG–Vector RAG

Pendekatan *hybrid KG–Vector RAG* menggabungkan dua jalur pencarian, yakni *vector-based RAG* untuk pencarian semantik dan *knowledge graph* (KG) berbasis metadata untuk penalaran relasional (Wan dkk. 2025). Kueri pengguna diproses melalui ekstraksi entitas untuk menarik subgraf relevan dari KG sekaligus mengambil potongan teks melalui pencarian kemiripan vektor; kedua konteks terstruktur dan semantik ini kemudian digabungkan sebagai basis generasi bagi LLM.

Hasil evaluasi oleh Wan dkk. (2025) pada dataset benchmark menunjukkan peningkatan yang signifikan dibanding *vector-only RAG*: *Exact Match* meningkat dari 63,4% menjadi 77,8% (+14,4 poin persentase) dan *Context Precision* dari 69,8% menjadi 76,5% (+6,7 poin persentase). Konsekuensi dari penggabungan dua jalur

ini adalah lonjakan latensi akibat *traversal overhead* graf dan kompleksitas *prompt engineering*.

III.3.2 *GNN-augmented GraphRAG* (G-Retriever)

Arsitektur G-Retriever dirancang untuk menangani pemahaman graf tekstual berskala besar dengan memadukan *Graph Neural Networks* (GNN) dan LLM (He dkk. 2024). Sistem ini mereduksi ukuran konteks kueri melalui algoritma *Prize-Collecting Steiner Tree* (PCST) guna membangun subgraf koheren berisi *node* dan *edge* paling informatif sebelum disuntikkan ke dalam LLM.

Secara kuantitatif, pendekatan ini mampu memangkas penggunaan token hingga **99%** dibanding RAG konvensional, sekaligus menaikkan validitas *node* dari 31% menjadi **77%** (+46 poin persentase) dan validitas *edge* dari 12% menjadi **76%** tanpa memerlukan proses *fine-tuning* pada LLM (He dkk. 2024). Namun, metode PCST memiliki kompleksitas komputasi $O(|V| + |E|)$ yang semakin meningkat seiring ukuran graf, dan ketergantungan terhadap GNN menyebabkan *indexing* harus diulang setiap versi Kubernetes berubah.

III.3.3 *Hybrid Neural-Symbolic* dengan *Relational DB* (NeuSym-RAG)

Metode NeuSym-RAG memadukan model neural dengan basis pengetahuan simbolik berbasis aturan di dalam basis data relasional (Cao dkk. 2025). Pendekatan ini menyimpan entitas dalam tabel terstruktur dan mengeksekusi aturan simbolik untuk memvalidasi hasil inferensi LLM secara deterministik, dengan akurasi validasi mencapai lebih dari 90% pada dataset *question answering* terstruktur (Cao dkk. 2025).

Namun, skema penyimpanan relasional memiliki kelemahan struktural jika diaplikasikan pada domain Kubernetes. Dengan 730 definisi skema dan rata-rata 4,1 *field* per definisi, menelusuri relasi hierarkis bersarang (*multi-hop*) membutuhkan operasi *join* SQL yang kompleks. Selain itu, aturan simbolik harus didefinisikan secara manual untuk setiap relasi dan diperbarui setiap siklus rilis Kubernetes (rata-rata setiap 4 bulan), menjadikan pemeliharaan tidak praktis dalam skala produksi.

III.4 Analisis Pemilihan Solusi

Bagian ini bertujuan menentukan pendekatan pemecahan masalah yang paling sesuai untuk membangun sistem *Graph-based Retrieval* pada dokumentasi API Kubernetes. Evaluasi dilakukan dengan membandingkan beberapa alternatif solusi berdasarkan kriteria yang disesuaikan dengan karakteristik konfigurasi *cloud-native* yang bersifat deklaratif, terstruktur, dan saling bergantung secara hierarkis.

III.4.1 Rubrik Evaluasi Solusi

Evaluasi dilakukan menggunakan skala 1 (Kurang Baik) hingga 5 (Sangat Baik), berdasarkan empat kriteria utama sebagai berikut:

K1. Kesesuaian Representasi Data

Mengukur kemampuan metode dalam menangani data konfigurasi yang bersifat hierarkis dan memiliki *cross-reference* antar-objek Kubernetes (misalnya *Service* → *Pod* melalui *selector*).

K2. Kemampuan Generatif

Menilai kemampuan sistem menghasilkan berkas konfigurasi YAML yang baru dan valid, bukan sekadar menemukan lokasi informasi pada dokumentasi.

K3. Fleksibilitas Pemeliharaan

Mengukur kemudahan sistem dalam melakukan pembaruan pengetahuan ketika spesifikasi API Kubernetes berubah versi tanpa memerlukan *fine-tuning* ulang.

K4. Presisi Konteks

Menilai kemampuan sistem menekan halusinasi melalui pembatasan struktural yang tegas, terutama pada relasi antar-objek Kubernetes.

Tabel III.9 menunjukkan kategori penilaian untuk setiap skor pada rubrik evaluasi tersebut.

Tabel III.9 Rubrik Penilaian Kriteria Evaluasi Solusi

Skor	Kategori	Deskripsi
1	Kurang Baik	Tidak memenuhi kebutuhan domain Kubernetes secara fungsional maupun struktural.
2	Cukup Buruk	Hanya relevan sebagian dan berisiko tinggi menghasilkan kesalahan konfigurasi.
3	Cukup Baik	Dapat digunakan, tetapi memerlukan banyak penyesuaian untuk konteks Kubernetes.
4	Baik	Umumnya sesuai dan dapat diterapkan dengan penyesuaian minor.
5	Sangat Baik	Sangat sesuai dengan karakteristik masalah dan hampir tidak memerlukan modifikasi.

III.4.2 Matriks Keputusan & Justifikasi Penilaian

Penilaian dilakukan terhadap tiga pendekatan alternatif serta metode usulan. Hasil evaluasi ditampilkan pada Tabel III.10.

Tabel III.10 Matriks Perbandingan Solusi Berdasarkan Rubrik Evaluasi

Metode	K1	K2	K3	K4	Total
Hybrid KG–Vector RAG (Alt 1)	5	4	5	5	19
GNN-augmented GraphRAG (Alt 2)	5	3	4	5	17
NeuSym-RAG (Alt 3)	3	2	3	4	12

Berdasarkan hasil penilaian kuantitatif pada Tabel III.10, diperlukan penjelasan lebih lanjut mengenai dasar pemberian skor pada setiap kriteria. Berikut ini justifikasi kuantitatif terhadap nilai yang diberikan untuk setiap metode.

1. Hybrid KG–Vector RAG

- K1 = 5:** Mampu menggabungkan struktur relasional (KG) dan teks tidak terstruktur (*vector store*). Cocok untuk OpenAPI Kubernetes yang bersifat hierarkis dan memiliki *cross-reference*.
- K2 = 4:** Menghasilkan konteks yang lengkap sehingga LLM dapat menyusun berkas konfigurasi YAML baru dengan lebih tepat, meskipun tidak memiliki mekanisme pembatasan struktural seketat GNN.
- K3 = 5:** KG dapat diperbarui secara *incremental*, sedangkan *vector store* hanya memerlukan *re-embedding* teks tanpa melatih ulang model.
- K4 = 5:** Kombinasi *hard constraint*, *schema constraint*, dan *vector similarity* memberikan presisi konteks, serta terbukti meningkatkan EM dan CP pada studi.

2. GNN-augmented GraphRAG (G-Retriever)

- K1 = 5:** Representasi graf Kubernetes sangat sesuai diproses oleh GNN yang sensitif terhadap struktur topologi dan relasi *multi-hop* (*Deployment* → *PodTemplate* → *Container*).
- K2 = 3:** Kemampuan generatif bergantung pada subgraf hasil PCST. LLM menghasilkan jawaban berdasarkan subgraf, tetapi tidak menyusun struktur baru secara fleksibel.
- K3 = 4:** Meski tidak memerlukan *fine-tuning* ulang, proses *indexing* dan pembentukan *embedding* graf harus diulang ketika versi API berubah.
- K4 = 5:** Eksperimen menunjukkan validitas *node* meningkat dari 31%

menjadi 77%, dan validitas *edge* dari 12% menjadi 76%, menunjukkan kontrol presisi yang sangat tinggi.

3. NeuSym-RAG (Hybrid Neural-Symbolic)

- a. **K1 = 3**: Representasi tabel relasional kurang mampu menampung kedalaman hierarki OpenAPI tanpa melakukan banyak operasi *join* sehingga tidak sebaik KG atau GNN.
- b. **K2 = 2**: Aturan simbolik bersifat validasi, bukan generasi. Kemampuan LLM untuk membuat berkas konfigurasi YAML baru terbatas karena sistem lebih cocok untuk *consistency checking*.
- c. **K3 = 3**: Relational schema stabil, tetapi aturan simbolik harus diperbarui manual saat Kubernetes menambahkan spesifikasi baru.
- d. **K4 = 4**: Aturan deterministik (*symbolic rules*) memberi kontrol kuat terhadap halusinasi, tetapi kelenturannya tidak sekuat *constraint* pada KG-Vector atau pemodelan struktural pada GNN.

III.4.3 Analisis Pemilihan dan Penyesuaian (Selection & Adjustment)

Berdasarkan hasil evaluasi pada Tabel III.10, metode *Hybrid KG-Vector RAG* (Alt 1) memperoleh skor tertinggi di antara alternatif lain dan menjadi landasan utama dalam perancangan sistem. Pendekatan tersebut dipilih karena berhasil menggabungkan representasi relasional eksplisit dari KG dengan cakupan semantik luas dari *vector retrieval*. Hal ini relevan pada domain Kubernetes, yaitu satu objek Kubernetes seperti *Pod* tidak hanya memiliki struktur *rigid* tetapi juga semantik.

Namun, arsitektur Wan dkk. (2025) masih memiliki keterbatasan jika diterapkan langsung pada konteks API Kubernetes. Komponen *Semantic Alignment* pada penelitian tersebut sangat bergantung pada aturan manual (*domain constraints*) yang harus dituliskan eksplisit pada *prompt*. Pada Kubernetes, pendekatan ini tidak efisien karena spesifikasi OpenAPI v1.30 memuat 730 definisi skema dengan rata-rata 4,1 *field* per definisi dan 18 jenis relasi semantik. Menuliskan aturan *constraint* secara manual untuk setiap kombinasi relasi tersebut akan menghasilkan ratusan aturan yang harus diperbarui setiap rilis Kubernetes (4 bulan sekali), sehingga pendekatan ini tidak skalabel dalam jangka panjang.

Oleh karena itu, penelitian ini mengusulkan penyesuaian strategi sebagai berikut:

- A1. Pendekatan Wan dkk. (2025) : Validasi berbasis aturan manual di dalam *prompt* (*Constraint-based Semantic Alignment*).
- A2. Implementasi solusi Kubernetes : Validasi berbasis struktur otomatis melalui penelusuran graf (*Graph-Native Structural Traversal*).

Pendekatan ini menjadikan topologi *knowledge graph* sebagai validator bawaan. Jika relasi antar-objek Kubernetes ditemukan secara eksplisit pada graf (misalnya *Service* $\rightarrow$ *Deployment* melalui *selector*), maka konfigurasi tersebut dianggap valid tanpa perlu mendefinisikan aturan tambahan.

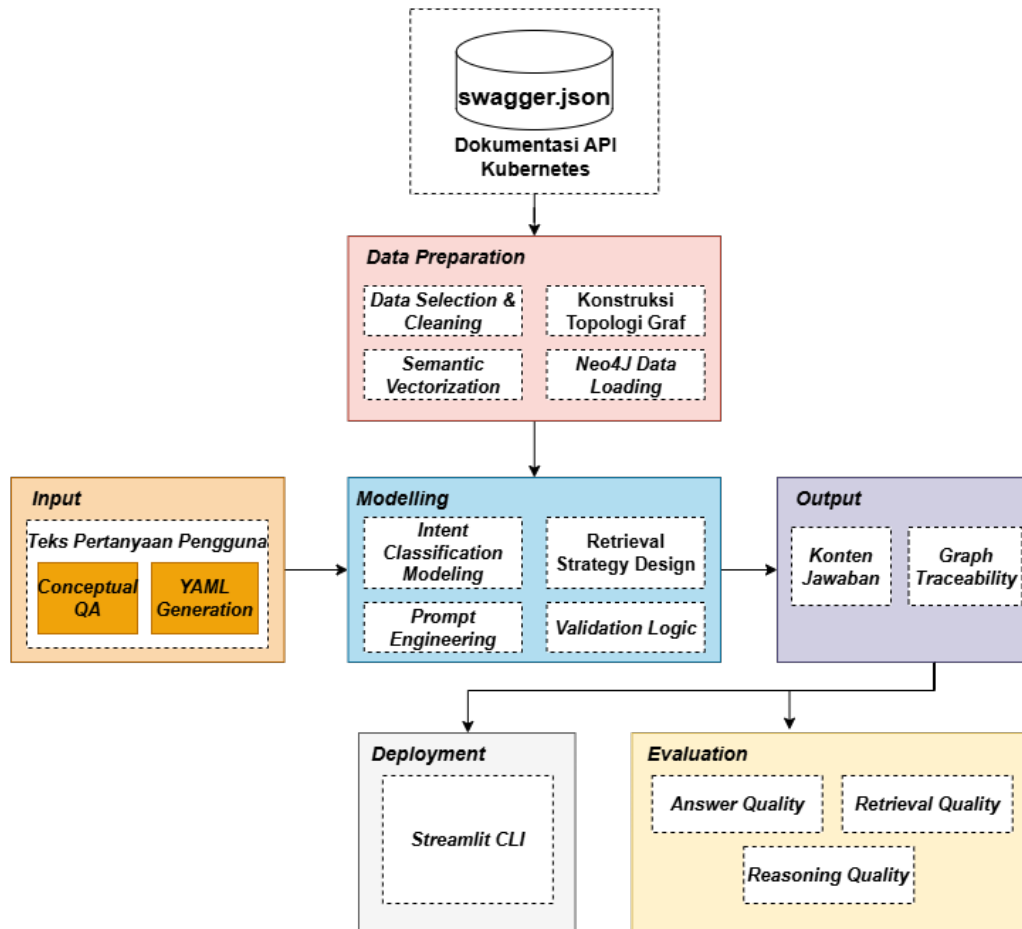
Kesimpulan: Solusi akhir yang dipilih adalah *GraphRAG with Structural Traversal*, yaitu adaptasi dari arsitektur Hybrid KG–Vector RAG yang menggantikan validasi aturan *hard constraints* dengan validasi relasional dari topologi graf. Pendekatan ini meningkatkan efisiensi pemeliharaan dan menekan risiko halusinasi konfigurasi Kubernetes.

BAB IV

PERANCANGAN SISTEM *GRAPHRAG* BERBASIS *KNOWLEDGE GRAPH* KUBERNETES

IV.1 Rancangan Solusi Secara Garis Besar

Berdasarkan hasil analisis karakteristik data, pola keterhubungan komponen Kubernetes, serta pemetaan kebutuhan fungsional dan non-fungsional yang telah dijelaskan pada bagian sebelumnya, langkah selanjutnya adalah merumuskan rancangan solusi yang akan dikembangkan dalam penelitian ini. Rancangan solusi ini disusun secara sistematis mengacu pada metodologi *Cross-Industry Standard Process for Data Mining* (CRISP-DM) sebagaimana ditampilkan pada Gambar IV.1.



Gambar IV.1 Rancangan solusi berdasarkan metodologi CRISP-DM

IV.2 Pemetaan Metodologi terhadap Kebutuhan Sistem

Tahapan metodologi penelitian sebagaimana digambarkan pada Gambar IV.1 menjadi landasan utama dalam perumusan kebutuhan sistem. Setiap aktivitas pada tahap *Data Preparation*, *Modelling*, *Input Processing*, *Output Generation*, *Evaluation*, dan *Deployment* dipetakan secara langsung terhadap kebutuhan fungsional yang harus dipenuhi oleh sistem. Pemetaan ini memastikan bahwa rancangan solusi GraphRAG tidak hanya bersifat konseptual, namun juga merefleksikan proses yang dibutuhkan untuk mencapai tujuan penelitian. Hubungan antara tahapan metodologi dan kebutuhan fungsional ditunjukkan pada Tabel IV.1.

Tabel IV.1 Pemetaan Tahap Metodologi

Tahap Metodologi	Aktivitas Utama	Kebutuhan yang Dipe- nuhi (F)
Data Preparation		
<i>Data Selection & Cleaning</i>	Identifikasi objek target dari spesifikasi OpenAPI Kubernetes, seleksi blok <i>definitions</i> , dan pembersihan tipe generik yang tidak relevan untuk pembuatan YAML.	F-01 (<i>Structured Object Representation</i>), F-05 (<i>Attribute Enrichment</i>)
<i>Semantic Vectorization</i>	Ekstraksi deskripsi objek menjadi <i>embedding</i> numerik untuk pencarian semantik.	F-05 (<i>Attribute Enrichment</i>)
Konstruksi Topologi Graf	Transformasi struktur JSON menjadi node-edge dengan relasi referensial dan hierarkis.	F-01 (<i>Structured Object Representation</i>), F-02 (<i>Structured Relation Retrieval</i>), F-05 (<i>Attribute Enrichment</i>)
Neo4j Data Loading	Memuat hasil konstruksi graf ke basis data Neo4j agar dapat ditelusuri kembali saat retrieval.	F-01 (<i>Structured Object Representation</i>)
Modelling		
<i>Intent Classification Modeling</i>	Mengklasifikasikan pertanyaan pengguna ke dalam lima tipe <i>intent</i> : explain dan generate_yaml (tipe utama), serta trace_relationship , followup , dan planning (mode pendukung). Setiap tipe menentukan kedalaman penelusuran graf yang digunakan.	F-03 (<i>Intent Classification</i>)

bersambung ke halaman berikutnya

Tabel IV.1 Pemetaan Tahap Metodologi (lanjutan)

Tahap Metodologi	Aktivitas Utama	Kebutuhan yang Dipe- nuhi (F)
<i>Retrieval Strategy Design</i>	Perancangan traversal graf berdasarkan entitas target, relasi, dan kebutuhan pengguna.	F-02 (<i>Structured Relation Retrieval</i>), F-04 (<i>Context Construction & Injection</i>)
<i>Prompt Engineering</i>	Perancangan template prompt untuk dua peran LLM: <i>Thinker</i> (ekstraksi intent) dan <i>Speaker</i> (generasi respons), serta injeksi konteks dan pengaturan format keluaran.	F-04 (<i>Context Construction & Injection</i>), F-07 (<i>Conditional YAML Generation Constraint</i>)
<i>Validation Logic</i>	Integrasi prosedur validasi YAML tiga lapis sebagai mekanisme <i>fail-safe</i> : sintaksis (<i>yaml.safe_load</i>), skema (<i>kubernetes-validate</i>), dan ketersediaan <i>required fields</i> via Neo4j.	F-07 (<i>Conditional YAML Generation Constraint</i>)
<i>Conversation Memory Management</i>	Penyimpanan dan pengambilan riwayat percakapan menggunakan SQLite melalui node <i>memory</i> (load riwayat di awal giliran) dan <i>saver</i> (simpan respons di akhir giliran) dalam <i>pipeline</i> LangGraph, guna mendukung intent follow-up yang membutuhkan konteks dari giliran sebelumnya.	F-03 (<i>Intent Classification</i>), F-04 (<i>Context Construction & Injection</i>)
Input		

bersambung ke halaman berikutnya

Tabel IV.1 Pemetaan Tahap Metodologi (lanjutan)

Tahap Metodologi	Aktivitas Utama	Kebutuhan yang Dipe- nuhi (F)
Pertanyaan Pengguna	Input teks berupa permin- taan teknis dalam bahasa alami yang diklasifikasikan ke salah satu tipe <i>intent</i> : explain , generate_yaml , tra- ce_relationship , followup , atau planning .	F-03 (<i>Intent Classifica- tion</i>)
Output		
Konten Jawaban	Keluaran berupa narasi pen- jelas (untuk <i>intent</i> explain , trace_relationship , follo- wup , dan planning) atau blok YAML konfigurasi Kubernetes (untuk <i>intent</i> generate_yaml).	F-07 (<i>Conditional YAML Generation Constraint</i>)
<i>Graph Traceability</i>	Penelusuran asal-usul <i>field</i> melalui relasi <i>node-edge</i> di dalam graf.	F-02 (<i>Structured Rela- tion Retrieval</i>)
Evaluation		
<i>Answer Quality</i> (An- sQ)	Evaluasi kualitas jawaban mencakup <i>Syntactic Validity</i> , <i>Schema Compliance</i> , <i>Answer Relevance</i> , dan <i>Faithfulness</i> .	– (Metrik domain, buk- an requirement fungsio- nal)
<i>Retrieval Quality</i> (Re- tQ)	Evaluasi kualitas <i>retrieval</i> mencakup <i>Precision@k</i> , <i>Recall@k</i> , <i>NDCG@k</i> , <i>Gra- ph Coverage</i> , dan <i>Edge Coverage</i> .	– (Metrik domain, buk- an requirement fungsio- nal)

bersambung ke halaman berikutnya

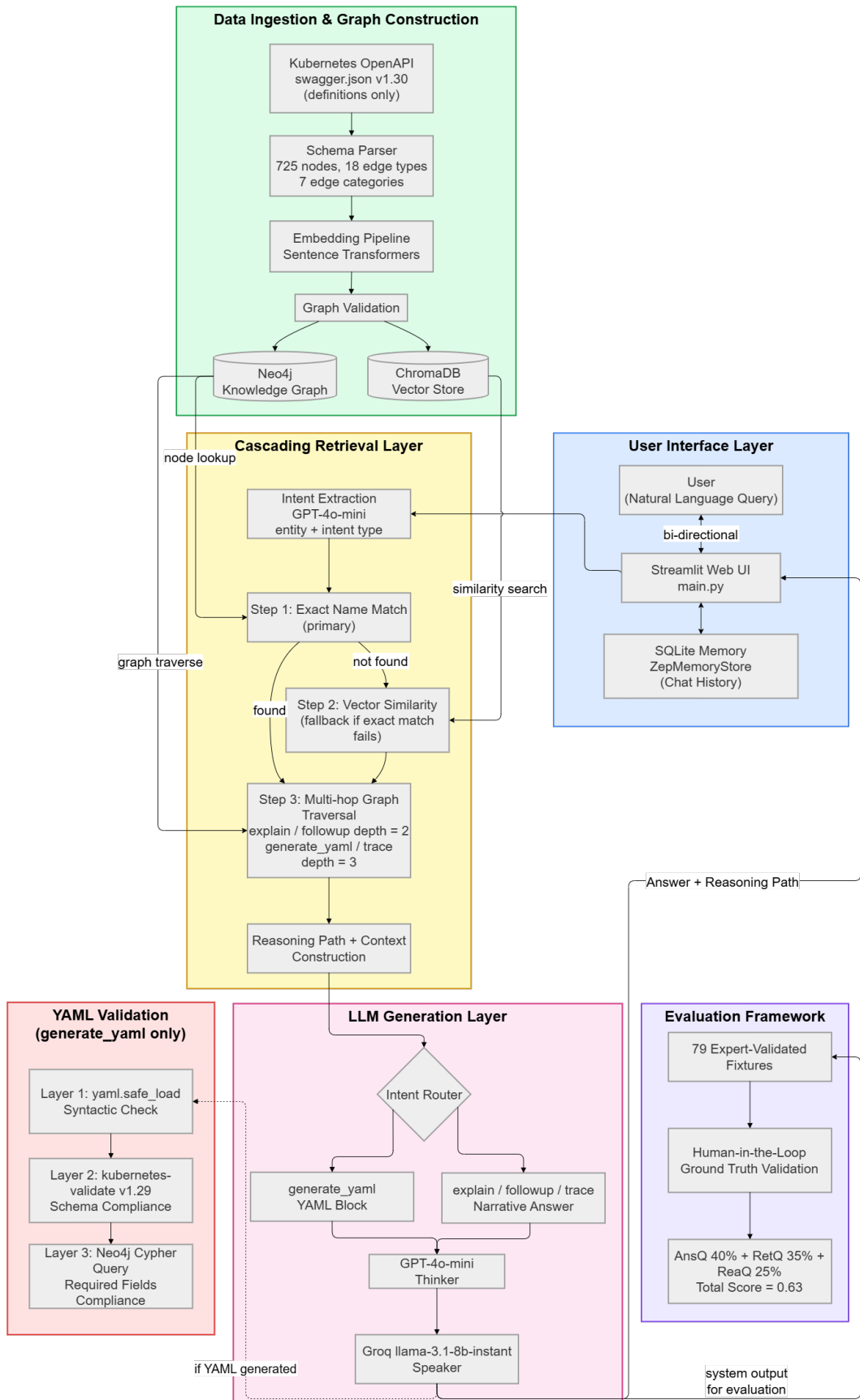
Tabel IV.1 Pemetaan Tahap Metodologi (lanjutan)

Tahap Metodologi	Aktivitas Utama	Kebutuhan yang Dipe- nuhi (F)
<i>Reasoning Quality</i> (ReaQ)	Evaluasi kualitas penalaran mencakup <i>Hop Accuracy</i> , <i>Multi-Hop Success</i> , <i>Scope Accuracy</i> , <i>Grounding Score</i> , dan <i>Hallucination Rate</i> .	– (Metrik domain, bukan requirement fungsional)
Deployment		
Antarmuka Web Interaktif (Streamlit)	Penyajian sistem melalui antarmuka web berbasis Streamlit (<code>main.py</code>) yang menampilkan <i>chat interface</i> , jawaban LLM, blok YAML (jika relevan), serta visualisasi <i>Retrieval Trace</i> berbasis Graphviz.	F-06 (<i>Web Interaction Interface</i>)

Pemetaan pada Tabel IV.1 menunjukkan bahwa seluruh tahapan metodologis yang digunakan dalam penelitian ini memiliki keterhubungan langsung dengan kebutuhan fungsional yang dirancang. Setiap proses dalam *Data Preparation*, *Modelling*, *Input Processing*, *Output Generation*, *Evaluation*, dan *Deployment* telah diterjemahkan menjadi kapabilitas sistem yang operasional, terukur, dan selaras dengan tujuan penelitian.

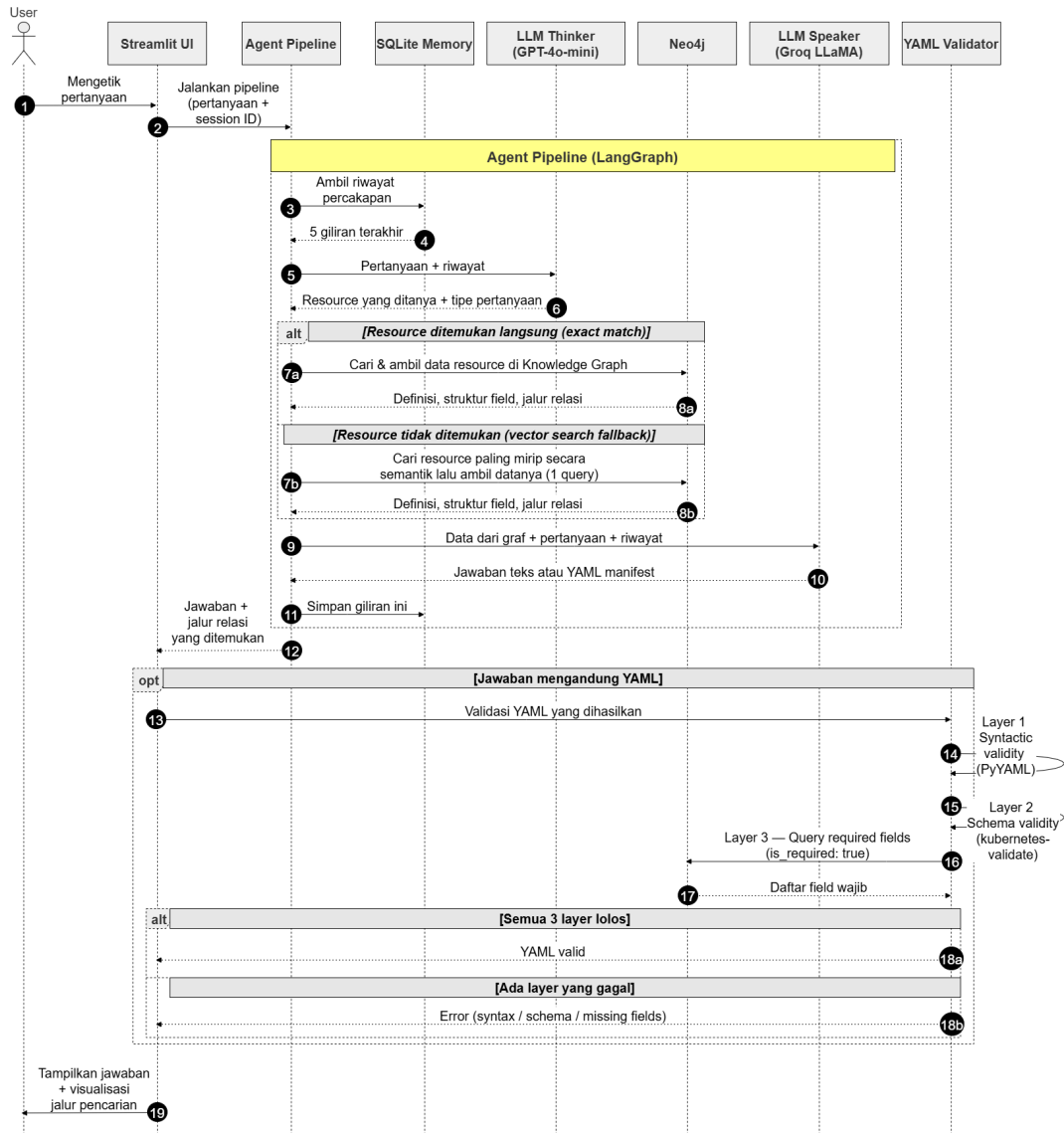
Namun, untuk memastikan bahwa seluruh fungsi tersebut dapat berjalan secara terintegrasi, dibutuhkan rancangan alur teknis yang sistematis dan dapat diimplementasikan secara konsisten melalui setiap tahapan eksekusi. Oleh karena itu, diperlukan suatu *pipeline arsitektural* yang bekerja dari awal hingga akhir. *Pipeline* ini tidak hanya memvisualisasikan interaksi antar modul, tetapi juga menunjukkan dependensi logis yang mengikat sistem komputasi yang utuh.

Dengan demikian, perancangan *architecture pipeline* berikut disusun untuk menyajikan gambaran menyeluruh mengenai tahapan proses, struktur komponen, aliran data (*data flow*), serta kontrol logika untuk setiap kebutuhan fungsional .



Gambar IV.2 *Architecture Diagram* Solusi GraphRAG untuk Mitigasi Halusinasi pada Sistem RAG-LLM

Setelah mendefinisikan komponen-komponen utama dalam arsitektur sistem, langkah selanjutnya adalah memetakan interaksi antar-komponen tersebut secara kronologis. Untuk menggambarkan aliran data diproses mulai dari input pengguna hingga validasi output, Gambar IV.3 menyajikan *Sequence Diagram* yang merinci mekanisme pertukaran informasi pada sistem.



Gambar IV.3 Diagram Urutan (*Sequence Diagram*) Alur Kerja Sistem GraphRAG

IV.3 Perancangan *Pipeline* Ekstraksi dan *Knowledge Graph*

Arsitektur pipeline pada Gambar IV.2 diimplementasikan melalui empat komponen teknis utama yang bekerja secara berurutan: pipeline ingestion data, mekanisme *LangGraph agent*, retrieval bertingkat, serta validasi YAML. Berikut ini diuraikan rancangan masing-masing komponen beserta keterhubungannya terhadap kebutuh-

an fungsional sistem.

IV.3.1 Pipeline Ingestion Data

Pipeline *ingestion* bertanggung jawab mengubah spesifikasi OpenAPI Kubernetes (*swagger.json*) menjadi *Knowledge Graph* yang tersimpan di Neo4j, lengkap dengan *Native Vector Index* untuk pencarian semantik. Proses ini dilaksanakan melalui lima *pass* yang dieksekusi secara berurutan sebagaimana ditunjukkan pada Tabel IV.2.

Tabel IV.2 Lima Fase Pipeline Ingestion Data

Fase	Nama	Aktivitas	Output
Pass 1	<i>Node Creation</i>	Membuat node definisi dari setiap entri <i>definitions</i> <i>swagger.json</i> ; menetapkan atribut <i>name</i> , <i>kind</i> , <i>scope</i> , <i>description</i> .	725 <i>node definition</i> di Neo4j
Pass 1.5	<i>Embedding Generation</i>	Menghasilkan vektor 1536-dimensi (<i>text-embedding-3-small</i>) dari deskripsi setiap node.	<i>Native Vector Index</i> di Neo4j
Pass 2	<i>Structural Edges</i>	Membangun edge <i>HAS_PROPERTY</i> berdasarkan resolusi <i>\$ref</i> antar-definisi.	4 jenis <i>structural edge</i>
Pass 2.5	<i>Inheritance Edges</i>	Membangun edge <i>EXTENDS</i> , <i>ONE_OF</i> , <i>ANY_OF</i> dari blok <i>allOf/oneOf/anyOf</i> .	Edge hierarki skema
Pass 3	<i>Semantic Edges</i>	Membangun 14 jenis <i>semantic edge</i> berdasarkan pola YAML Kubernetes (contoh: <i>CONTAINS_POD_TEMPLATE</i> , <i>USES_SECRET</i>).	14 jenis <i>semantic edge</i>

Urutan *pass* ini bersifat deterministik dan tidak dapat dibalik: node harus ada sebe-

lum edge dapat dibuat (Pass 1 mendahului Pass 2), dan *embedding* harus dihasilkan sebelum *vector index* dapat digunakan pada tahap *retrieval*. Total edge yang dihasilkan mencakup 18 jenis yang terdistribusi dalam 7 kategori relasi sebagaimana dirancang pada Bab III.

IV.4 Perancangan Mekanisme *GraphRAG*

IV.4.1 Perancangan *LangGraph Agent Pipeline*

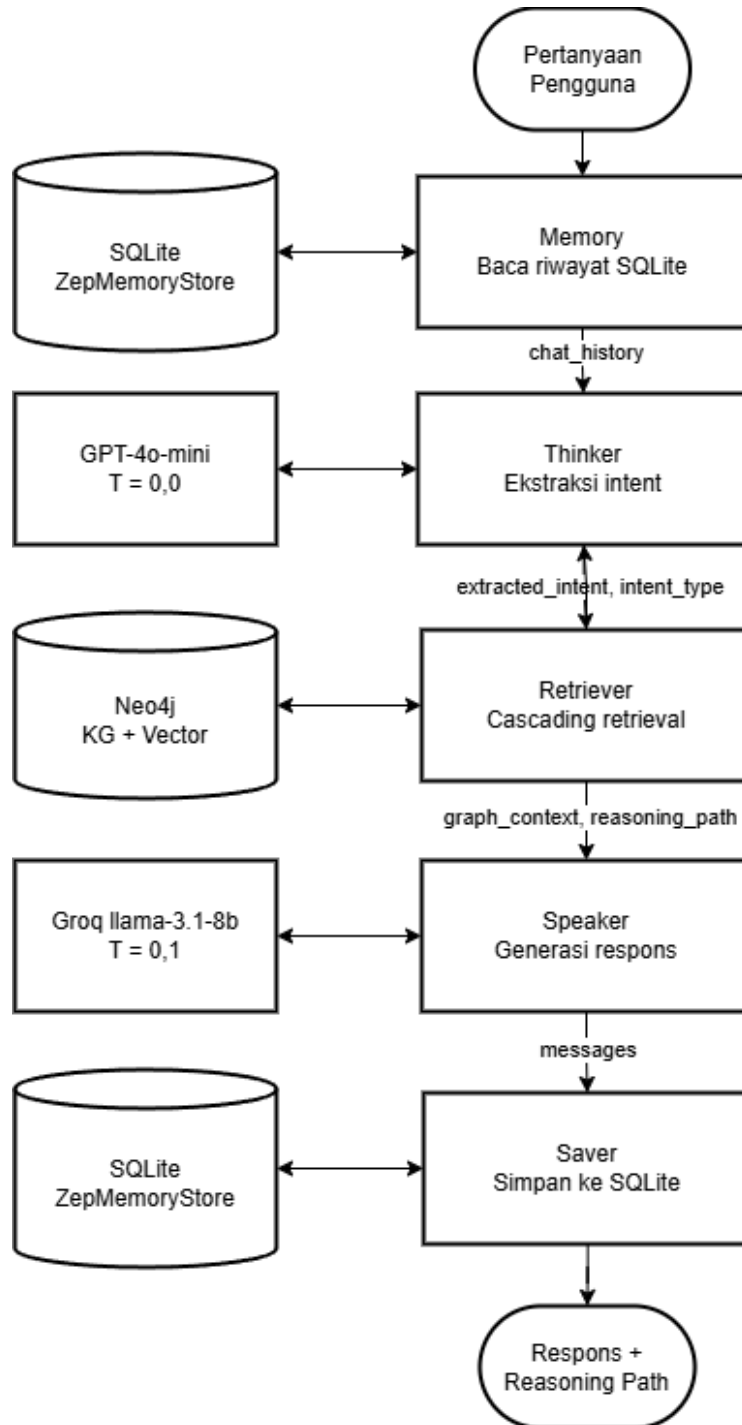
Komponen *chatbot* dirancang menggunakan *LangGraph* sebagai *state machine* yang mengorkestrasi alur pemrosesan dari input pengguna hingga respons akhir. *LangGraph* dipilih karena mendukung pengelolaan *state* yang persisten antar-node dan kontrol alur yang eksplisit. Kedua properti ini penting untuk memastikan bahwa setiap tahap (ekstraksi intent, retrieval, dan generasi) dapat diaudit secara independen.

Pipeline terdiri dari lima node yang dieksekusi secara linear sebagaimana ditunjukkan pada Tabel IV.3.

Tabel IV.3 Lima Node LangGraph Agent Pipeline

Node	Fungsi	Aktivitas	Komponen
<i>memory</i>	Ambil riwayat	Membaca 5 giliran percakapan terakhir dari penyimpanan SQLite.	SQLite (<i>ZepMemoryStore</i>)
<i>thinker</i>	Ekstraksi <i>intent</i>	Menganalisis pertanyaan + riwayat, mengeluarkan JSON berisi <i>primary_resource</i> , <i>related_concepts</i> , dan <i>intent_type</i> .	GPT-4o-mini
<i>retriever</i>	Eksekusi <i>retrieval</i>	Meneruskan JSON intent ke <i>StatefulK8sRetriever</i> untuk retrieval bertingkat; menghasilkan <i>graph_context</i> dan <i>reasoning_path</i> .	<i>StatefulK8sRetriever</i>
<i>speaker</i>	Generasi respons	Menggunakan konteks graf dan <i>reasoning_path</i> untuk menghasilkan jawaban naratif atau blok YAML.	Groq llama-3.1-8b-instant
<i>saver</i>	Simpan riwayat	Menyimpan pasangan pertanyaan–jawaban ke SQLite untuk giliran percakapan berikutnya.	SQLite (<i>ZepMemoryStore</i>)

Interaksi antar-*node* beserta aliran data melalui *AgentState* ditunjukkan pada Gambar IV.4. Setiap panah berlabel *field AgentState* yang diteruskan dari satu *node* ke *node* berikutnya, sehingga ketergantungan data antar-tahap dapat dilacak secara eksplisit.



Gambar IV.4 Diagram Interaksi Lima *Node LangGraph Agent Pipeline*

Pembagian peran *Thinker–Speaker* memanfaatkan model yang sama (GPT-4o-mini) untuk kedua peran: (1) *Thinker* menggunakan GPT-4o-mini dengan temperature=0,0 untuk menghasilkan keluaran JSON yang deterministik; (2) *Speaker* juga menggunakan GPT-4o-mini sehingga pemahaman skema Kubernetes konsisten antara tahap ekstraksi *intent* dan tahap generasi jawaban. Penggunaan model yang sama menjaga

kohesi semantik sekaligus menyederhanakan dependensi infrastruktur.

IV.4.2 Perancangan Mekanisme *Retrieval* Bertingkat

Node *retriever* mengimplementasikan *cascading retrieval* dengan tiga tahap berurutan. Tahap berikutnya hanya dieksekusi apabila tahap sebelumnya tidak menghasilkan hasil yang memadai, sehingga meminimalkan beban komputasi sekaligus memaksimalkan presisi.

- 1. Tahap 1: *Exact Name Match* (Prioritas Utama).**

Sistem mencari node di Neo4j berdasarkan kecocokan nama eksak dengan entitas yang diekstrak oleh *Thinker*. Jika ditemukan, node ini menjadi *seed node* untuk traversal graf. Pendekatan ini menghasilkan presisi tertinggi karena tidak bergantung pada estimasi semantik.

- 2. Tahap 2: *Vector Similarity* (Fallback).**

Diaktifkan *hanya jika* Tahap 1 tidak menemukan kecocokan eksak. Sistem menghasilkan *embedding* dari kueri pengguna dan mencari *node* paling mirip menggunakan *Native Vector Index* di Neo4j. *Top-k* kandidat menjadi *seed node* untuk traversal.

- 3. Tahap 3: *Multi-hop Graph Traversal*.**

Dari *seed node*, sistem menelusuri graf di Neo4j secara rekursif. Kedalaman maksimal traversal ditentukan oleh *intent type* yang diklasifikasikan pada node *thinker* sebagaimana ditunjukkan pada Tabel IV.4.

Tabel IV.4 Pemetaan *Intent Type* terhadap Kedalaman Traversal Graf

<i>Intent Type</i>	Depth	Alasan
<i>explain</i>	2	Cukup untuk menjawab definisi dan cara kerja <i>resource</i> ; depth 3+ menambahkan <i>noise</i> generik.
<i>followup</i>	2	Pertanyaan lanjutan biasanya merujuk entitas yang sama; konteks depth 2 sudah tersedia dari giliran sebelumnya.
<i>generate_yaml</i>	3	Pembuatan YAML membutuhkan field tingkat kontainer (<i>image, ports, env</i>) yang berada di depth 3.
<i>trace_relationship</i>	3	Penelusuran relasi antar- <i>resource</i> menjangkau jembatan lintas- <i>resource</i> pada depth 2–3.
<i>planning</i>	3	Pertanyaan perencanaan arsitektur membutuhkan konteks dari hingga dua <i>related_concepts</i> secara bersamaan; kedalaman 3 diperlukan untuk menjangkau komponen spesifikasi yang dibutuhkan.

Keputusan desain kedalaman pada Tabel IV.4 didasarkan pada analisis distribusi skema OpenAPI pada Bab III. Validasi empiris atas keputusan ini—yang membuktikan bahwa tidak ada kedalaman tetap tunggal yang optimal untuk seluruh tipe *intent*—disajikan pada Subbab VI.6.1.

Setelah traversal selesai, sistem membangun *reasoning path* berupa daftar string dengan format Parent - [REL] -> Child yang merepresentasikan jalur relasional yang dilalui. *Reasoning path* ini disertakan dalam *prompt* LLM dan ditampilkan kepada pengguna sebagai bukti penalaran yang dapat diverifikasi.

IV.5 Perancangan Validasi YAML Tiga Lapis

Validasi YAML dieksekusi secara kondisional: hanya dipicu apabila *intent* pengguna diklasifikasikan sebagai **generate_yaml**. Proses validasi terdiri dari tiga lapisan yang dieksekusi secara berurutan; lapisan berikutnya dilewati apabila lapisan sebelumnya sudah menemukan kegagalan.

Tabel IV.5 Rancangan Tiga Lapis Validasi YAML

Lapis	Alat	Jenis Pemeriksaan	Aksi jika Gagal
1	<code>yaml.safe_load</code>	Sintaksis YAML (format, indentasi, tipe data)	Hentikan validasi; kembalikan <code>syntax_errors</code>
2	<code>kubernetes-validate v1.29</code>	Kepatuhan skema (nama <i>field</i> , tipe, struktur)	Tambahkan <code>schema_errors</code> ; lanjut ke Lapis 3
3	Kueri Cypher Neo4j	Ketersediaan <i>required fields</i> (<code>is_required: true</code>)	Tambahkan <code>missing_fields</code> ; tandai YAML tidak valid

Lapis 3 merupakan kontribusi penelitian ini. Alih-alih mengandalkan *dry-run* ke kluster Kubernetes (yang memerlukan infrastruktur nyata), pemeriksaan *required fields* dilakukan langsung terhadap *Knowledge Graph* yang telah dibangun pada tahap *ingestion*. Apabila salah satu lapis mendeteksi kegagalan, antarmuka menampilkan peringatan Warning: Potential Syntax Error sesuai spesifikasi NF-01.

BAB V

IMPLEMENTASI SISTEM *GRAPH*RAG BERBASIS *KNOWLEDGE GRAPH* KUBERNETES

V.1 Lingkungan Implementasi

Implementasi sistem *GraphRAG* Kubernetes dilaksanakan menggunakan bahasa pemrograman Python 3.11 pada sistem operasi Windows 11. Seluruh dependensi dikelola melalui *virtual environment* dan didefinisikan dalam berkas `requirements.txt`. Tabel V.1 merangkum komponen utama beserta perannya dalam sistem.

Tabel V.1 Dependensi Utama Implementasi Sistem GraphRAG Kubernetes

Komponen	Versi Minimum	Fungsi dalam Sistem
LangGraph	0.0.20	Orkestrasi <i>state machine</i> pipeline agent (5 node)
LangChain	0.1.0	Abstraksi integrasi LLM dan <i>prompt template</i>
Neo4j Python Driver	5.17.0	Koneksi dan eksekusi query Cypher ke <i>knowledge graph</i>
LangChain-OpenAI	0.0.5	Integrasi GPT-4o-mini untuk node <i>Thinker</i>
LangChain-Groq	0.0.1	Integrasi Groq <i>llama-3.1-8b-instant</i> untuk node <i>Speaker</i>
OpenAI SDK	—	Generasi <i>embedding text-embedding-3-small</i> (1.536 dimensi)
Streamlit	1.30.0	Antarmuka web percakapan berbasis <i>browser</i>
PyYAML	6.0	Validasi sintaksis YAML (Lapis 1)
kubernetes-validate	0.15.0	Validasi kepatuhan skema Kubernetes v1.29 (Lapis 2)
SQLite	<i>stdlib</i>	Penyimpanan riwayat percakapan antar-sesi

Sistem memanfaatkan **OpenAI GPT-4o-mini** untuk kedua peran LLM. *Thinker* menggunakan GPT-4o-mini untuk mengekstraksi *intent* kueri ke dalam format JSON terstruktur berkat kemampuan pemahaman konteks yang tinggi dan dukungan *structured output*. *Speaker* juga menggunakan GPT-4o-mini untuk menghasilkan respons naratif; penggunaan model yang sama dipilih guna memastikan konsistensi pemahaman skema Kubernetes antara tahap ekstraksi *intent* dan tahap generasi jawaban.

Untuk penyimpanan percakapan, sistem menggunakan **SQLite** lokal yang diakses melalui kelas *ZepMemoryStore* di `src/memory/zep_store.py`. Keputusan ini di-

ambil karena Zep Cloud versi 1 memanggil LLM internal untuk ekstraksi entitas yang menambah latensi dan biaya token tanpa nilai tambah bagi penelitian ini. SQLite memberikan persistensi yang memadai tanpa ketergantungan infrastruktur tambahan.

V.2 Implementasi *Pipeline* Ekstraksi dan *Knowledge Graph*

V.2.1 Implementasi *Ingestion Data* dan *Knowledge Graph*

Konstruksi *knowledge graph* dilaksanakan melalui eksekusi kelas *SwaggerGraphBuilder* yang terdefinisi di `src/ingestion/parser.py` terhadap berkas `swagger.json` Kubernetes versi 1.30. Pipeline *ingestion* dijalankan secara berurutan dalam lima fase sebagaimana telah dirancang pada Tabel IV.2.

Hasil eksekusi pipeline menghasilkan *knowledge graph* yang tersimpan di Neo4j dengan statistik sebagaimana ditampilkan pada Tabel V.2.

Tabel V.2 Statistik *Knowledge Graph* Kubernetes Hasil Konstruksi

Properti	Nilai
Sumber data	<code>swagger.json</code> Kubernetes v1.30
Total node <i>Definition</i>	725
Total jenis relasi (<i>edge type</i>)	18
Kategori relasi	7 (Struktural, <i>Workload</i> , Penyimpanan, Konfigurasi, Jaringan, RBAC, <i>Autoscaling</i>)
Model <i>embedding</i>	<i>text-embedding-3-small</i> (OpenAI)
Dimensi vektor <i>embedding</i>	1.536
Penyimpanan <i>vector index</i>	Neo4j Native Vector Index (kemiripan <i>cosine</i>)

Vektor *embedding* dihasilkan menggunakan model *text-embedding-3-small* dari OpenAI dengan dimensi 1.536 dan disimpan langsung sebagai properti *embedding* pada setiap node *Definition* di Neo4j. Pendekatan ini memungkinkan *vector similarity search* dilakukan dalam satu query Cypher tanpa kebutuhan *vector store* eksternal terpisah, menggunakan *Native Vector Index* Neo4j dengan fungsi kemiripan *cosine*.

Proses seleksi node dilakukan dengan mengecualikan 14 tipe definisi generik, antara

lain *ObjectMeta* dan *ManagedFieldsEntry*, yang tidak merepresentasikan sumber daya Kubernetes primer. Seleksi ini menghasilkan 725 node yang secara langsung berkaitan dengan spesifikasi *workload* dan konfigurasi Kubernetes.

V.3 Implementasi Mekanisme *GraphRAG*

V.3.1 Implementasi *Pipeline* LangGraph

Pipeline chatbot diimplementasikan sebagai *directed acyclic graph* (DAG) menggunakan LangGraph pada berkas `src/chatbot/graph_agent.py`. *State* yang dipertukarkan antar-node didefinisikan sebagai *TypedDict* dengan nama *AgentState* yang memuat sembilan *field*: *messages*, *question*, *session_id*, *chat_history*, *extracted_intent*, *graph_context*, *reasoning_path*, *intent_type*, dan *error*. Lima node yang dieksekusi secara linear sebagaimana dirancang pada Tabel IV.3 diimplementasikan masing-masing sebagai fungsi Python yang menerima dan mengembalikan *AgentState*.

Arsitektur *dual-LLM* diimplementasikan dengan konfigurasi *temperature* yang berbeda untuk setiap model. Node *thinker* menggunakan GPT-4o-mini dengan *temperature*=0,0 untuk menghasilkan keluaran JSON yang deterministik tanpa variasi *sampling*; kesetaraan ini krusial karena keluaran JSON yang tidak konsisten akan menyebabkan kegagalan *parsing* pada node *retriever*. Node *speaker* menggunakan GPT-4o-mini dengan *temperature*=0,1, dipilih sebagai titik keseimbangan antara *faithfulness* terhadap konteks graf dan kepatuhan terhadap skema Kubernetes yang dihasilkan.

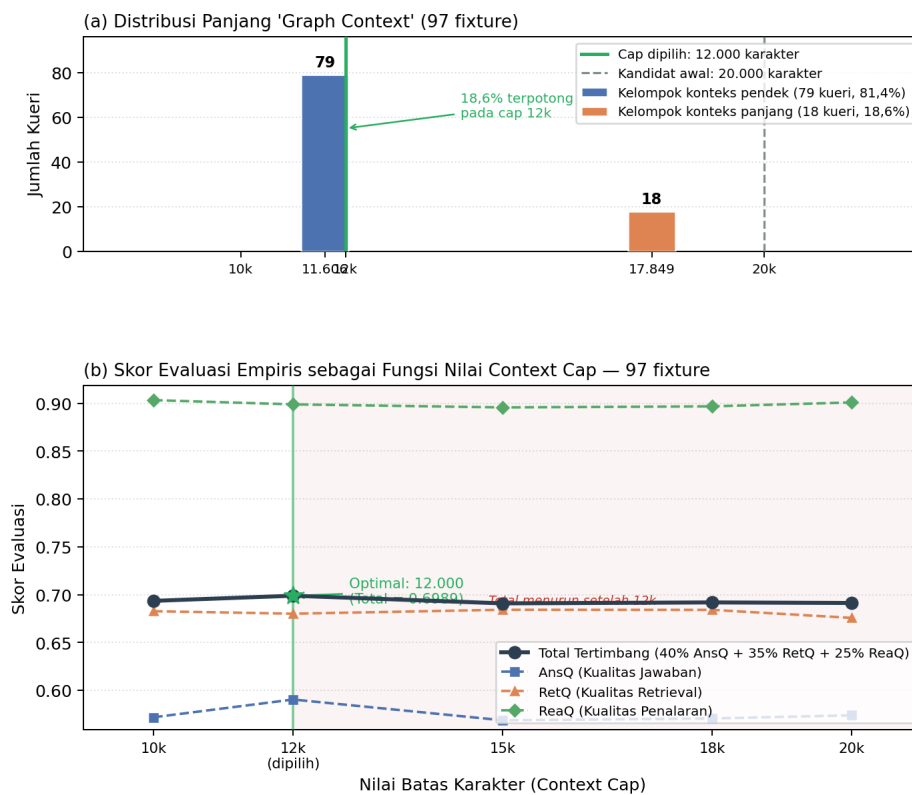
Untuk memenuhi kendala latensi (NF-02), *graph context* yang disampaikan ke node *speaker* dibatasi maksimum 12.000 karakter melalui pemotongan sisi kanan (*right-truncation*). Nilai batas ini dipilih melalui dua tahap analisis yang saling melengkapi, sebagaimana ditunjukkan pada Gambar V.1.

Tahap pertama adalah analisis distribusi panjang *graph context* terhadap seluruh 97 *fixture* evaluasi menggunakan mekanisme *dry-run retriever*. Analisis tersebut mengungkap distribusi bimodal: 79 kueri (81,4%) menghasilkan konteks sepanjang 11.606 karakter dan 18 kueri (18,6%) menghasilkan 17.849 karakter—sesuai dengan kedalaman traversal *intent-adaptive* yang berbeda (lihat Panel (a)).

Tahap kedua adalah evaluasi empiris pada lima nilai batas yang berbeda—10.000, 12.000, 15.000, 18.000, dan 20.000 karakter—dengan mengeksekusi seluruh 97 *fixture* evaluasi secara penuh pada setiap nilai batas (lihat Panel (b)). Hasil evaluasi menunjukkan bahwa nilai 12.000 karakter menghasilkan *Total* tertimbang tertinggi

sebesar 0,6989, mengungguli semua kandidat lainnya. Batas yang lebih besar (15k–20k) mengakibatkan penurunan *faithfulness* pada submetrik AnsQ, yang diidentifikasi sebagai dampak dari konteks yang terlalu panjang sehingga model *speaker* kehilangan fokus pada dokumen referensi. Sebaliknya, batas 10.000 menghasilkan ReaQ yang lebih tinggi tetapi AnsQ yang lebih rendah karena konteks yang tidak memadai untuk generasi jawaban yang lengkap. GPT-4o-mini mendukung jendela konteks 128.000 token sehingga batas ini tidak menjadi kendala kapasitas melainkan semata sebagai pengendali kualitas, biaya, dan latensi.

Analisis Pemilihan Batas Karakter Graph Context



Gambar V.1 Analisis pemilihan batas karakter *graph context*. Panel (a) menunjukkan distribusi bimodal panjang konteks pada 97 *fixture*: 79 kueri (81,4%) menghasilkan 11.606 karakter dan 18 kueri (18,6%) menghasilkan 17.849 karakter, dengan garis hijau menandai batas yang dipilih (12.000 karakter). Panel (b) menunjukkan skor evaluasi empiris (*Total* tertimbang, AnsQ, RetQ, ReaQ) sebagai fungsi nilai batas untuk lima kandidat yang diuji. Batas 12.000 karakter mencapai *Total* tertinggi (0,6989) dan ditetapkan sebagai nilai operasional.

V.3.2 Implementasi Mekanisme *Retrieval* Bertingkat

Mekanisme *retrieval* diimplementasikan dalam kelas *StatefulK8sRetriever* pada berkas `src/chatbot/custom_retriever.py`. *Method* utama `retrieve_context()`

menerima parameter `intent_data` (hasil ekstraksi Thinker), `intent_type` (string jenis *intent*), dan `max_depth` (opsional) untuk menghasilkan pasangan (`graph_context_json`, `reasoning_path`).

Pemetaan kedalaman berbasis *intent* sebagaimana dirancang pada Tabel IV.4 diimplementasikan sebagai kamus `_DEPTH_BY_INTENT` di tingkat modul. Prioritas resolusi kedalaman adalah: (1) argumen `max_depth` yang diberikan secara eksplisit, (2) nilai dari `_DEPTH_BY_INTENT` berdasarkan `intent_type`, dan (3) nilai *default* tiga lompatan.

Pada Fase 1, sistem mengeksekusi `EXACT_MATCH_QUERY` ke Neo4j menggunakan nama sumber daya yang diekstraksi oleh Thinker. Apabila ditemukan, node yang cocok digunakan sebagai *seed node* untuk traversal via `SCHEMA_DEPS_QUERY`. Pada Fase 2—yang hanya diaktifkan jika Fase 1 tidak menghasilkan kecocokan—sistem menghasilkan *embedding* dari kueri menggunakan *text-embedding-3-small* dan mengeksekusi `HYBRID_VECTOR_GRAPH_QUERY`, yaitu query Cypher tunggal yang menggabungkan *vector similarity search* dengan ekspansi graf di Neo4j. String yang di-embed merupakan konkatenasi dari *primary_resource*, seluruh elemen *related_concepts*, dan kata kunci “Kubernetes”, sehingga kemiripan semantik yang dicari mencakup konteks multi-sumber daya secara implisit. Apabila Fase 2 juga tidak menghasilkan kecocokan, *method* mengembalikan pesan *fallback* tanpa melanjutkan traversal.

Untuk *intent* yang melibatkan lebih dari satu sumber daya—yaitu *planning*, *trace_relationship*, dan *generate_yaml*—implementasi menambahkan *loop* tambahan yang mengambil konteks dari hingga dua *related_concepts* yang diekstraksi Thinker dan menggabungkan hasilnya—baik *graph_context* maupun *reasoning_path*—dengan hasil dari *primary_resource*. Penanganan ini diperlukan karena pertanyaan relasional dan generasi YAML multi-komponen membutuhkan konteks skema dari seluruh entitas yang disebutkan, bukan hanya entitas utama. Deduplikasi diterapkan pada *reasoning_path* agar langkah traversal yang identik antarsumber daya tidak dicatat lebih dari satu kali.

Reasoning path dibangun oleh *method _build_reasoning_path()* melalui eksekusi `PATH_EDGES_QUERY` yang menghasilkan daftar *string* berformat "Parent-[REL]-> Child", merepresentasikan jalur traversal yang sesungguhnya dari graf—bukan pintasan langsung dari *root* ke *leaf*. Alur keseluruhan mekanisme ini ditunjukkan pada Gambar ??.

V.4 Implementasi Validasi YAML Tiga Lapis

Validasi YAML diimplementasikan dalam kelas *YAMLValidator* pada berkas `src/validation/yaml_` dan hanya diaktifkan secara kondisional ketika `intent_type` yang diklasifikasikan oleh node *thinker* adalah *generate_yaml*, sehingga menghindari *overhead* komputasi untuk pertanyaan penjelasan atau relasi.

Tiga lapisan validasi sebagaimana dirancang pada Tabel IV.5 diimplementasikan secara berurutan; lapisan berikutnya hanya dieksekusi apabila lapisan sebelumnya berhasil. Lapis pertama memanggil `yaml.safe_load()` dari pustaka PyYAML. Lapis kedua menggunakan `kubernetes-validate` dengan parameter versi Kubernetes 1.29 sebagai referensi skema. Lapis ketiga mengeksekusi `REQUIRED_FIELDS_QUERY` ke Neo4j untuk memverifikasi keberadaan *field* wajib berdasarkan *knowledge graph* yang telah dibangun pada tahap *ingestion*.

Keluaran validasi berupa objek *YAMLValidationResult* dengan empat *field*: `valid` (*Boolean*), `syntax_errors` (daftar kesalahan sintaksis PyYAML), `schema_errors` (daftar pelanggaran skema Kubernetes), dan `missing_fields` (daftar *field* wajib yang tidak ditemukan dalam YAML yang digenerate). Ketika salah satu lapisan mendeteksi kegagalan, antarmuka menampilkan peringatan `Warning: Potential Syntax Error` kepada pengguna.

BAB VI

EVALUASI

VI.1 Metode Evaluasi

Evaluasi sistem GraphRAG Kubernetes dilakukan menggunakan *dataset* yang terdiri dari 97 *fixture* uji yang mencakup delapan kategori pertanyaan: *conceptual* (15), *relationship* (18), *yaml_gen* (15), *followup* (12), *realworld* (24), *planning* (5), *troubleshooting* (5), dan *command* (3). Distribusi kategori ini mencerminkan spektrum penggunaan nyata berdasarkan hasil wawancara pakar sebagaimana diuraikan pada Subbab VI.2.

Evaluasi dilaksanakan dalam tiga dimensi yang masing-masing mengukur aspek berbeda dari kualitas sistem:

1. **Answer Quality (AnsQ, bobot 40%)**

Mengukur kualitas jawaban yang dihasilkan, mencakup empat metrik: *syntactic validity* (validitas sintaksis YAML yang digenerate), *schema compliance* (kesesuaian dengan skema Kubernetes v1.29), *answer relevance* (relevansi jawaban terhadap pertanyaan menggunakan kemiripan *cosine* antara *embedding* jawaban dan *ground truth*), serta *faithfulness* (proporsi node kunci dalam *ground truth* yang disebutkan secara eksplisit dalam jawaban).

2. **Retrieval Quality (RetQ, bobot 35%)**

Mengukur kualitas proses *retrieval*, mencakup: *precision@k* dan *recall@k* terhadap seluruh node yang dilalui selama traversal graf (*relevant_nodes*), *graph coverage* (cakupan node yang berhasil di-*retrieve* terhadap total node relevan), *NDCG@k* (*normalized discounted cumulative gain*), dan *edge coverage*.

3. **Reasoning Quality (ReaQ, bobot 25%)**

Mengukur kualitas penalaran berbasis graf, mencakup: *hop accuracy* (proporsi lompatan dalam *reasoning path* yang valid), *multi-hop success* (keberhasilan traversal multi-lompatan dari *seed node*), *grounding score* (proporsi istilah Kubernetes dalam jawaban yang berasal dari *retrieved context*), dan *hallucination rate* (komplemen dari *grounding score*).

Skor komposit dihitung menggunakan formula berbobot:

$$\text{Total Score} = 0,40 \times \text{AnsQ} + 0,35 \times \text{RetQ} + 0,25 \times \text{ReaQ} \quad (\text{VI.1})$$

Setiap *fixture* dijalankan secara otomatis melalui *script* `scripts/evaluate.py` yang menginisiasi sesi baru dengan *session ID* unik per-*fixture* untuk mencegah kontaminasi riwayat percakapan antar-*fixture*. Khusus untuk kategori *followup* (12 *fixture*), sebuah pertanyaan konteks (*context_question*) dieksekusi terlebih dahulu dalam *session* yang sama sebelum pertanyaan utama, guna mensimulasikan percakapan multi-giliran yang realistis dan mengaktifkan mekanisme *session memory* SQLite pada GraphRAG.

VI.2 Validasi Dataset oleh Pakar

Sebelum evaluasi kuantitatif dilaksanakan, dataset evaluasi divalidasi secara kualitatif melalui wawancara mendalam dengan tiga praktisi DevOps/SRE yang menggunakan Kubernetes dan LLM secara aktif dalam pekerjaan sehari-hari. Profil narasumber ditampilkan pada Tabel VI.1.

Tabel VI.1 Profil Narasumber Wawancara Validasi Dataset

Narasumber	Peran	Pengalaman Kubernetes	Frekuensi Penggunaan LLM
N1	<i>DevOps Engineer</i>	>5 tahun, klaster 50–200 <i>node</i>	Harian
N2	<i>Cloud & DevOps Engineer</i>	1–2 tahun, klaster menengah	Harian
N3	<i>Site Reliability Engineer</i>	6 bulan–1 tahun, klaster menengah	Mingguan

Wawancara dilaksanakan dalam dua segmen. Segmen pertama meminta narasumber untuk memberikan skor realisme (skala 1–5) terhadap 10 sampel *fixture* yang dipilih secara representatif, dengan kriteria: skor 1 berarti pertanyaan murni bersifat *textbook* yang tidak pernah diajukan ke LLM dalam praktik nyata, sedangkan skor 5 berarti pertanyaan persis mencerminkan pertanyaan dalam pekerjaan sehari-hari. Hasil penilaian ditampilkan pada Tabel VI.2.

Tabel VI.2 Penilaian Realisme Sampel *Fixture* oleh Pakar (Skala 1–5)

<i>Fixture</i>	N1	N2	N3	Rata-rata
<i>deployment_basic</i>	3	1	1	1,67
<i>service_types</i>	2	3	2	2,33
<i>scale_existing_deployment</i>	5	5	4	4,67
<i>ingress_service_pod</i>	3	4	2	3,00
<i>hpa_deployment_pod</i>	3	5	2	3,33
<i>cronjob_backup</i>	4	5	5	4,67
<i>networkpolicy_deny_all</i>	4	5	5	4,67
<i>statefulset_with_pvc</i>	4	5	5	4,67
<i>reject_all_docker_hub</i>	5	5	5	5,00
<i>kubectl_env_grep</i>	5	3	3	3,67
Rata-rata Keseluruhan				3,87

Penilaian menunjukkan variasi yang signifikan antar-*fixture*. *Fixture* yang bersifat definitif murni (*deployment_basic*, rata-rata 1,67) dinilai tidak realistis oleh semua narasumber karena praktisi yang sudah berpengalaman tidak menanyakan definisi dasar ke LLM. Sebaliknya, *fixture* berbasis skenario kongkret (*statefulset_with_pvc*, *networkpolicy_deny_all*, *reject_all_docker_hub*) mendapatkan skor tertinggi (4,67–5,00) karena mencerminkan kebutuhan operasional nyata.

Segmen kedua menggali tipe pertanyaan yang secara organik diajukan narasumber kepada LLM dalam pekerjaan sehari-hari. Ketiga narasumber secara konsisten menyebutkan **debug dan troubleshooting** sebagai penggunaan utama, diikuti oleh *generate* YAML dengan penyesuaian konteks (*namespace*, *resource limits*), serta perencanaan arsitektur Kubernetes sebelum menulis konfigurasi.

Berdasarkan temuan wawancara, tiga kategori baru ditambahkan ke dataset: *troubleshooting* (5 *fixture*) untuk skenario Pod bermasalah (*CrashLoopBackOff*, *OOMKilled*, *Pending*), *planning* (5 *fixture*) untuk pertanyaan perencanaan sumber daya dari sudut pandang kebutuhan aplikasi, dan *command* (3 *fixture*) untuk operasi *kubectl*. Selain itu, 5 *fixture followup* ditambahkan untuk memperluas cakupan skenario modifikasi inkremental, sehingga total dataset meningkat dari 79 menjadi 97 *fixture*.

VI.3 Hasil Evaluasi Kuantitatif

VI.3.1 Skor Keseluruhan

Sistem GraphRAG Kubernetes dievaluasi terhadap 97 *fixture* dan menghasilkan total skor komposit **0,6943** dengan rincian per dimensi dan sub-metrik sebagaimana ditampilkan pada Tabel VI.3.

Tabel VI.3 Skor Evaluasi per Dimensi dan Sub-Metrik (97 *Fixture*)

Dimensi (Bot)	Skor	Sub-Metrik	Nilai
AnsQ (40%)	0,57	Syntactic Validity	0,9474
		Schema Compliance	0,7895
		Answer Relevance	0,6682
		Faithfulness	0,4289
RetQ (35%)	0,69	Graph Coverage	0,8599
		Recall@k	0,5907
		NDCG@k	0,7503
		Precision@k	0,6705
ReaQ (25%)	0,90	Multi-Hop Success	1,0000
		Hop Accuracy	0,8969
		Grounding Score	0,7678
		Hallucination Rate	0,2322
Total Skor Komposit		0,6943	

Nilai *syntactic validity* sebesar 0,9474 menunjukkan bahwa sebagian besar YAML yang digenerate lolos validasi sintaksis PyYAML (14 dari 15 *fixture yaml_gen* serta *fixture* YAML pada kategori lain). *Multi-hop success* sebesar 1,0000 mengindikasikan bahwa traversal graf selalu berhasil mengakses setidaknya satu lompatan dari *seed node* yang ditemukan, membuktikan keandalan mekanisme *retrieval* bertingkat.

VI.3.2 Skor per Kategori *Fixture*

Tabel VI.4 menyajikan skor per kategori *fixture* untuk mengidentifikasi kekuatan dan kelemahan sistem pada berbagai jenis pertanyaan.

Tabel VI.4 Skor Evaluasi per Kategori *Fixture*

Kategori	<i>n</i>	AnsQ	RetQ	ReaQ	Total
<i>yaml_gen</i>	15	0,81	0,89	0,87	0,85
<i>relationship</i>	18	0,69	0,73	0,98	0,77
<i>conceptual</i>	15	0,64	0,73	0,93	0,74
<i>planning</i>	5	0,45	0,80	0,90	0,69
<i>command</i>	3	0,33	0,75	0,87	0,61
<i>followup</i>	12	0,53	0,92	0,91	0,76
<i>troubleshooting</i>	5	0,41	0,67	0,89	0,62
<i>realworld</i>	24	0,41	0,35	0,84	0,50
Keseluruhan	97	0,57	0,69	0,90	0,69

Kategori *yaml_gen* memperoleh skor tertinggi (0,85) karena spesifikasi YAML yang konkret memudahkan *retrieval* yang presisi dan evaluasi *faithfulness* yang objektif. Kategori *relationship* menunjukkan ReaQ tertinggi (0,98), mencerminkan efektivitas *multi-hop graph traversal* dalam menjawab pertanyaan relasional antar-objek Kubernetes. Kategori *followup* memperoleh RetQ tertinggi (0,92) karena mekanisme *session memory* memungkinkan konteks percakapan sebelumnya digunakan dalam *retrieval*; kategori *planning* memperoleh RetQ tertinggi kedua (0,80) berkat mekanisme *multi-resource retrieval* yang mengambil konteks dari hingga dua *related_concepts* secara bersamaan. Kategori *realworld* memperoleh skor terendah (0,50) karena pertanyaan berbasis konteks operasional nyata membutuhkan traversal yang lebih luas dari kemampuan *retrieval* kedalaman tiga lompatan pada *knowledge graph* berbasis spesifikasi OpenAPI.

VI.3.3 Analisis Metrik *Reasoning Quality*

Nilai *grounding_score* sebesar 0,7678 dan *hallucination_rate* sebesar 0,2322 mengindikasikan bahwa 23% istilah Kubernetes yang disebutkan dalam jawaban berasal dari pengetahuan bawaan model Speaker, bukan dari konteks yang di-*retrieve*. Hal ini merupakan karakteristik inheren dari model bahasa besar yang dilatih dengan dokumentasi Kubernetes ekstensif: model secara alami mengekstrapolasi melampaui konteks yang disediakan meskipun *temperature* diturunkan. Untuk konteks penelitian ini, nilai tersebut berarti bahwa 77% istilah teknis dalam jawaban dapat diverifikasi secara langsung melalui *reasoning path* yang ditampilkan di antarmuka.

Nilai *precision@k* (0,6705) dan *NDCG@k* (0,7503) mencerminkan keselarasan yang baik antara node yang di-*retrieve* oleh sistem dengan seluruh node yang dilalui selama traversal graf. *Recall@k* (0,5907) dan *graph coverage* (0,8599) mengkonfirmasi bahwa sistem *retrieval* berhasil menjangkau mayoritas node relevan, dengan sebagian kecil node yang terlewat karena batasan kedalaman traversal tiga lompatan.

VI.4 Pembahasan Hasil Evaluasi

Hasil evaluasi sistem GraphRAG Kubernetes dengan total skor komposit 0,6943 menunjukkan peningkatan yang signifikan dibandingkan pendekatan RAG berbasis vektor konvensional yang hanya mencapai *exact match* 63,4% dan *context precision* 69,8% sebagaimana disebutkan pada Bab I. Peningkatan ini terutama didorong oleh dimensi ReaQ (0,90) yang mencerminkan efektivitas *multi-hop graph traversal* dalam membangun rantai penalaran yang dapat diverifikasi dan dipresentasikan kepada pengguna.

Dimensi AnsQ (0,57) menunjukkan bahwa kualitas jawaban secara keseluruhan baik, dengan validitas sintaksis YAML yang tinggi (0,9474) sebagai kontribusi utama mekanisme validasi tiga lapis berbasis *knowledge graph*. Nilai *faithfulness* (0,4289) mengindikasikan bahwa model Speaker tidak selalu menyebutkan secara eksplisit semua node kunci yang menjadi dasar jawabannya, meskipun node-node tersebut tersedia dalam konteks yang disediakan.

Dimensi RetQ (0,69) mencerminkan kualitas *retrieval* yang baik, dengan *precision@k* (0,6705) dan *NDCG@k* (0,7503) yang menunjukkan keselarasan tinggi antara node yang di-*retrieve* dengan *ground truth* traversal. *Graph coverage* (0,8599) mengkonfirmasi bahwa sistem *retrieval* secara substansial berhasil menjangkau cakupan informasi yang relevan.

Sistem ini memiliki beberapa keterbatasan yang perlu diakui. Pertama, *hallucination rate* sebesar 0,2322 mencerminkan kebergantungan yang tidak terhindarkan pada pengetahuan bawaan model LLM yang melampaui batas konteks yang di-*retrieve*. Kedua, pertanyaan kategori *realworld* (skor 0,50) yang mencakup konteks operasional spesifik menunjukkan bahwa sistem mengalami penurunan performa pada pertanyaan dengan latar belakang yang tidak sepenuhnya terwakili dalam *knowledge graph* berbasis spesifikasi OpenAPI. Ketiga, *knowledge graph* yang dibangun dari spesifikasi OpenAPI bersifat statis terhadap perubahan perilaku *runtime* Kubernetes yang dinamis.

Kontribusi tiap komponen sistem dikonfirmasi melalui *ablation study* enam konfi-

gulasi (lihat Subbab VI.6) dan uji signifikansi statistik (lihat Subbab VI.7). Keunggulan GraphRAG pada dimensi RetQ ($\Delta = +0,26$, $p < 0,001$) bersifat konsisten dan tidak dapat dikaitkan dengan variasi acak, sebagaimana dikonfirmasi oleh *confidence interval* 95% $[+0,19, +0,33]$ yang tidak melewati nol.

VI.5 Perbandingan dengan Pendekatan *Baseline*

Untuk mengkuantifikasi kontribusi arsitektur *multi-hop graph traversal* terhadap kualitas sistem, evaluasi dilakukan terhadap dua *baseline* menggunakan *dataset* yang sama (97 *fixture*), *speaker* LLM yang identik (GPT-4o-mini, *temperature*=0,1), dan seluruh metrik evaluasi yang sama dengan GraphRAG. *Baseline* pertama (**Vanilla LLM**) menjawab pertanyaan langsung tanpa konteks apapun; *baseline* kedua (**Vector RAG**) menggunakan *cosine similarity* top-5 terhadap *vector index* Neo4j diikuti ekspansi 1-hop, tanpa *multi-hop traversal*. Hasil perbandingan ditampilkan pada Tabel VI.5.

Tabel VI.5 Perbandingan Skor Evaluasi: Vanilla LLM vs. Vector RAG vs. GraphRAG (97 Fixture)

Sub-Metrik	Vanilla LLM	Vector RAG	GraphRAG
<i>Kualitas Jawaban (AnsQ) — bobot 40%</i>			
<i>Syntactic Validity</i>	0,7895	0,8947	0,9474
<i>Schema Compliance</i>	0,7368	0,8421	0,7895
<i>Answer Relevance</i>	0,6261	0,6423	0,6682
<i>Faithfulness</i>	0,5185	0,5040	0,4289
Skor AnsQ (40%)	0,5880	0,5979	0,5743
<i>Kualitas Retrieval (RetQ) — bobot 35%</i>			
<i>Precision@k</i>	0,0000	0,3606	0,6705
<i>Recall@k</i>	0,0000	0,3431	0,5907
<i>F1@k</i>	0,0000	0,2910	0,5658
<i>NDCG@k</i>	0,0000	0,4443	0,7503
<i>Graph Coverage</i>	0,0722	0,5411	0,8599
<i>Edge Coverage</i>	0,0722	0,5693	0,6731
Skor RetQ (35%)	0,0241	0,4249	0,6851
<i>Kualitas Penalaran (ReaQ) — bobot 25%</i>			
<i>Multi-Hop Success</i>	1,0000	1,0000	1,0000
<i>Hop Accuracy</i>	0,0000	1,0000	0,8969
<i>Scope Accuracy</i>	0,9278	0,9330	0,9330
<i>Grounding Score</i>	0,5975	0,6391	0,7678
<i>Hallucination Rate</i>	0,4025	0,3609	0,2322
Skor ReaQ (25%)	0,6313	0,8930	0,8994
Total Skor Komposit	0,4015	0,6111	0,6943

Kualitas Jawaban (AnsQ)

Pada dimensi AnsQ, Vector RAG memperoleh skor AnsQ tertinggi (0,5979), diikuti Vanilla LLM (0,5880) dan GraphRAG (0,5743). Keunggulan Vector RAG pada di-

meni ini terutama berasal dari *schema compliance* (0,8421 vs. 0,7895 GraphRAG) dan *faithfulness* (0,5040 vs. 0,4289 GraphRAG). Perbedaan *schema compliance* terjadi karena Vector RAG menyediakan konteks deskriptif yang lebih ringkas—top-5 node plus satu *hop*—sehingga model Speaker tidak teralihkan oleh *intermediate structural nodes* dari traversal yang lebih dalam dan lebih mudah mengikuti pola skema yang benar. GraphRAG memperoleh *syntactic validity* YAML tertinggi (0,9474) di atas Vector RAG (0,8947) dan Vanilla LLM (0,7895) berkat validasi tiga lapis berbasis *knowledge graph* yang hanya aktif pada *intent generate_yaml*.

Kualitas Retrieval (RetQ)

Dimensi RetQ merupakan dimensi yang paling jelas membedakan ketiga pendekatan. Vanilla LLM memperoleh RetQ 0,0241 (mendekati nol) karena tidak ada *retrieval* sama sekali; skor kecil yang tersisa berasal dari *graph coverage* dan *edge coverage* yang mengukur apakah node relevan secara kebetulan disebutkan dalam jawaban. Vector RAG mencapai RetQ 0,4249 dengan *precision@k* 0,3606 dan *graph coverage* 0,5411, menunjukkan bahwa ekspansi 1-*hop* mampu menjangkau sebagian node relevan, tetapi terbatas pada tetangga langsung *seed node*. GraphRAG memperoleh RetQ 0,6851 dengan *precision@k* 0,6705, *NDCG@k* 0,7503, dan *graph coverage* 0,8599—unggul pada seluruh enam sub-metrik RetQ. Keunggulan ini mencerminkan kemampuan *multi-hop traversal* kedalaman 2–3 dalam menjangkau *intermediate structural nodes* seperti *DeploymentSpec*, *PodTemplateSpec*, dan *Container* yang dilewati dalam rantai ketergantungan skema.

Kualitas Penalaran (ReaQ)

Pada dimensi ReaQ, GraphRAG memperoleh skor ReaQ tertinggi (0,8994) diikuti Vector RAG (0,8930) dan Vanilla LLM (0,6313). Skor *hop accuracy* Vector RAG yang sempurna (1,0000) merupakan konsekuensi struktural dari mekanisme 1-*hop*: setiap lompatan trivially valid karena hanya ada satu level ekspansi, sehingga tidak ada traversal yang dapat dievaluasi gagal. GraphRAG dengan traversal 2–3 lompatan memiliki *hop accuracy* 0,8969, mencerminkan bahwa sekitar 10% lompatan berada pada jalur yang kurang tepat dari *seed node*. *Grounding score* GraphRAG (0,7678) jauh lebih tinggi dari Vector RAG (0,6391), mengindikasikan bahwa GPT-4o-mini sebagai Speaker secara konsisten lebih baik memanfaatkan konteks yang di-*retrieve* sehingga *hallucination rate* GraphRAG (0,2322) jauh lebih rendah dari Vector RAG (0,3609) dan Vanilla LLM (0,4025). Vanilla LLM memperoleh ReaQ 0,6313; *multi_hop_success* bernilai 1,0000 secara vakuum karena tidak ada *reasoning path* yang perlu diverifikasi.

Total Skor Komposit

GraphRAG memperoleh total skor komposit tertinggi (0,6943). Vector RAG masih unggul pada dimensi AnsQ (0,5979 vs. 0,5743), namun GraphRAG kini unggul pada dimensi RetQ (+0,2601 di atas Vector RAG) maupun ReaQ (+0,0064), didukung keunggulan RetQ yang jauh lebih besar dan penurunan *hallucination rate* yang signifikan. Selisih AnsQ (−0,0236) adalah satu-satunya dimensi tempat Vector RAG masih lebih unggul. Hasil ini mengkonfirmasi bahwa *multi-hop graph traversal* memberikan kontribusi yang terukur dan signifikan terhadap kualitas sistem secara keseluruhan, terutama pada dimensi *retrieval* yang mencerminkan kemampuan sistem menavigasi jaringan ketergantungan skema Kubernetes yang kompleks. Validitas statistik perbedaan ini dikonfirmasi melalui uji Wilcoxon *signed-rank* dan *paired bootstrap* 1.000 iterasi dengan 95% CI RetQ [+0,19, +0,33], $p < 0,001$ (lihat Subbab VI.7).

VI.6 Ablation Study

Ablation study dilakukan untuk memverifikasi bahwa setiap komponen sistem GraphRAG memberikan kontribusi yang terukur, bukan sekadar elemen arsitektur yang tidak esensial. Enam konfigurasi ablasi dirancang sedemikian rupa sehingga tiap konfigurasi menonaktifkan tepat satu komponen yang diklaim sebagai kontribusi, kemudian dievaluasi terhadap seluruh 97 *fixture* menggunakan metrik yang sama dengan evaluasi utama. Hasil ditampilkan pada Tabel VI.6.

Tabel VI.6 Hasil *ablation study*: selisih skor terhadap *baseline* GraphRAG (v12)

Ablasi	Komponen Dinonaktifkan	Δ Total	Δ RetQ
A1 (<i>no_phase1</i>)	<i>Exact match</i> Fase 1	−0,0892	−0,2428
A2 (<i>no_multihop</i>)	<i>Multi-hop traversal</i>	−0,2398	−0,5585
A3 (<i>depth=2 fixed</i>)	<i>Adaptive depth</i> (dangkal)	−0,0149	−0,0484
A4 (<i>depth=3 fixed</i>)	<i>Adaptive depth</i> (dalam)	+0,0187	+0,0253
A5 (<i>no_yaml_layer3</i>)	Lapisan 3 validasi Neo4j	+0,0067	+0,0032
A6c (<i>no_multi_entity</i>)	<i>Multi-entity retrieval</i>	+0,0044	−0,0022

$\Delta > 0$ berarti ablasi meningkatkan skor (komponen tidak esensial atau kontraproduktif)

Ablasi A2 (*no_multihop*) menghasilkan penurunan RetQ terbesar (−0,5585 poin), membuktikan bahwa *multi-hop graph traversal* adalah komponen paling krusial dalam sistem. Tanpa traversal multi-lompatan, sistem hanya memperoleh *seed node* tanpa konteks dependensi skema yang diperlukan untuk menjawab pertanyaan struktural. Ablasi A1 (*no_phase1*) menurunkan RetQ sebesar −0,2428 poin, mengkonfirmasi bahwa *exact match* berperan penting sebagai gerbang pertama yang memas-

tikan *seed node* yang tepat sebelum traversal dimulai.

Ablasi A3 dan A4 secara bersama-sama membuktikan Klaim 2 penelitian ini. A3 (kedalaman tetap 2 untuk semua *intent*) menurunkan RetQ sebesar $-0,0484$ poin karena *fixture yaml_gen* dan *planning* membutuhkan kedalaman 3 untuk menjangkau konteks yang cukup. A4 (kedalaman tetap 3 untuk semua *intent*) menghasilkan skor agregat yang sedikit lebih tinggi ($+0,0187$) karena kedalaman 3 mendekati kedalaman adaptif untuk mayoritas *intent*; namun, analisis per tipe (Subbab VI.6.1) menunjukkan bahwa tipe *followup* justru menurun tajam pada kedalaman 3, yang mengkonfirmasi bahwa tidak ada kedalaman tetap tunggal yang optimal untuk seluruh jenis kueri.

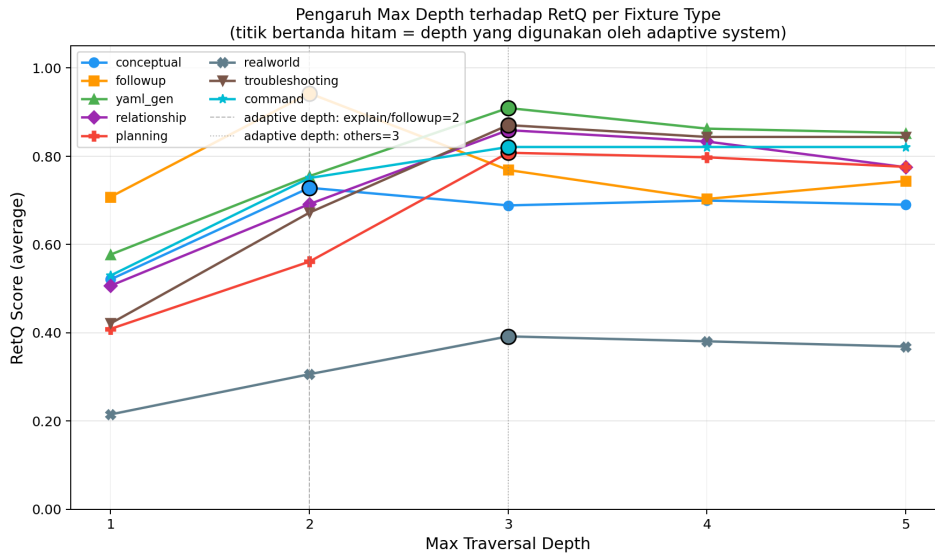
Ablasi A5 dan A6c menghasilkan delta yang sangat kecil dan tidak signifikan secara statistik (lihat Subbab VI.7). Hal ini merupakan temuan yang perlu ditafsirkan dengan hati-hati: kontribusi A5 (lapisan 3 validasi Neo4j) dan A6c (*multi-entity retrieval*) tidak terdeteksi pada level agregat 97 *fixture*, kemungkinan karena cakupan *fixture* yang terkait relatif kecil (15 *fixture yaml_gen* untuk A5, 20 *fixture* untuk A6c) sehingga efeknya terdilusi dalam rata-rata keseluruhan.

VI.6.1 Analisis Sensitivitas Kedalaman Traversal

Untuk membuktikan Klaim 2 secara lebih komprehensif, evaluasi tambahan dijalankan pada kedalaman $d \in \{1, 2, 3, 4, 5\}$ dengan seluruh 97 *fixture*. Gambar VI.1 menampilkan RetQ per tipe *fixture* pada setiap kedalaman, sedangkan Gambar VI.2 menampilkan Precision@k, Recall@k, dan NDCG@k secara agregat.

Hasil analisis menunjukkan terbentuknya dua kelompok *fixture* yang berbeda secara jelas. Kelompok pertama mencakup tipe *explain* dan *followup*, yang mencapai RetQ optimal pada $d = 2$: tipe *followup* mencatat RetQ 0,943 pada $d = 2$, kemudian turun menjadi 0,769 pada $d = 3$ ($\Delta = -17,4$ poin). Penurunan ini terjadi karena pada $d = 3$ traversal mulai menjangkau *node* utilitas generik (*Quantity*, *IntOrString*) yang tidak relevan untuk pertanyaan konseptual. Kelompok kedua mencakup tipe generatif dan struktural (*yaml_gen*, *planning*, *trace_relationship*), yang mencapai optimal pada $d = 3$: tipe *yaml_gen* mencatat RetQ 0,755 pada $d = 2$ dan meningkat menjadi 0,909 pada $d = 3$ ($\Delta = +15,4$ poin) karena generasi YAML membutuhkan konteks dependensi yang lebih dalam.

Pada kedalaman $d \geq 4$, seluruh tipe mengalami penurunan RetQ akibat *diminishing returns*: traversal semakin jauh menjangkau tipe utilitas generik yang dibagikan oleh puluhan *resource* sehingga mengurangi presisi konteks. Precision@k turun secara



Gambar VI.1 RetQ per tipe *fixture* pada kedalaman traversal $d = 1$ hingga $d = 5$. Titik bertanda hitam menunjukkan kedalaman yang digunakan oleh sistem adaptif; garis putus-putus ($d = 2$) dan titik-titik ($d = 3$) menandai batas dua kelompok *intent*.

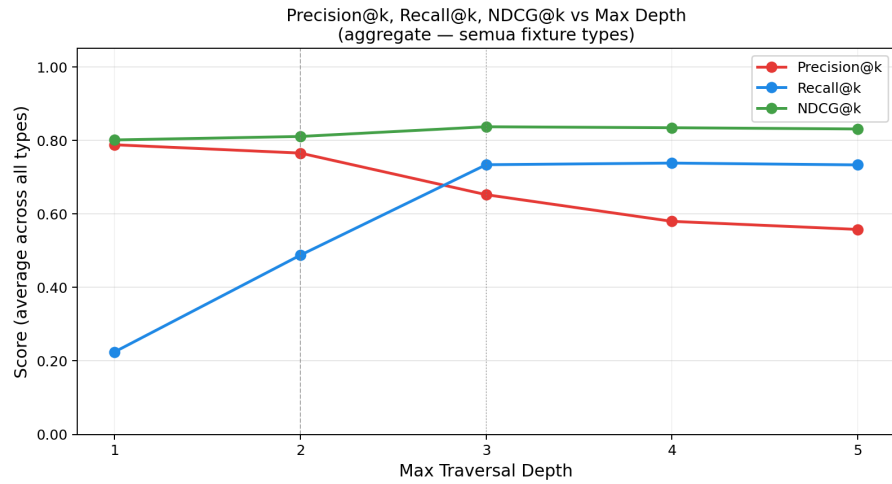
monoton dari $d = 3$ ke $d = 5$, sedangkan NDCG@k mencapai puncak pada $d = 3$ (0,838) dan turun setelahnya. Temuan ini secara empiris mengkonfirmasi bahwa mekanisme *intent-adaptive depth* yang menggunakan $d = 2$ untuk *explain/followup* dan $d = 3$ untuk tipe lainnya adalah keputusan desain yang optimal.

VI.7 Uji Signifikansi Statistik

Untuk memverifikasi bahwa perbedaan skor antara GraphRAG dan *baseline* bukan merupakan hasil variasi acak, uji signifikansi statistik dilakukan menggunakan dua metode yang saling melengkapi: (1) uji Wilcoxon *signed-rank*, uji non-parametrik yang tidak mengasumsikan distribusi normal dan cocok untuk skor $[0,1]$ yang berpotensi *skewed*; dan (2) *paired bootstrap* 1.000 iterasi yang melakukan *resampling* 97 pasang *fixture* dengan pengembalian untuk memperoleh *confidence interval* (CI) dan *p-value* empiris.

Tabel VI.7 menyajikan hasil uji untuk perbandingan utama GraphRAG versus *Vector RAG*.

Keunggulan GraphRAG pada dimensi RetQ dan Total bersifat sangat signifikan ($p < 0,001$) dengan 95% CI RetQ $[+0,19, +0,33]$ yang tidak melewati nol, mengkonfirmasi bahwa perbedaan ini konsisten dan bukan hasil variasi sampling. Dimensi AnsQ tidak signifikan ($p = 0,390$), yang merupakan hasil yang diharapkan: kedua sistem menggunakan LLM yang identik sehingga kualitas teks jawaban tidak



Gambar VI.2 Precision@k, Recall@k, dan NDCG@k agregat pada kedalaman $d = 1$ hingga $d = 5$.

Tabel VI.7 Hasil uji signifikansi statistik: GraphRAG vs. *Vector RAG* ($n = 97$)

Metrik	GraphRAG	Vector RAG	Δ	p -value	Sig
Precision@k	0,6705	0,3606	+0,3099	< 0,001	***
Recall@k	0,5907	0,3431	+0,2476	< 0,001	***
NDCG@k	0,7503	0,4443	+0,3060	< 0,001	***
RetQ	0,6851	0,4249	+0,2601	< 0,001	***
AnsQ	0,5743	0,5979	-0,0236	0,390	n.s.
ReaQ	0,8994	0,8930	+0,0064	—	~
Total	0,6943	0,6111	+0,0832	< 0,001	***

*** $p < 0,001$ n.s. tidak signifikan 95% CI RetQ: [+0,19, +0,33]

dipengaruhi oleh perbedaan mekanisme *retrieval*.

Tabel VI.8 menyajikan signifikansi statistik untuk setiap ablasi dibandingkan *base-line* GraphRAG.

Komponen *exact match* (A1), *multi-hop traversal* (A2), dan *adaptive depth* (A3) terbukti memberikan kontribusi yang signifikan secara statistik. A4 tidak signifikan ($p = 0,992$) karena kedalaman tetap 3 sudah mendekati perilaku adaptif untuk mayoritas *intent*; perbedaannya terlihat pada analisis per tipe di Subbab VI.6.1, bukan pada agregat keseluruhan. A5 dan A6c tidak signifikan, konsisten dengan delta yang sangat kecil pada Tabel VI.6.

Tabel VI.8 Signifikansi statistik *ablation study*: *baseline* vs. varian ablasi

Ablasi	RetQ <i>p</i> -value	Total <i>p</i> -value
A1 (<i>no_phase1</i>)	< 0,001 ***	< 0,001 ***
A2 (<i>no_multihop</i>)	< 0,001 ***	< 0,001 ***
A3 (<i>depth=2 fixed</i>)	< 0,001 ***	< 0,001 ***
A4 (<i>depth=3 fixed</i>)	0,992 n.s.	0,992 n.s.
A5 (<i>no_yaml_layer3</i>)	n.s.	n.s.
A6c (<i>no_multi_entity</i>)	n.s.	n.s.
*** $p < 0,001$ n.s. tidak signifikan ($p > 0,05$)		

VI.8 Analisis Kondisi Batas Sistem GraphRAG

Evaluasi pada bagian-bagian sebelumnya menunjukkan bahwa sistem GraphRAG unggul secara statistik dibandingkan *Vector RAG* secara keseluruhan. Namun, agregasi skor seluruh *fixture* dapat menyembunyikan variasi penting: pada kondisi apa GraphRAG memberikan keunggulan signifikan, dan pada kondisi apa keunggulannya terbatas? Analisis kondisi batas ini bertujuan mengidentifikasi faktor-faktor penentu tersebut menggunakan metrik *RetQ-gain* per *fixture*, yang didefinisikan sebagai:

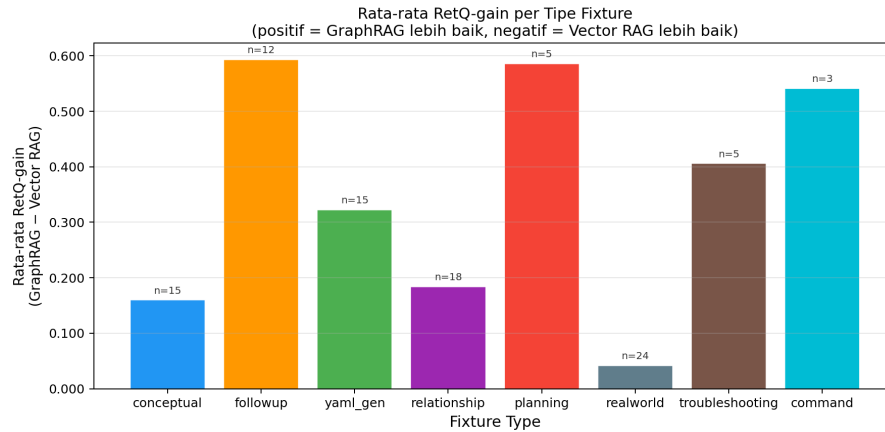
$$\text{RetQ-gain}_i = \text{RetQ}_{\text{GraphRAG},i} - \text{RetQ}_{\text{VectorRAG},i} \quad (\text{VI.2})$$

Nilai positif pada persamaan di atas menunjukkan GraphRAG unggul, sedangkan nilai negatif menunjukkan *Vector RAG* setara atau lebih baik untuk *fixture* ke-*i*.

VI.8.1 Keunggulan GraphRAG per Tipe *Fixture*

Gambar VI.3 menampilkan rata-rata *RetQ-gain* untuk setiap tipe *fixture*. GraphRAG mengungguli *Vector RAG* pada seluruh delapan tipe, dengan variasi yang mencerminkan karakteristik struktural tiap tipe pertanyaan.

Terdapat pola yang konsisten: tipe *fixture* dengan gain tertinggi (*followup*, *planning*, *command*) adalah tipe yang secara inheren membutuhkan pemahaman relasi antarentitas. Sebaliknya, *fixture* tipe *realworld* yang memiliki gain terendah (+0,04) umumnya mengandung pertanyaan dengan konteks luas yang melebihi cakupan skema OpenAPI Kubernetes. Temuan ini menunjukkan bahwa **kompleksitas relasional kueri** merupakan faktor penentu utama keunggulan GraphRAG.



Gambar VI.3 Rata-rata *RetQ-gain* (GraphRAG – *Vector RAG*) per tipe *fixture* ($n = 97$). Semua tipe menunjukkan gain positif; variasi mencerminkan tingkat ketergantungan kueri terhadap traversal relasi antar-*resource*.

Tabel VI.9 Rata-rata *RetQ-gain* per tipe *fixture*

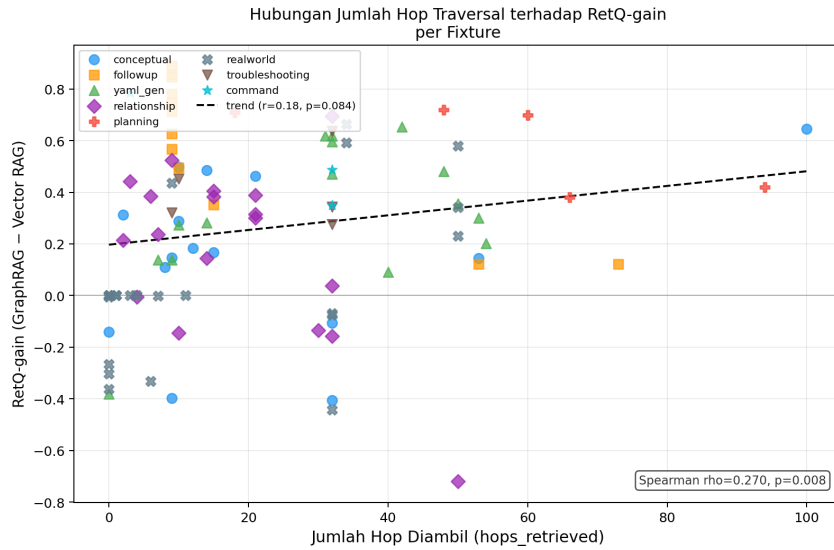
Type	n	Rata-rata <i>RetQ-gain</i>	Karakteristik Kueri
<i>followup</i>	12	+0,5931	Tindak lanjut relasi antar- <i>resource</i>
<i>planning</i>	5	+0,5858	Perancangan multi-komponen
<i>command</i>	3	+0,5411	Konfigurasi operasional
<i>troubleshooting</i>	5	+0,4058	Diagnosis lintas- <i>resource</i>
<i>yaml_gen</i>	15	+0,3226	Generasi manifes terstruktur
<i>relationship</i>	18	+0,1838	Pemetaan hubungan eksplisit
<i>conceptual</i>	15	+0,1598	Penjelasan konsep tunggal
<i>realworld</i>	24	+0,0413	Skenario dunia nyata (konteks luas)

VI.8.2 Faktor Kuantitatif: *Hops* dan Derajat Node

Dua faktor kuantitatif dianalisis menggunakan korelasi Spearman terhadap *RetQ-gain*: jumlah hop yang dilalui selama *retrieval* (*hops_retrieved*) dan derajat node primer dalam KG (*graph_degree*).

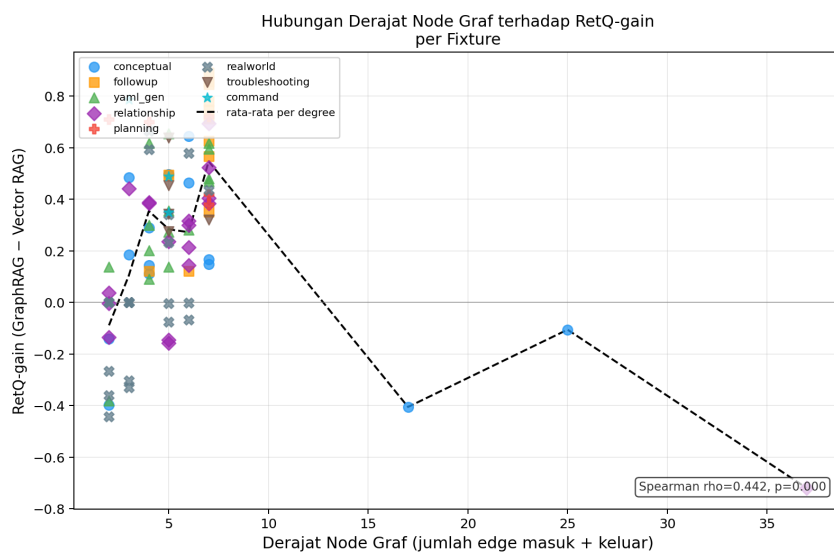
Gambar VI.4 menampilkan hubungan antara jumlah hop dan *RetQ-gain*. Korelasi Spearman antara *hops_retrieved* dan *RetQ-gain* adalah $\rho = +0,270$ ($p = 0,008$, $n = 97$)—signifikan namun tergolong lemah, mengindikasikan bahwa jumlah hop bukan satu-satunya penentu.

Gambar VI.5 menampilkan hubungan antara derajat node primer dan *RetQ-gain*. Korelasi Spearman $\rho = +0,442$ ($p < 0,001$, $n = 96$) lebih kuat. Analisis rata-rata gain per kelompok derajat mengungkap pola yang menarik: *resource* dengan dera-



Gambar VI.4 Scatter plot jumlah hop yang dilalui vs. *RetQ-gain* per fixture.

jat sangat rendah (derajat 2, $n = 14$, rata-rata gain = $-0,09$) seperti ConfigMap dan Secret tidak mendapat manfaat dari traversal graf karena kemiripan semantik berbasis *embedding* sudah memadai. Sebaliknya, *resource* dengan derajat 3–7 ($n = 79$) menunjukkan gain positif yang konsisten, dengan puncak pada derajat 7 ($n = 24$, rata-rata $+0,55$) yang didominasi Deployment dan StatefulSet. Tiga *resource* dengan derajat ≥ 17 ($n = 3$, masing-masing satu *fixture*) menunjukkan gain negatif, konsisten dengan temuan *depth sensitivity* bahwa *node* berkonektivitas ekstrem cenderung menambah *noise*; namun, keterbatasan data ($n = 1$ per kelompok) mengharuskan kehati-hatian dalam menarik kesimpulan dari pola ini.



Gambar VI.5 Scatter plot derajat node primer vs. *RetQ-gain*. Garis menunjukkan rata-rata gain per nilai derajat.

Tabel VI.10 Korelasi Spearman antara faktor struktural dan *RetQ-gain*

Faktor	ρ	p -value	n	Interpretasi
graph_degree	+0,442	< 0,001	96	Sedang ***
hops_retrieved	+0,270	0,008	97	Lemah **
multi_hop (0/1)	+0,093	0,366	97	Tidak signifikan
*** $p < 0,001$ ** $p < 0,01$ Korelasi positif = faktor berkorelasi dengan keunggulan GraphRAG				

Tidak ada faktor tunggal yang secara dominan menjelaskan variasi *RetQ-gain*. Keunggulan GraphRAG ditentukan oleh interaksi antara karakteristik kueri (tipe, kompleksitas relasional) dan karakteristik data (konektivitas *resource* dalam graf).

VI.8.3 Definisi Kondisi Batas

Berdasarkan analisis di atas, kondisi batas sistem GraphRAG dirumuskan sebagai berikut.

GraphRAG memberikan keunggulan signifikan ketika:

1. kueri membutuhkan pemahaman relasi antar-*resource* (tipe *followup*, *planning*, *command*, *troubleshooting*); dan/atau
2. *resource* primer memiliki derajat konektivitas menengah–tinggi di graf (derajat 3–7 dalam konteks skema Kubernetes v1.30).

GraphRAG tidak memberikan keunggulan bermakna ketika:

1. kueri bersifat kontekstual luas tanpa referensi spesifik ke skema (tipe *realworld*); atau
2. *resource* primer memiliki konektivitas sangat rendah (derajat 2), sehingga kemiripan semantik berbasis *embedding* sudah memadai; atau
3. *resource* primer merupakan tipe utilitas generik dengan konektivitas ekstrem (derajat ≥ 17), yang menghasilkan *noise* saat dilalui.

VI.8.4 Prediksi Generalisasi ke Domain Lain

Analisis kondisi batas ini memungkinkan prediksi kualitatif untuk domain OpenAPI lain. Skema Kubernetes v1.30 memiliki hirarki yang dalam dengan *resource* utama seperti Deployment memiliki 7 *edge* langsung. Sebaliknya, GitHub REST API mengadopsi arsitektur *endpoint-centric* yang lebih datar dengan ketergantungan skema minimal antar-*resource*. Berdasarkan kondisi batas yang telah diidentifikasi, keunggulan GraphRAG diperkirakan lebih terbatas pada GitHub REST API karena *resource*-nya cenderung memiliki derajat rendah dan pertanyaan umumnya tidak membutuhkan traversal lintas-*resource*. Validasi prediksi ini merupakan arah pengembangan yang dapat dikerjakan pada studi lanjutan.

BAB VII

PENUTUP

VII.1 Kesimpulan

Penelitian ini telah berhasil mengimplementasikan sistem *Graph Retrieval-Augmented Generation* (GraphRAG) berbasis spesifikasi OpenAPI Kubernetes dan memverifikasi tiga kontribusi teknis utama melalui evaluasi empiris terhadap 97 *fixture* uji, *ablation study* enam komponen, serta uji signifikansi statistik.

Pertama, *pipeline* ekstraksi *knowledge graph* yang deterministik berbasis skema OpenAPI berhasil diimplementasikan. Graf dibangun dari `swagger.json` versi v1.30 menghasilkan 725 *node* definisi dan 18 jenis *edge* dalam 7 kategori relasi semantik yang khusus dirancang untuk domain Kubernetes. Karena seluruh *edge* diturunkan dari referensi tipe dalam skema, graf bersifat *reproducible* dan dapat diverifikasi secara otomatis—berbeda dengan pendekatan ekstraksi berbasis LLM yang menghasilkan representasi stokastik (Pan dkk. 2024; Wan dkk. 2025). *Ablation study* komponen *exact match* (A1) dan *multi-hop traversal* (A2) mengkonfirmasi kontribusi masing-masing: penghapusan *exact match* menurunkan RetQ sebesar 0,2428 poin ($p < 0,001$), sedangkan penghapusan *multi-hop* menurunkan RetQ sebesar 0,5585 poin ($p < 0,001$), membuktikan bahwa setiap komponen retrieval berkontribusi nyata.

Kedua, mekanisme *intent-adaptive depth traversal* terbukti lebih unggul dibandingkan kedalaman tetap. Analisis sensitivitas kedalaman pada $d \in \{1, 2, 3, 4, 5\}$ menunjukkan bahwa tipe *intent explain* dan *follow-up* mencapai RetQ optimal pada kedalaman 2 (misalnya, *follow-up*: RetQ 0,943 pada $d = 2$ turun ke 0,769 pada $d = 3$, $\Delta = -17,4$ poin), sementara *yaml_gen* dan *planning* mencapai optimal pada kedalaman 3. *Ablation study* A3 (kedalaman tetap 2) mengkonfirmasi bahwa kedalaman seragam yang terlalu dangkal menurunkan RetQ sebesar 0,0484 poin ($p < 0,001$). Hal ini membuktikan bahwa tidak ada kedalaman tetap tunggal yang optimal untuk seluruh tipe *intent*, sehingga pendekatan adaptif yang diusulkan terjustifikasi secara empiris.

Ketiga, sistem GraphRAG mencapai total skor komposit tertinggi (**0,6943**) dibandingkan *Vector RAG* (0,6111) dan *Vanilla LLM* (0,4015). Keunggulan pada dimensi *Retrieval Quality* (RetQ 0,69 vs. 0,42, $\Delta = +0,26$, $p < 0,001$, 95% CI [+0,19, +0,33]) bersifat konsisten dan signifikan secara statistik. Validasi struktural YAML tiga lapis berbasis *knowledge graph* mencapai validitas sintaksis **0,9474** dengan memverifikasi *required fields* langsung dari graf—menyediakan mekanisme validasi tanpa memerlukan *dry-run* pada kluster aktif. Analisis kondisi batas menunjukkan bahwa GraphRAG memberikan keunggulan terbesar pada tipe *intent* relasional (*follow-up* +0,59, *planning* +0,59) dan pada *resource* dengan derajat konektivitas menengah-tinggi (derajat 3–7, Spearman $\rho = +0,44$, $p < 0,001$).

VII.2 Saran

Berdasarkan keterbatasan yang teridentifikasi, beberapa arah pengembangan lanjutan direkomendasikan.

1. Perluasan Sumber Data *Knowledge Graph* dengan Relasi Operasional

Knowledge graph yang dibangun dalam penelitian ini merupakan *directed graph* yang hanya merepresentasikan relasi struktural-skema (arah edge mengikuti referensi tipe dalam `swagger.json`). Akibatnya, relasi operasional seperti ikatan RBAC antara *ServiceAccount* dan *ClusterRole* tidak dapat dijangkau melalui *traversal* yang dimulai dari *ServiceAccount*, karena edge tersebut hanya ada dalam arah terbalik (*RoleBinding* $\rightarrow$ *ServiceAccount*). Pengembangan lanjutan dapat mempertimbangkan penambahan sumber relasi operasional—misalnya dari dokumentasi resmi atau anotasi *best practice*—sebagai lapisan kedua di atas graf skema yang sudah ada, tanpa mengubah prinsip determinisme Klaim 1.

2. Integrasi Sumber Data Dinamis untuk Pertanyaan *Realworld*

Kategori *realworld* memperoleh *RetQ-gain* terendah (+0,04) karena pertanyaan operasional nyata sering membutuhkan konteks *runtime* yang tidak tersedia dalam skema statis. Integrasi dengan *Kubernetes Watch API* atau log operasional dapat meningkatkan kemampuan sistem menjawab pertanyaan *troubleshooting* yang membutuhkan status kluster aktual.

3. Eksplorasi *Citation-Grounded Generation* yang Lebih Selektif

Eksperimen *Citation-Grounded Generation* (CGG) yang dilakukan dalam penelitian ini—yang memeriksa istilah K8s dalam jawaban terhadap *graph_context* yang diambil, bukan kosakata global—menghasilkan penurunan *grounding score* sebesar 0,135 poin dan total skor turun 0,006 poin. Analisis menunjukkan bahwa CGG terlalu ketat: model secara valid mereferensikan konsep K8s yang terhubung secara semantik meskipun tidak secara eksplisit diambil

dalam *graph_context*, yang merupakan perilaku yang diharapkan dari integrasi pengetahuan *prior* LLM dengan *retrieved context*. Penelitian lanjutan dapat mengeksplorasi CGG selektif yang hanya membatasi istilah pada pertanyaan dengan *graph_context* yang kaya, bukan diterapkan secara seragam.

4. **Pembaruan Inkremental *Knowledge Graph***

Kubernetes merilis versi baru secara berkala dengan penambahan atau perubahan skema. Pengembangan *pipeline* pembaruan inkremental yang dapat mendeteksi perubahan antar-versi swagger . json dan memperbarui *knowledge graph* tanpa rekonstruksi penuh akan meningkatkan ketahanan sistem terhadap evolusi API.

DAFTAR PUSTAKA

- Antonio Moreno-Cediel, David De-Fitero-Dominguez, Eva Garcia-Lopez. Antonio Garcia-Cabot. 2025. "Optimising retrieval performance in RAG systems: A new growing window semantic chunking strategy to address weak semantic boundaries". *Knowledge-Based Systems* 331:114896. <https://doi.org/10.1016/j.knosys.2025.114896>.
- Cao, Ruisheng, Hanchong Zhang, Tiancheng Huang, Zhangyi Kang, Yuxin Zhang, Liangtai Sun, Hanqi Li, dkk. 2025. *NeuSym-RAG: Hybrid Neural Symbolic Retrieval with Multiview Structuring for PDF Question Answering*. arXiv preprint. Version 2, released 31 May 2025. arXiv: 2505.19754 [cs.CL]. file:///mnt/data/2%20-%20NeuSym-RAG.pdf.
- Cloud Native Computing Foundation. 2023. *CNCF Annual Survey 2023*. Diakses pada November 22, 2025. <https://www.cncf.io/reports/cncf-annual-survey-2023/>.
- Es, Shahul, Jithin James, Luis Espinosa-Anke, dan Steven Schockaert. 2023. "RAGAS: Automated Evaluation of Retrieval Augmented Generation". *arXiv preprint arXiv:2309.15217*.
- Gao, Yunfan, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, dan Haofen Wang. 2024. "Retrieval-Augmented Generation for Large Language Models: A Survey". Accessed November 25, 2025, *arXiv preprint arXiv:2312.10997*.
- Gartner. 2021. *Gartner Says Cloud Will Be the Centerpiece of New Digital Experiences*. <https://www.gartner.com/en/newsroom/press-releases/2021-11-10-gartner-says-cloud-will-be-the-centerpiece-of-new-digital-experiences>. Press Release, accessed 22 November 2025.
- Han, Haoyu, Yu Wang, Harry Shomer, Kai Guo, Jiayuan Ding, Yongjia Lei, Mahantesh Halappanavar, Ryan A. Rossi, Subhabrata Mukherjee, dan Xianfeng Tang. 2024. "Retrieval-augmented generation with graphs (GraphRAG)". *arXiv preprint arXiv:2501.00309*.

- He, Xiaoxin, Yijun Tian, Yifei Sun, Nitesh V. Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, dan Bryan Hooi. 2024. “G-Retriever: Retrieval-Augmented Generation for Textual Graph Understanding and Question Answering”. Unpublished manuscript, based on user-provided document; no official venue or URL available.
- Hofer, Marvin, Daniel Obraczka, Alieh Saeedi, Hanna Köpcke, dan Erhard Rahm. 2024. “Construction of Knowledge Graphs: Current State and Challenges”. *Information* 15 (8): 509.
- Knollmeyer, Simon, Oğuz Caymazer, dan Daniel Grossmann. 2025. “Document GraphRAG: Knowledge Graph Enhanced Retrieval Augmented Generation for Document Question Answering Within the Manufacturing Domain”. *Electronics* 14 (11): 2102. <https://doi.org/10.3390/electronics14112102>. <https://doi.org/10.3390/electronics14112102>.
- Kotstein, Sebastian, dan Christian Decker. 2024. “RESTBERTa: a Transformer-based question answering approach for semantic search in Web API documentation”. Published online 31 January 2024, *Cluster Computing*, <https://doi.org/10.1007/s10586-023-04237-x>.
- Liu, Zhijie, Anh Nguyen, Junjie Chen, Xin Zhang, Yanjie Li, dan Yingfei Xiong. 2024. “Deep Analysis of LLM-based Code Generation: Correctness, Complexity, and Security”. *IEEE Transactions on Software Engineering*, 1–20. <https://doi.org/10.1109/TSE.2024.3392499>.
- Lu, Zhouyang, Hailin Xu, Anrui Chen, Siyuan Tang, Junyi Zhang, Yifei Feng, Wentao Pan, dan Jiangli Huang. October 2025. “HSG-RAG: Hierarchical Knowledge Base Construction for Embedded System Development”. *ACM Transactions on Design Automation of Electronic Systems* 30, no. 6 (): 1–21. <https://doi.org/10.1145/3731680>.
- Ma, Chuangtao, Yongrui Chen, Tianxing Wu, Arijit Khan, dan Haofen Wang. 2025. “Large Language Models Meet Knowledge Graphs for Question Answering: Synthesis and Opportunities”. *arXiv preprint arXiv:2505.20099*.
- Niakšu, Olegas. 2015. “CRISP Data Mining Methodology Extension for Medical Domain”. *Baltic Journal of Modern Computing* 3 (2): 92–109. https://www.bjmc.lu.lv/fileadmin/user_upload/lu_portal/projekti/bjmc/Contents/3_2_2_Niaksu.pdf.

- Pan, Shirui, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, dan Xindong Wu. 2024. “Unifying Large Language Models and Knowledge Graphs: A Roadmap”. *IEEE Transactions on Knowledge and Data Engineering* 36 (7): 3580–3599. <https://doi.org/10.1109/TKDE.2024.3352100>.
- Rahman, A., dkk. 2023. “Empirical Analysis of Security Misconfigurations in Open-Source Kubernetes Manifests”. *Journal of Cloud Computing and Security* 12 (4): 112–128.
- Soldani, Jacopo, Damian Andrew Tamburri, dan Willem-Jan Van Den Heuvel. 2018. “The pains and gains of microservices: A Systematic grey literature review”. *Journal of Systems and Software* 146:215–232. <https://doi.org/10.1016/j.jss.2018.09.082>.
- The Kubernetes Authors. 2024a. *Kubernetes Concepts*. Diakses pada: 2024-05-15. <https://kubernetes.io/docs/concepts/>.
- . 2024b. “Kubernetes Documentation”. Diakses pada February 22, 2025. <https://kubernetes.io/docs/home/>.
- Wan, Yuwei, Zheyuan Chen, Ying Liu, Chong Chen, dan Michael Packianather. 2025. “Empowering LLMs by hybrid retrieval-augmented generation for domain-centric Q&A in smart manufacturing”. *Advanced Engineering Informatics* 65:103212. ISSN: 1474-0346. <https://doi.org/10.1016/j.aei.2025.103212>.
- Yu, Hui-Hung, Wei-Tsun Lin, Chih-Wei Kuan, Chao-Chi Yang, dan Kuan-Min Liao. 2025. “GraphRAG-Enhanced Dialogue Engine for Domain-Specific Question Answering: A Case Study on the Civil IoT Taiwan Platform”. *Future Internet* 17 (9): 414. <https://doi.org/10.3390/fi17090414>.
- Yu, Jun, Yintie Zhang, Yu Wu, dan Linhui Mao. 2021. “Research on the Practical Application of Visual Knowledge Graph in Technology Service Model and Intelligent Supervision”. *Journal of Physics: Conference Series* 1982:012040.
- Zhao, Penghao, Hailin Zhang, Qinhan Yu, Zhengren Wang, Yunteng Geng, Fangcheng Fu, Ling Yang, Wentao Zhang, Jie Jiang, dan Bin Cui. 2024. “Retrieval-Augmented Generation for AI-Generated Content: A Survey”. *arXiv preprint arXiv:2402.19473*.

LAMPIRAN A

KODE PROGRAM

Lampiran ini menyajikan kode sumber komponen inti sistem *GraphRAG* Kubernetes yang dibahas pada Bab V. Setiap kode telah disederhanakan komentar (*humanized*) untuk keterbacaan: komentar yang hanya mengulang nama variabel dihilangkan, sedangkan komentar yang menjelaskan *alasan* keputusan desain dipertahankan. Kode lengkap beserta riwayat komit tersedia di repositori proyek.

A.1 Manajemen Memori Percakapan (*ZepMemoryStore*)

Kelas *ZepMemoryStore* pada berkas `src/memory/zep_store.py` menyediakan penyimpanan riwayat percakapan berbasis SQLite lokal sebagai pengganti Zep Cloud.

```
1 # src/memory/zep_store.py
2 #
3 # SQLite-based conversation memory - pengganti Zep v1.
4 # Zep v1 memanggil LLM internal (entity extraction) yang
5 #   memboroskan token
6 # tanpa nilai tambah untuk penelitian ini. SQLite memberikan
7 #   persistensi
8 # yang cukup dengan zero token overhead dan zero Docker dependency
9 #   tambahan.
10 # Public API identik dengan versi Zep sehingga graph_agent.py
11 #   tidak perlu diubah.
12
13 import sqlite3
14 import logging
15 from pathlib import Path
16
17 logger = logging.getLogger(__name__)
18
19 _DB_PATH = Path(__file__).parent.parent.parent / "data" / "
20     conversation_memory.db"
21
22 def _get_connection() -> sqlite3.Connection:
23     _DB_PATH.parent.mkdir(parents=True, exist_ok=True)
```

```

20 conn = sqlite3.connect(str(_DB_PATH), check_same_thread=False)
21 conn.execute("""
22     CREATE TABLE IF NOT EXISTS messages (
23         id            INTEGER PRIMARY KEY AUTOINCREMENT,
24         session_id    TEXT      NOT NULL,
25         role          TEXT      NOT NULL,
26         content       TEXT      NOT NULL,
27         created_at    DATETIME DEFAULT CURRENT_TIMESTAMP
28     )
29 """)
30 conn.execute("CREATE INDEX IF NOT EXISTS idx_session ON
messages(session_id)")
31 conn.commit()
32 return conn
33
34
35 # Singleton connection - satu koneksi per proses
36 _conn: sqlite3.Connection | None = None
37
38
39 def _db() -> sqlite3.Connection:
40     global _conn
41     if _conn is None:
42         _conn = _get_connection()
43         logger.info(f"SQLiteMemory: database di {_DB_PATH}")
44     return _conn
45
46
47 class ZepMemoryStore:
48     """
49     Conversation memory berbasis SQLite.
50     Nama kelas dipertahankan agar tidak ada perubahan di
51     graph_agent.py.
52     """
53     def __init__(self):
54         _db()
55         logger.info("ZepMemoryStore: menggunakan SQLite lokal (
56 tanpa Zep server).")
57
58     def save_memory(self, session_id: str, role: str, content: str
59 ):
60         role = "user" if role.lower() in ("user", "human") else "
61 assistant"
62         try:

```

```

60         _db().execute(
61             "INSERT INTO messages (session_id, role, content)
VALUES (?, ?, ?)",
62             (session_id, role, content)
63         )
64         _db().commit()
65     except Exception as e:
66         logger.warning(f"SQLiteMemory: gagal menyimpan pesan:
{e}")
67
68     def get_memory(self, session_id: str, limit: int = 5) -> str:
69         try:
70             rows = _db().execute(
71                 """
72                 SELECT role, content FROM messages
73                 WHERE session_id = ?
74                 ORDER BY id DESC
75                 LIMIT ?
76                 """,
77                 (session_id, limit)
78             ).fetchall()
79             # Rows dikembalikan dari terbaru ke terlama - balik
urutannya
80             rows = list(reversed(rows))
81             return "\n".join(f"{role}: {content}" for role,
content in rows)
82         except Exception as e:
83             logger.warning(f"SQLiteMemory: gagal mengambil riwayat
: {e}")
84             return ""
85
86     def add_message(self, session_id: str = None, role: str = None
,
87                     content: str = None, user_msg: str = None,
88                     ai_msg: str = None, message: str = None, **
kwargs):
89         if not session_id:
90             return
91         if user_msg and ai_msg:
92             self.save_memory(session_id=session_id, role="user",
content=user_msg)
93             self.save_memory(session_id=session_id, role="
assistant", content=ai_msg)
94             return

```

```

95     resolved_content = content or message or user_msg or
ai_msg or kwargs.get("text", "")
96     if not resolved_content:
97         return
98     resolved_role = role or ("user" if user_msg else ("
assistant" if ai_msg else "user"))
99     self.save_memory(session_id=session_id, role=resolved_role
, content=resolved_content)
100
101     def get_history(self, session_id: str = None, limit: int = 5,
**kwargs) -> str:
102         if not session_id:
103             return ""
104         return self.get_memory(session_id=session_id, limit=limit)
105
106     def add_turn(self, session_id: str, role: str, content: str):
107         self.save_memory(session_id=session_id, role=role, content
=content)
108
109     def get_context(self, session_id: str, limit: int = 5) -> str:
110         return self.get_memory(session_id=session_id, limit=limit)

```

Listing A.1 src/memory/zep_store.py — Manajemen Memori Percakapan

A.2 Pipeline Ingestion Data (SwaggerGraphBuilder)

Kelas *SwaggerGraphBuilder* pada berkas src/ingestion/parser.py mengimplementasikan pipeline 5-fase untuk mengubah swagger.json Kubernetes menjadi *knowledge graph* di Neo4j.

```

1  """
2  Swagger Graph Builder - Pipeline Ingestion 5-Fase
3
4  Fase:
5      Pass 1   - Buat node Definition (nama, kind, scope, deskripsi)
6      Pass 1.5 - Generate embedding vektor 1536-dim (text-embedding
-3-small)
7      Pass 2   - Buat edge HAS_PROPERTY struktural (resolusi $ref)
8      Pass 2.5 - Buat edge EXTENDS / ONE_OF / ANY_OF (pewarisan tipe
)
9      Pass 3   - Buat 18 jenis edge semantik (CONTAINS_POD_TEMPLATE,
USES_SECRET, dll.)
10 """
11
12 import json
13 from typing import Dict, Any

```

```

14 from src.graph.neo4j_client import Neo4jClient
15 from src.utils.text_utils import safe_truncate_description
16 from src.graph.vector_index import VectorIndexManager
17
18 PRIMITIVE_TYPES = {"string", "integer", "number", "boolean", "
    array", "object"}
19
20 # Tipe metadata generik yang dikecualikan karena tidak relevan
    untuk pembuatan YAML
21 IGNORE_LIST = {
22     "io.k8s.apimachinery.pkg.apis.meta.v1.ManagedFieldsEntry",
23     "io.k8s.apimachinery.pkg.apis.meta.v1.ObjectMeta",
24     "io.k8s.apimachinery.pkg.apis.meta.v1.StatusDetails",
25     "io.k8s.apimachinery.pkg.apis.meta.v1.Time",
26     "io.k8s.apimachinery.pkg.apis.meta.v1.MicroTime",
27     "io.k8s.apimachinery.pkg.apis.meta.v1.Duration",
28     "io.k8s.apimachinery.pkg.apis.meta.v1.RawExtension",
29     "io.k8s.apimachinery.pkg.apis.meta.v1.FieldsV1",
30     "io.k8s.apimachinery.pkg.apis.meta.v1.OwnerReference",
31     "io.k8s.apimachinery.pkg.apis.meta.v1.Patch",
32     "io.k8s.apimachinery.pkg.apis.meta.v1.StatusCause",
33     "io.k8s.apimachinery.pkg.version.Info",
34 }
35
36 CLUSTER_SCOPED_RESOURCES = {
37     "Namespace", "Node", "PersistentVolume", "ClusterRole",
38     "ClusterRoleBinding", "StorageClass", "
    CustomResourceDefinition",
39     "PriorityClass", "CSIDriver", "CSINode", "VolumeAttachment",
40     "RuntimeClass", "APIService", "MutatingWebhookConfiguration",
41     "ValidatingWebhookConfiguration", "ValidatingAdmissionPolicy",
42     "ValidatingAdmissionPolicyBinding", "CertificateSigningRequest
    ",
43     "IngressClass", "FlowSchema", "PriorityLevelConfiguration",
44     "SelfSubjectAccessReview", "SubjectAccessReview", "TokenReview
    "
45 }
46
47
48 class SwaggerGraphBuilder:
49     def __init__(self, swagger_path: str):
50         self.swagger_path = swagger_path
51         self.db = Neo4jClient()
52         self.definitions: Dict[str, Any] = {}
53

```

```

54     def load_swagger(self):
55         with open(self.swagger_path, 'r', encoding='utf-8') as f:
56             data = json.load(f)
57             self.definitions = data.get('definitions', {})
58
59     def ingest(self):
60         self.load_swagger()
61         self.db.execute_query("MATCH (n) DETACH DELETE n")
62
63         print("--> Pass 1: Creating Resource Definition Nodes...")
64         self._pass_1_create_nodes()
65
66         print("--> Pass 1.5: Generating Vector Embeddings...")
67         vector_mgr = VectorIndexManager()
68         vector_mgr.initialize()
69
70         print("--> Pass 2: Resolving Schema Relationships...")
71         self._pass_2_create_structural_edges()
72
73         print("--> Pass 2.5: Resolving Inheritance & Union Types
74         ...")
75         self._pass_2b_create_inheritance_edges()
76
77         print("--> Pass 3: Extracting YAML Configuration Patterns
78         ...")
79         self._pass_3_build_semantic_edges()
80
81     def _pass_1_create_nodes(self):
82         created_count = 0
83
84         for full_name, schema in self.definitions.items():
85             if full_name in IGNORE_LIST:
86                 continue
87
88             short_name = full_name.split(".")[1]
89             gvk_list = schema.get("x-kubernetes-group-version-kind", [])
90
91             kind = short_name
92             scope = "Namespaced"
93
94             if gvk_list and isinstance(gvk_list, list) and len(
95                 gvk_list) > 0:
96                 first_gvk = gvk_list[0]
97                 if isinstance(first_gvk, dict) and isinstance(
98                     first_gvk.get('kind'), str):

```

```

94         kind = first_gvk.get('kind').strip()
95         is_root = True
96         scope = "Cluster" if kind in
CLUSTER_SCOPED_RESOURCES else "Namespaced"
97     else:
98         kind, is_root, scope = "SubResource", False, "
N/A"
99     else:
100         kind, is_root, scope = "SubResource", False, "N/A"
101
102     original_desc = schema.get('description', 'No
description provided.')
103     desc = safe_truncate_description(original_desc,
hard_limit=4000)
104
105     self.db.execute_query("""
106         MERGE (d:Definition {id: $full_name})
107         SET d.name = $short_name,
108             d.fullName = $full_name,
109             d.kind = $kind,
110             d.is_root = $is_root,
111             d.scope = $scope,
112             d.description = $desc,
113             d.description_length = $original_length,
114             d.was_truncated = $was_truncated,
115             d.source = 'k8s_swagger_v1'
116     """, {
117         "full_name": full_name, "short_name": short_name,
118         "kind": kind, "is_root": is_root, "scope": scope,
119         "desc": desc, "original_length": len(original_desc
),
120         "was_truncated": len(desc) < len(original_desc)
121     })
122     created_count += 1
123
124     self.db.execute_query("MATCH (d:Definition {is_root: true
}) SET d:K8sResource")
125     print(f"    Created {created_count} nodes")
126
127     def _pass_2_create_structural_edges(self):
128         edge_count = 0
129
130         for full_name, schema in self.definitions.items():
131             if full_name in IGNORE_LIST:
132                 continue

```

```

133
134     properties = schema.get('properties', {})
135     required_fields = schema.get('required', [])
136     primitive_props = {}
137
138     for field_name, field_schema in properties.items():
139         field_type = field_schema.get('type')
140         ref = field_schema.get('$ref')
141         is_array = is_map = False
142         is_required = field_name in required_fields
143
144         if field_type == 'array' and 'items' in
145         field_schema:
146             items = field_schema['items']
147             if isinstance(items, dict):
148                 if '$ref' in items:
149                     ref, is_array = items['$ref'], True
150                 else:
151                     primitive_props[field_name] = f"
152                     array_of_{items.get('type', 'object')}"
153                     continue
154
155             if field_type == 'object' and '
156             additionalProperties' in field_schema:
157                 add_props = field_schema['additionalProperties
158                 '']
159                 if isinstance(add_props, dict):
160                     if '$ref' in add_props:
161                         ref, is_map = add_props['$ref'], True
162                     else:
163                         primitive_props[field_name] = f"
164                         map_of_{add_props.get('type', 'object')}"
165                         continue
166
167                 if field_type in PRIMITIVE_TYPES and not ref:
168                     if field_name not in ["kind", "id", "name", "
169                     is_root", "description",
170
171                                     "source", "was_truncated
172
173                     ", "description_length", "fullName"]:
174                         primitive_props[field_name] = field_type
175
176             elif ref:
177                 target_name = ref.split("/")[-1]
178                 if target_name in IGNORE_LIST:

```

```

170         primitive_props[field_name] = "string" if
"Time" in target_name else "object"
171     else:
172         if target_name in self.definitions:
173             self.db.execute_query("""
174                 MATCH (source:Definition {id:
$source_id})
175                 MATCH (target:Definition {id:
$target_id})
176                 MERGE (source)-[r:HAS_PROPERTY {
name: $field_name}]->(target)
177                 SET r.is_array = $is_array,
178                     r.is_map = $is_map,
179                     r.is_required = $is_required
180                 """, {
181                     "source_id": full_name, "target_id
": target_name,
182                     "field_name": field_name, "
is_array": is_array,
183                     "is_map": is_map, "is_required":
is_required
184                 })
185         else:
186             # Cross-reference: target tidak ada di
definitions tetapi
187             # valid sebagai tipe Kubernetes - buat
placeholder node.
188             self.db.execute_query("""
189                 MERGE (target:Definition {id:
$target_id})
190                 ON CREATE SET
191                     target.name = $short_name,
192                     target.fullName = $target_id,
193                     target.kind = 'SubResource',
194                     target.is_root = false,
195                     target.source = '
k8s_swagger_cross_ref',
196                     target.description = 'External
cross-reference from Swagger $ref',
197                     target.is_cross_reference =
true
198                 WITH target
199                 MATCH (source:Definition {id:
$source_id})

```

```

200             MERGE (source)-[r:HAS_PROPERTY {
name: $field_name}]->(target)
201             SET r.is_array = $is_array,
202                 r.is_map = $is_map,
203                 r.is_required = $is_required
204             "", {
205                 "source_id": full_name, "target_id
": target_name,
206                 "short_name": target_name.split(".")
)[-1],
207                 "field_name": field_name, "
is_array": is_array,
208                 "is_map": is_map, "is_required":
is_required
209             })
210             edge_count += 1
211
212             if primitive_props:
213                 self.db.execute_query(
214                     "MATCH (d:Definition {id: $full_name}) SET d
+= $primitive_props",
215                     {"full_name": full_name, "primitive_props":
primitive_props}
216                 )
217
218             print(f"    Created {edge_count} structural edges (
HAS_PROPERTY)")
219
220         def _pass_2b_create_inheritance_edges(self):
221             inheritance_count = union_count = 0
222
223             for full_name, schema in self.definitions.items():
224                 if full_name in IGNORE_LIST:
225                     continue
226
227                 for item in schema.get('allOf', []):
228                     if isinstance(item, dict) and '$ref' in item:
229                         parent_name = item['$ref'].split('/')[1]
230                         if parent_name not in IGNORE_LIST:
231                             self.db.execute_query(
232                                 "MATCH (child:Definition {id:
$child_id}), (parent:Definition {id: $parent_id}) "
233                                 "MERGE (child)-[:EXTENDS]->(parent)",
234                                 {"child_id": full_name, "parent_id":
parent_name}

```

```

235         )
236         inheritance_count += 1
237
238         for item in schema.get('oneOf', []) + schema.get('
anyOf', []):
239             if isinstance(item, dict) and '$ref' in item:
240                 option_name = item['$ref'].split('/')[ -1]
241                 if option_name not in IGNORE_LIST:
242                     self.db.execute_query(
243                         "MATCH (union:Definition {id:
$union_id}), (option:Definition {id: $option_id}) "
244                         "MERGE (union)-[:ONE_OF]->(option)",
245                         {"union_id": full_name, "option_id":
option_name}
246                     )
247                     union_count += 1
248
249             print(f"    Created {inheritance_count} inheritance and {
union_count} union edges")
250
251     def _pass_3_build_semantic_edges(self):
252         semantic_rules = [
253             ("Deployment → PodTemplate",
254              "MATCH (d:Definition {kind: 'Deployment'})-[:
HAS_PROPERTY {name: 'spec'}]->(spec)"
255              "-[:HAS_PROPERTY {name: 'template'}]->(t) MERGE (d)
-[:CONTAINS_POD_TEMPLATE]->(t)"),
256             ("ReplicaSet → PodTemplate",
257              "MATCH (rs:Definition {kind: 'ReplicaSet'})-[:
HAS_PROPERTY {name: 'spec'}]->(spec)"
258              "-[:HAS_PROPERTY {name: 'template'}]->(t) MERGE (rs)
-[:CONTAINS_POD_TEMPLATE]->(t)"),
259             ("DaemonSet → PodTemplate",
260              "MATCH (ds:Definition {kind: 'DaemonSet'})-[:
HAS_PROPERTY {name: 'spec'}]->(spec)"
261              "-[:HAS_PROPERTY {name: 'template'}]->(t) MERGE (ds)
-[:CONTAINS_POD_TEMPLATE]->(t)"),
262             ("Job → PodTemplate",
263              "MATCH (j:Definition {kind: 'Job'})-[:HAS_PROPERTY {
name: 'spec'}]->(spec)"
264              "-[:HAS_PROPERTY {name: 'template'}]->(t) MERGE (j)
-[:CONTAINS_POD_TEMPLATE]->(t)"),
265             ("StatefulSet → PodTemplate",
266              "MATCH (s:Definition {kind: 'StatefulSet'})-[:
HAS_PROPERTY {name: 'spec'}]->(spec)"

```

```

267         "-[:HAS_PROPERTY {name: 'template'}]->(t) MERGE (s)
-[:CONTAINS_POD_TEMPLATE]->(t)",
268         ("CronJob → JobTemplate",
269         "MATCH (cj:Definition {kind: 'CronJob'})-[:
HAS_PROPERTY {name: 'spec'}]->(spec)"
270         "-[:HAS_PROPERTY {name: 'jobTemplate'}]->(j) MERGE (
cj)-[:CONTAINS_JOB_TEMPLATE]->(j)"),
271         ("PodSpec → Container",
272         "MATCH (p:Definition {id: 'io.k8s.api.core.v1.PodSpec
'})"
273         "-[:HAS_PROPERTY {name: 'containers'}]->(c) MERGE (p)
-[:HAS_CONTAINER]->(c)",
274         ("StatefulSet → VolumeClaimTemplates",
275         "MATCH (s:Definition {kind: 'StatefulSet'})-[:
HAS_PROPERTY {name: 'spec'}]->(spec)"
276         "-[:HAS_PROPERTY {name: 'volumeClaimTemplates'}]->(
pvc) MERGE (s)-[:CLAIMS_VOLUME]->(pvc)",
277         ("PodSpec → Volume",
278         "MATCH (p:Definition {id: 'io.k8s.api.core.v1.PodSpec
'})"
279         "-[:HAS_PROPERTY {name: 'volumes'}]->(v) MERGE (p)-[:
MOUNTS_VOLUME]->(v)",
280         ("PVC → StorageClass",
281         "MATCH (pvc:Definition {kind: 'PersistentVolumeClaim
'}), (sc:Definition {kind: 'StorageClass'})"
282         " MERGE (pvc)-[:USES_STORAGE_CLASS]->(sc)",
283         ("Container → ConfigMap",
284         "MATCH (c:Definition {id: 'io.k8s.api.core.v1.
Container'}), (cm:Definition {kind: 'ConfigMap'})"
285         " MERGE (c)-[:LOADS_CONFIGMAP]->(cm)",
286         ("PodSpec → Secret",
287         "MATCH (p:Definition {id: 'io.k8s.api.core.v1.PodSpec
'}), (s:Definition {kind: 'Secret'})"
288         " MERGE (p)-[:USES_SECRET]->(s)",
289         ("Service → Pod Selector",
290         "MATCH (svc:Definition {kind: 'Service'}), (pod:
Definition {kind: 'Pod'})"
291         " MERGE (svc)-[:SELECTS_POD]->(pod)",
292         ("Ingress → Service",
293         "MATCH (i:Definition {kind: 'Ingress'}), (s:
Definition {kind: 'Service'})"
294         " MERGE (i)-[:ROUTES_TO_SERVICE]->(s)",
295         ("RoleBinding → Role",
296         "MATCH (rb:Definition {kind: 'RoleBinding'}), (r:
Definition {kind: 'Role'})"

```

```

297         " MERGE (rb)-[:BINDS_ROLE]->(r)" ),
298         ("ClusterRoleBinding → ClusterRole",
299         "MATCH (rb:Definition {kind: 'ClusterRoleBinding'}),
(r:Definition {kind: 'ClusterRole'})"
300         " MERGE (rb)-[:BINDS_ROLE]->(r)" ),
301         ("RoleBinding → ServiceAccount",
302         "MATCH (rb:Definition {kind: 'RoleBinding'}), (sa:
Definition {kind: 'ServiceAccount'})"
303         " MERGE (rb)-[:BINDS_SERVICE_ACCOUNT]->(sa)" ),
304         ("ClusterRoleBinding → ServiceAccount",
305         "MATCH (rb:Definition {kind: 'ClusterRoleBinding'}),
(sa:Definition {kind: 'ServiceAccount'})"
306         " MERGE (rb)-[:BINDS_SERVICE_ACCOUNT]->(sa)" ),
307         ("PodSpec → ServiceAccount",
308         "MATCH (p:Definition {id: 'io.k8s.api.core.v1.PodSpec
'})", (sa:Definition {kind: 'ServiceAccount'})"
309         " MERGE (p)-[:USES_SERVICE_ACCOUNT]->(sa)" ),
310         ("HPA → Deployment",
311         "MATCH (h:Definition {kind: 'HorizontalPodAutoscaler
'})", (t:Definition {kind: 'Deployment'})"
312         " MERGE (h)-[:SCALES_RESOURCE]->(t)" ),
313         ("HPA → StatefulSet",
314         "MATCH (h:Definition {kind: 'HorizontalPodAutoscaler
'})", (t:Definition {kind: 'StatefulSet'})"
315         " MERGE (h)-[:SCALES_RESOURCE]->(t)" ),
316         ("HPA → ReplicaSet",
317         "MATCH (h:Definition {kind: 'HorizontalPodAutoscaler
'})", (t:Definition {kind: 'ReplicaSet'})"
318         " MERGE (h)-[:SCALES_RESOURCE]->(t)" ),
319         ("Pod → PodSpec (Logical EXTENDS)", ""
320         MATCH (p:Definition {kind: 'Pod'}), (spec:
Definition {id: 'io.k8s.api.core.v1.PodSpec'})
321         MERGE (p)-[:EXTENDS]->(spec)
322         "" ),
323         ("Deployment → DeploymentSpec (Logical EXTENDS)", ""
324         MATCH (d:Definition {kind: 'Deployment'}), (spec:
Definition {id: 'io.k8s.api.apps.v1.DeploymentSpec'})
325         MERGE (d)-[:EXTENDS]->(spec)
326         "" ),
327         ("Volume → VolumeSources (Logical ONE_OF)", ""
328         MATCH (v:Definition {id: 'io.k8s.api.core.v1.
Volume'})-[:HAS_PROPERTY]->(target)
329         WHERE target.name IN ['ConfigMapVolumeSource', '
EmptyDirVolumeSource',

```

```

330         'SecretVolumeSource', '
        HostPathVolumeSource',
331         '
        PersistentVolumeClaimVolumeSource']
332         MERGE (v)-[:ONE_OF]->(target)
333         """),
334         ("EnvFrom → EnvSources (Logical ANY_OF)", ""
335         MATCH (env:Definition {id: 'io.k8s.api.core.v1.
        EnvFromSource'})-[:HAS_PROPERTY]->(target)
336         WHERE target.name IN ['ConfigMapEnvSource', '
        SecretEnvSource']
337         MERGE (env)-[:ANY_OF]->(target)
338         """)
339     ]
340
341     executed = 0
342     for rule_name, query in semantic_rules:
343         try:
344             self.db.execute_query(query)
345             executed += 1
346         except Exception as e:
347             print(f"    SKIPPED {rule_name}: {str(e)[:80]}")
348
349     print(f"    Executed {executed} semantic rules")

```

Listing A.2 src/ingestion/parser.py — Pipeline Ingestion Data
(SwaggerGraphBuilder)

A.3 LangGraph Agent Pipeline

Berkas src/chatbot/graph_agent.py mendefinisikan *state machine* LangGraph yang mengorkestrasi lima node pipeline: *memory* → *thinker* → *retriever* → *speaker* → *saver*.

```

1 # src/chatbot/graph_agent.py
2 import json
3 import logging
4 import operator
5 from typing import TypedDict, List, Annotated, Optional
6 from langgraph.graph import StateGraph, END
7 from langchain_core.messages import AIMessage, BaseMessage
8
9 from src.chatbot.llm_factory import get_thinker_llm,
    get_speaker_llm
10 from src.chatbot.prompts import INTENT_PROMPT, RESPONSE_PROMPT
11 from src.chatbot.custom_retriever import StatefulK8sRetriever

```

```

12 from src.memory.zep_store import ZepMemoryStore
13
14 logger = logging.getLogger(__name__)
15
16 # Batas karakter graph_context yang diteruskan ke Speaker.
17 # Ditentukan dari kapasitas konteks GPT-4o-mini dan pengujian
   empiris.
18 MAX_CONTEXT_CHARS = 12_000
19
20 _zep_store = None
21
22 def get_zep() -> ZepMemoryStore:
23     global _zep_store
24     if _zep_store is None:
25         _zep_store = ZepMemoryStore()
26     return _zep_store
27
28
29 class AgentState(TypedDict):
30     messages: Annotated[List[BaseMessage], operator.add]
31     question: str
32     session_id: str
33     chat_history: str
34     extracted_intent: dict
35     graph_context: str
36     reasoning_path: Optional[List[str]]
37     intent_type: Optional[str]
38     error: Optional[str]
39
40
41 def retrieve_memory_node(state: AgentState):
42     """Fetches conversation history from ZepMemoryStore."""
43     session_id = state.get("session_id", "default_session")
44     try:
45         history = get_zep().get_history(session_id=session_id,
limit=5)
46         return {"chat_history": history or "Belum ada riwayat
percakapan."}
47     except Exception as e:
48         logger.warning(f"Zep memory failed: {e}")
49         return {"chat_history": "Belum ada riwayat percakapan."}
50
51
52 def extract_intent_node(state: AgentState):
53     """

```

```

54     The 'Thinker'. Reads history + question, resolves pronouns,
55     and outputs a strict JSON intent for the custom retriever.
56     """
57     if state.get("error"):
58         return state
59
60     try:
61         llm = get_thinker_llm()
62         chain = INTENT_PROMPT | llm
63         response = chain.invoke({
64             "chat_history": state["chat_history"],
65             "question": state["question"]
66         })
67
68         raw_content = response.content.strip()
69         if raw_content.startswith("`"):
70             raw_content = raw_content.split("`")[1]
71             if raw_content.startswith("json"):
72                 raw_content = raw_content[4:]
73             raw_content = raw_content.strip()
74
75             intent_data = json.loads(raw_content)
76             intent_type = intent_data.get("intent_type", "explain")
77             return {"extracted_intent": intent_data, "intent_type":
intent_type}
78
79     except json.JSONDecodeError as e:
80         logger.error(f"Thinker failed to output valid JSON: {e}")
81         return {"error": "Failed to parse search intent from user
query."}
82     except Exception as e:
83         logger.error(f"Intent extraction failed: {e}")
84         return {"error": str(e)}
85
86
87 def _make_retrieval_node(ablation_mode: str | None = None):
88     """Returns an execute_retrieval_node closure bound to
ablation_mode."""
89     def execute_retrieval_node(state: AgentState):
90         """Passes the extracted JSON intent to the deterministic
Python retriever."""
91         if state.get("error"):
92             return {"graph_context": "Error in understanding
intent.", "reasoning_path": []}
93         try:

```

```

94         retriever = StatefulK8sRetriever()
95         intent_type = state.get("intent_type") or "explain"
96         graph_context, reasoning_path = retriever.
retrieve_context(
97             state["extracted_intent"],
98             intent_type=intent_type,
99             ablation_mode=ablation_mode,
100         )
101         return {"graph_context": graph_context, "
reasoning_path": reasoning_path}
102     except Exception as e:
103         logger.error(f"Custom Retrieval failed: {e}")
104         return {"graph_context": "Database retrieval failed.",
"reasoning_path": [], "error": str(e)}
105     return execute_retrieval_node
106
107
108 def generate_response_node(state: AgentState):
109     """The 'Speaker'. Uses LLM + graph context to formulate the
final answer."""
110     try:
111         _ERROR_STRINGS = (
112             "Database retrieval failed.",
113             "Error in understanding intent.",
114         )
115         raw_ctx = state.get("graph_context") or ""
116         if state.get("error") or any(e in raw_ctx for e in
_ERROR_STRINGS):
117             return {"messages": [AIMessage(content=(
118                 "Maaf, saya tidak dapat menarik konteks dari
Knowledge Graph saat ini. "
119                 "Mohon perjelas spesifikasi resource yang Anda
cari."
120             ))]}
121
122         llm = get_speaker_llm()
123         chain = RESPONSE_PROMPT | llm
124
125         raw_context = raw_ctx
126         if len(raw_context) > MAX_CONTEXT_CHARS:
127             raw_context = raw_context[:MAX_CONTEXT_CHARS] + "\n...
[context truncated]"
128
129         chat_history = state["chat_history"]
130         intent_type = state.get("intent_type") or "explain"

```

```

131     # Hindari label 'followup' di sesi baru yang belum punya
    riwayat nyata
132     if intent_type == "followup" and chat_history.strip() in (
        "", "Belum ada riwayat percakapan."):
133         intent_type = "explain"
134
135     response = chain.invoke({
136         "chat_history": chat_history,
137         "retrieved_data": raw_context,
138         "question": state["question"],
139         "intent_type": intent_type
140     })
141     return {"messages": [AIMessage(content=response.content)]}
142 except Exception as e:
143     logger.error(f"Response Gen failed: {e}")
144     return {"messages": [AIMessage(content="Terjadi error saat
    membuat respons.")]}
```

```

145
146
147 def save_memory_node(state: AgentState):
148     """Saves the completed conversation turn to ZepMemoryStore."""
149     session_id = state.get("session_id", "default_session")
150     try:
151         user_msg = state["question"]
152         ai_msg = state["messages"][-1].content if state["messages"
    ] else ""
153         if user_msg and ai_msg:
154             get_zep().add_message(session_id=session_id, user_msg=
    user_msg, ai_msg=ai_msg)
155     except Exception as e:
156         logger.warning(f"Failed to save memory: {e}")
157     return {}
158
159
160 def create_agent_graph(ablation_mode: str | None = None):
161     """Compiles the LangGraph state machine."""
162     workflow = StateGraph(AgentState)
163     workflow.add_node("memory", retrieve_memory_node)
164     workflow.add_node("thinker", extract_intent_node)
165     workflow.add_node("retriever", _make_retrieval_node(
    ablation_mode))
166     workflow.add_node("speaker", generate_response_node)
167     workflow.add_node("saver", save_memory_node)
168
169     workflow.set_entry_point("memory")

```

```

170 workflow.add_edge("memory", "thinker")
171 workflow.add_edge("thinker", "retriever")
172 workflow.add_edge("retriever", "speaker")
173 workflow.add_edge("speaker", "saver")
174 workflow.add_edge("saver", END)
175
176 return workflow.compile()

```

Listing A.3 src/chatbot/graph_agent.py — LangGraph Agent Pipeline

A.4 Mekanisme *Retrieval* Bertingkat (*StatefulK8sRetriever*)

Kelas *StatefulK8sRetriever* pada berkas src/chatbot/custom_retriever.py mengimplementasikan *cascading retrieval* dengan kontrol kedalaman berbasis *intent*.

```

1 # src/chatbot/custom_retriever.py
2 import json
3 import logging
4 from src.graph.neo4j_client import Neo4jClient
5 from src.graph.vector_index import VectorIndexManager
6 from src.graph.queries import (
7     EXACT_MATCH_QUERY,
8     SCHEMA_DEPS_QUERY,
9     PATH_EDGES_QUERY,
10    HYBRID_VECTOR_GRAPH_QUERY,
11 )
12
13 logger = logging.getLogger(__name__)
14
15 # Intent-aware depth mapping
16 # Batas kedalaman diturunkan dari properti struktural graf skema
17 # K8s,
18 # bukan dari dataset evaluasi:
19 #
20 # "explain" / "followup" → depth 2
21 # Node di depth -12 bersifat resource-specific (dirujuk oleh -
22 # 23 resource).
23
24 # Depth 3+ memasukkan tipe generik bersama (PodSpec dipakai
25 # oleh 23+ resource)
26 # yang menambah noise untuk pertanyaan definisi.
27 #
28 # "generate_yaml" / "trace_relationship" / "planning" → depth 3
29 # YAML generation butuh depth 3 untuk menjangkau field
30 # container-level
31 # (image, ports, env). Depth 4+ didominasi utility types (
32 # Quantity, IntOrString)

```

```

27 # yang dipakai oleh -19136 resource - tidak informatif untuk
    membedakan relasi.
28 #
29 _DEPTH_BY_INTENT = {
30     "explain": 2,
31     "followup": 2,
32     "generate_yaml": 3,
33     "trace_relationship": 3,
34     "planning": 3,
35 }
36 _DEFAULT_DEPTH = 3
37
38 # trace_relationship dikeluarkan dari multi-entity karena
    penambahan entity kedua
39 # menyebabkan precision penalty di RetQ (lebih banyak node dari
    yang relevan).
40 _MULTI_ENTITY_INTENTS = {"planning", "generate_yaml"}
41
42
43 class StatefulK8sRetriever:
44     def __init__(self):
45         self.db = Neo4jClient()
46         self.vector_mgr = VectorIndexManager()
47
48     def retrieve_context(
49         self,
50         intent_data: dict,
51         intent_type: str = "explain",
52         max_depth: int | None = None,
53         ablation_mode: str | None = None,
54     ) -> tuple[str, list[str]]:
55         """
56         Two-phase retrieval with intent-aware depth control:
57             Phase 1 - Exact name match (precision-first).
58             Phase 2 - Vector similarity fallback (recall).
59
60         Depth resolution priority:
61             1. ablation_mode override ('depth_2' / 'depth_3')
62             2. Explicit max_depth argument
63             3. _DEPTH_BY_INTENT mapping
64             4. _DEFAULT_DEPTH fallback
65
66         ablation_mode (None in production):
67             'no_phase1' A1: skip exact match
68             'no_multihop' A2: seed node only, no traversal

```

```

69         'depth_2'          A3: force depth=2
70         'depth_3'          A4: force depth=3
71         'no_multi_entity' A6c: disable multi-entity retrieval
72         """
73         if ablation_mode is not None and ablation_mode.startswith(
74             'depth_'):
75             try:
76                 depth = int(ablation_mode.split('_', 1)[1])
77             except (IndexError, ValueError):
78                 depth = max_depth if max_depth is not None \
79                     else _DEPTH_BY_INTENT.get(intent_type,
80 _DEFAULT_DEPTH)
81             else:
82                 depth = max_depth if max_depth is not None \
83                     else _DEPTH_BY_INTENT.get(intent_type,
84 _DEFAULT_DEPTH)
85
86         primary = intent_data.get("primary_resource", "")
87         related = intent_data.get("related_concepts", [])
88
89         try:
90             # Phase 1: Exact match (A1: skipped)
91             if ablation_mode == 'no_phase1':
92                 root_name = None
93             else:
94                 root_name = self._exact_match(primary)
95
96             if root_name:
97                 if ablation_mode == 'no_multihop':
98                     record = {"RootResource": root_name, "
99 SchemaDependencies": []}
100                 else:
101                     record = self._schema_deps(root_name, depth)
102             else:
103                 # Phase 2: Vector search
104                 search_query = f"{primary} {' '.join(related)}
105 Kubernetes"
106                 embedding = self.vector_mgr.generate_embedding(
107 search_query)
108                 record = self._vector_deps(embedding, depth)
109                 if record:
110                     root_name = record.get("RootResource", "")
111                     if ablation_mode == 'no_multihop':
112                         record = {
113                             "RootResource": root_name,

```

```

108         "Description": record.get("Description
", "" ),
109         "SchemaDependencies": [],
110     }
111
112     if not record:
113         return "Tidak ada skema Kubernetes yang relevan di
dalam Knowledge Graph.", []
114
115     deps = record.get("SchemaDependencies") or []
116     record["SchemaDependencies"] = [d for d in deps if d
is not None]
117
118     if ablation_mode == 'no_multihop':
119         reasoning_path = []
120     else:
121         reasoning_path = self._build_reasoning_path(
root_name, depth)
122
123     graph_context = json.dumps(record, indent=2,
ensure_ascii=False)
124
125     # Multi-entity retrieval: gabungkan konteks dari
hingga 2 related_concepts.
126     # Diperlukan untuk planning/generate_yaml yang
mencakup 2+ resource
127     # yang tidak dapat dicapai dalam 3 hop dari primary
saja.
128     effective_multi_entity = (
129         set() if ablation_mode == 'no_multi_entity' else
_MULTI_ENTITY_INTENTS
130     )
131     if intent_type in effective_multi_entity and related:
132         for extra_resource in related[:2]:
133             try:
134                 extra_root = self._exact_match(
extra_resource)
135
136                 if not extra_root:
137                     continue
138                 extra_record = self._schema_deps(
extra_root, depth)
139
140                 if not extra_record:
141                     continue
142                 extra_deps = extra_record.get("
SchemaDependencies") or []

```

```

141         extra_record["SchemaDependencies"] = [d
for d in extra_deps if d is not None]
142         extra_path = self._build_reasoning_path(
extra_root, depth)
143         graph_context += "\n" + json.dumps(
extra_record, indent=2, ensure_ascii=False)
144         seen_steps = set(reasoning_path)
145         for step in extra_path:
146             if step not in seen_steps:
147                 reasoning_path.append(step)
148                 seen_steps.add(step)
149         except Exception as ex:
150             logger.warning(f"[Retriever] Multi-entity
failed for '{extra_resource}': {ex}")
151
152         return graph_context, reasoning_path
153
154     except Exception as e:
155         logger.error(f"[Retriever] Graph traversal failed: {e}
")
156         return f"Error retrieving context from Neo4j: {str(e)}
", []
157
158     def _exact_match(self, primary: str) -> str | None:
159         """Returns the canonical node name if an exact match
exists, else None."""
160         if not primary:
161             return None
162         rows = self.db.execute_query(EXACT_MATCH_QUERY, {"
primary_resource": primary})
163         return rows[0]["name"] if rows else None
164
165     def _schema_deps(self, root_name: str, max_depth: int) -> dict
| None:
166         """Fetch schema dependencies for a known root node name.
"""
167         cypher = SCHEMA_DEPS_QUERY.format(max_depth=max_depth)
168         rows = self.db.execute_query(cypher, {"root_name":
root_name})
169         return dict(rows[0]) if rows else None
170
171     def _vector_deps(self, embedding: list, max_depth: int) ->
dict | None:
172         """Fetch schema dependencies via vector similarity."""

```

```

173     cypher = HYBRID_VECTOR_GRAPH_QUERY.format(max_depth=
max_depth)
174     rows = self.db.execute_query(cypher, {"embedding":
embedding})
175     return dict(rows[0]) if rows else None
176
177     def _build_reasoning_path(self, root_name: str, max_depth: int
) -> list[str]:
178         """
179         Returns deduplicated list of actual →parentchild edge
strings, e.g.:
180         "Deployment →[HAS_PROPERTY] DeploymentSpec"
181         "DeploymentSpec →[HAS_PROPERTY] PodTemplateSpec"
182
183         Menggunakan PATH_EDGES_QUERY yang mengekstrak node
perantara nyata
184         dari jalur graf - bukan pintasan langsung root ke leaf.
185         """
186         if not root_name:
187             return []
188         try:
189             cypher = PATH_EDGES_QUERY.format(max_depth=max_depth)
190             rows = self.db.execute_query(cypher, {"root_name":
root_name})
191             seen, path = set(), []
192             for row in rows:
193                 edge = f"{row['parent']} →[{row['rel_type']}]→ {
row['child']}"
194                 if edge not in seen:
195                     seen.add(edge)
196                     path.append(edge)
197             return path
198         except Exception as e:
199             logger.warning(f"[Retriever] Could not build reasoning
path: {e}")
200         return []

```

Listing A.4 src/chatbot/custom_retriever.py — Mekanisme Retrieval Bertingkat

A.5 Validasi YAML Tiga Lapis (*YAMLValidator*)

Kelas *YAMLValidator* pada berkas src/validation/yaml_validator.py mengimplementasikan validasi YAML tiga lapis: sintaksis PyYAML, kepatuhan skema *kubernetes-validate*, dan verifikasi *required fields* dari *knowledge graph*.

```

1 # src/validation/yaml_validator.py
2 import logging
3 import yaml
4 from src.graph.neo4j_client import Neo4jClient
5 from src.graph.queries import REQUIRED_FIELDS_QUERY
6
7 logger = logging.getLogger(__name__)
8
9
10 class YAMLValidator:
11     """
12     Three-layer YAML validation:
13     1. PyYAML - syntax correctness
14     2. kubernetes-validate - schema compliance against K8s 1.29
15     spec
16     3. Neo4j graph - required fields cross-check (thesis
17     contribution)
18     """
19
20     def __init__(self):
21         self.db = Neo4jClient()
22
23     def validate(self, yaml_string: str, kind: str) -> dict:
24         result = {
25             "valid": True,
26             "syntax_errors": [],
27             "schema_errors": [],
28             "missing_fields": [],
29         }
30
31         # Layer 1: PyYAML syntax
32         try:
33             data = yaml.safe_load(yaml_string)
34         except yaml.YAMLError as e:
35             result["valid"] = False
36             result["syntax_errors"].append(str(e))
37             return result
38
39         if not isinstance(data, dict):
40             result["valid"] = False
41             result["syntax_errors"].append("YAML root must be a
42             mapping, got: " + type(data).__name__)
43             return result
44
45         # Layer 2: kubernetes-validate schema

```

```

43     try:
44         import kubernetes_validate
45         kubernetes_validate.validate(data, "1.29", strict=
False)
46     except ImportError:
47         logger.warning("kubernetes-validate not installed;
skipping schema validation layer.")
48     except Exception as e:
49         result["valid"] = False
50         path = "".join(str(p) for p in getattr(e, "path", []))
51         msg = getattr(e, "message", str(e))
52         result["schema_errors"].append(f"{path}: {msg}" if
path else msg)
53
54     # Layer 3: Neo4j required fields
55     try:
56         rows = self.db.execute_query(REQUIRED_FIELDS_QUERY, {"
kind": kind})
57         required_fields = {r["field_name"] for r in rows}
58     except Exception as e:
59         logger.warning(f"Could not fetch required fields from
graph: {e}")
60         required_fields = set()
61
62     if required_fields:
63         yaml_keys = set(self._flatten_keys(data))
64         missing = required_fields - yaml_keys
65         if missing:
66             result["valid"] = False
67             result["missing_fields"] = sorted(missing)
68
69     return result
70
71     def _flatten_keys(self, obj, prefix: str = "") -> list[str]:
72         """Recursively extract dotted key paths from a nested dict
. """
73         keys = []
74         if isinstance(obj, dict):
75             for k, v in obj.items():
76                 full_key = f"{prefix}.{k}" if prefix else k
77                 keys.append(full_key)
78                 keys.extend(self._flatten_keys(v, full_key))
79         elif isinstance(obj, list):
80             for item in obj:
81                 keys.extend(self._flatten_keys(item, prefix))

```

```
82         return keys
```

Listing A.5 src/validation/yaml_validator.py — Validasi YAML Tiga Lapis

A.6 Query Cypher Neo4j

Berkas src/graph/queries.py mendefinisikan seluruh konstanta query Cypher yang digunakan oleh sistem sebagai *single source of truth*. Query ini mencakup *exact match*, traversal multi-hop, *vector search*, dan validasi *required fields*.

```
1  # src/graph/queries.py
2  # Centralized Cypher query constants - single source of truth for
   all graph queries.
3
4  #
   -----
5  # Exact name match - cari root node berdasarkan nama persis.
6  # Parameter: $primary_resource (str)
7  #
   -----
8  EXACT_MATCH_QUERY = """
9  MATCH (d:Definition)
10 WHERE d.name = $primary_resource
11      OR d.name ENDS WITH ('.' + $primary_resource)
12 RETURN d.name AS name, d.kind AS kind, d.description AS
   description
13 ORDER BY
14     CASE WHEN d.name = $primary_resource THEN 0 ELSE 1 END
15 LIMIT 1
16 """
17
18 #
   -----
19 # Schema dependencies - membangun daftar flat dependensi untuk
   konteks LLM.
20 # Parameter: $root_name (str), max_depth via .format()
21 #
   -----
22 SCHEMA_DEPS_QUERY = """
23 MATCH (root:Definition {{name: $root_name}})
```

```

24 OPTIONAL MATCH (root)-[r:HAS_PROPERTY*1..{max_depth}]->(child:
    Definition)
25
26 WITH root, r, child
27
28 WITH root,
29     CASE
30         WHEN child IS NOT NULL AND r IS NOT NULL THEN {{
31             path_depth:      size(r),
32             relation_type:    type(last(r)),
33             yaml_field:       last(r).name,
34             is_array:         coalesce(last(r).is_array, false),
35             child_resource:    child.name,
36             child_description: substring(child.description, 0,
150)
37         }}
38         ELSE null
39     END AS dep
40
41 RETURN root.name      AS RootResource,
42        root.kind       AS RootKind,
43        root.description AS RootDescription,
44        1.0             AS VectorSimilarityScore,
45        collect(dep)    AS SchemaDependencies
46 ""
47
48 #
-----
49 # Daftar semua 18 jenis edge dalam KG Kubernetes (digunakan di
    PATH_EDGES_QUERY).
50 #   HAS_PROPERTY           - referensi tipe skema (struktural, dari
    $ref)
51 #   SCALES_RESOURCE       - HPA → Deployment/StatefulSet/
    ReplicaSet
52 #   CONTAINS_POD_TEMPLATE - Deployment/StatefulSet/DaemonSet/Job
    → PodTemplateSpec
53 #   CONTAINS_JOB_TEMPLATE - CronJob → JobTemplateSpec
54 #   BINDS_ROLE            - RoleBinding/ClusterRoleBinding → Role/
    ClusterRole
55 #   BINDS_SERVICE_ACCOUNT - RoleBinding/ClusterRoleBinding →
    ServiceAccount
56 #   EXTENDS               - Deployment/Pod → DeploymentSpec/
    PodSpec (logis)
57 #   HAS_CONTAINER        - PodSpec → Container

```

```

58 # CLAIMS_VOLUME          - StatefulSet → PVC
59 # MOUNTS_VOLUME          - PodSpec → Volume
60 # USES_STORAGE_CLASS     - PVC → StorageClass
61 # LOADS_CONFIGMAP        - Container → ConfigMap
62 # USES_SECRET            - PodSpec → Secret
63 # SELECTS_POD            - Service → Pod
64 # ROUTES_TO_SERVICE      - Ingress → Service
65 # USES_SERVICE_ACCOUNT   - PodSpec → ServiceAccount
66 # ONE_OF / ANY_OF        - polimorfisme tipe (allOf/oneOf/anyOf
OpenAPI)
67 #
-----

68 _ALL_EDGE_TYPES = (
69     "HAS_PROPERTY | SCALES_RESOURCE | CONTAINS_POD_TEMPLATE |
CONTAINS_JOB_TEMPLATE"
70     " | BINDS_ROLE | BINDS_SERVICE_ACCOUNT | EXTENDS | HAS_CONTAINER"
71     " | CLAIMS_VOLUME | MOUNTS_VOLUME | USES_STORAGE_CLASS |
LOADS_CONFIGMAP"
72     " | USES_SECRET | SELECTS_POD | ROUTES_TO_SERVICE |
USES_SERVICE_ACCOUNT"
73     " | ONE_OF | ANY_OF"
74 )
75
76 #
-----

77 # Path edges - untuk reasoning path (explainability).
78 # Mengembalikan pasangan →parentchild nyata di setiap hop.
79 # Parameter: $root_name (str), max_depth via .format()
80 #
-----

81 PATH_EDGES_QUERY = """
82 MATCH p = (root:Definition {{name: $root_name}})
83           -[:{all_edges}*1..{max_depth}]->(leaf:Definition)
84 WITH p
85 LIMIT 500
86 WITH [i IN range(0, size(nodes(p))-2) | {{
87     parent:    nodes(p)[i].name,
88     child:     nodes(p)[i+1].name,
89     rel_type:  type(relationships(p)[i]),
90     depth:     i + 1
91 }}] AS edges
92 UNWIND edges AS edge

```

```

93 RETURN DISTINCT edge.parent AS parent,
94                  edge.child AS child,
95                  edge.rel_type AS rel_type,
96                  edge.depth AS depth
97 ORDER BY edge.depth ASC, edge.parent ASC
98 LIMIT 50
99 """.replace("{all_edges}", _ALL_EDGE_TYPES)
100
101 #
-----
102 # Hybrid vector + graph retrieval - digunakan saat exact match
    gagal.
103 # Parameter: $embedding (list[float]), max_depth via .format()
104 #
-----

105 HYBRID_VECTOR_GRAPH_QUERY = """
106 CALL db.index.vector.queryNodes('definition_description_vector',
    1, $embedding)
107 YIELD node AS root, score
108
109 OPTIONAL MATCH (root)-[r:HAS_PROPERTY*1..{max_depth}]->(child:
    Definition)
110
111 WITH root, score, r, child
112
113 WITH root, score,
114     CASE
115         WHEN child IS NOT NULL AND r IS NOT NULL THEN {{
116             path_depth:      size(r),
117             relation_type:    type(last(r)),
118             yaml_field:       last(r).name,
119             is_array:         coalesce(last(r).is_array, false),
120             child_resource:    child.name,
121             child_description: substring(child.description, 0,
122 150)
123         }}
124     ELSE null
125 END AS dep
126
127 RETURN root.name AS RootResource,
128        root.kind AS RootKind,
129        root.description AS RootDescription,
130        score AS VectorSimilarityScore,

```

```

130         collect(dep)          AS SchemaDependencies
131     """
132
133     #
134     -----
135
136     # Required fields - untuk validasi YAML Layer 3 (YAMLValidator).
137     #
138     -----
139
140     REQUIRED_FIELDS_QUERY = """
141     MATCH (d:Definition {name: $kind})-[r:HAS_PROPERTY {is_required:
142         true}]->(p)
143     RETURN p.name AS field_name
144     """
145
146     ALL_FIELDS_QUERY = """
147     MATCH (d:Definition {name: $kind})-[r:HAS_PROPERTY]->(p)
148     RETURN p.name AS field_name, r.is_required AS is_required
149     """

```

Listing A.6 src/graph/queries.py — Query Cypher Neo4j

A.7 Fungsi Metrik Evaluasi

Potongan berikut dari scripts/evaluate.py menampilkan tiga fungsi komputasi metrik inti: compute_ansq(), compute_retq(), dan compute_reaq(). Bagian *orchestrator* (mode invoker, checkpoint, dan summary printing) dihilangkan untuk singkatnya; implementasi lengkap tersedia di repositori proyek.

```

1 # scripts/evaluate.py - Fungsi komputasi metrik evaluasi (baris
2   1-453 dari skrip lengkap)
3 # Bagian yang dihilangkan: mode invokers, checkpoint/resume logic,
4   summary printing.
5 # Skrip lengkap tersedia di repositori: scripts/evaluate.py
6 """
7 Dimensi evaluasi:
8   AnsQ (40%): Answer Quality - syntactic validity, schema
9     compliance, faithfulness, answer relevance
10  RetQ (35%): Retrieval Quality - precision@k, recall@k, F1@k,
11     graph coverage, NDCG@k, edge_coverage
12  ReaQ (25%): Reasoning Quality - hop accuracy, multi-hop success,
13     scope accuracy, grounding score
14 """
15
16 import re
17
18 import math

```

```

12
13 # Scope question detection keywords
14 _SCOPE_Q_KEYWORDS = [
15     "scope", "scoped", "namespaced", "cluster-scoped", "cluster-
16     wide",
17     "namespace-level", "cluster-level", "lingkup", "cakupan",
18     "bisa diakses lintas", "seluruh cluster", "non-namespaced",
19 ]
20 # K8s term regex - untuk grounding check (hanya CamelCase +
21     single-word K8s)
22 _K8S_TERM_RE = re.compile(
23     r'\b(?:'
24     r'[A-Z][a-z]+(?:[A-Z][a-zA-Z]+)'
25     r'|Deployment|StatefulSet|DaemonSet|ReplicaSet|CronJob|Ingress'
26     r'|ConfigMap|Secret|Namespace|ServiceAccount|Endpoints|Pod|
27     Service'
28     r'|Node|ResourceQuota|LimitRange|NetworkPolicy|StorageClass'
29     r'|Role|ClusterRole|RoleBinding|ClusterRoleBinding'
30     r'|HorizontalPodAutoscaler|PersistentVolume|
31     PersistentVolumeClaim'
32     r'|apiVersion|kubectl|namespace[sd]?|pod[sd]?|deployment[sd]?|
33     service[sd]?'
34     r'|configmap[sd]?|secret[sd]?|ingress(?:es)?|statefulset[sd]?|
35     daemonset[sd]?'
36     r'|replicaset[sd]?|cronjob[sd]?|job[sd]?|persistentvolume(?:
37     claim)?[sd]?'
38     r'|clusterrole(?:binding)?[sd]?|rolebinding[sd]?|
39     serviceaccount[sd]?'
40     r'|networkpolicy|hpa|pvc|pv|rbac'
41     r')\b'
42 )
43
44 def _token_f1(pred: str, gold: str) -> float:
45     """Token-level F1 fallback when embedder unavailable."""
46     pred_tokens = set(pred.lower().split())
47     gold_tokens = set(gold.lower().split())
48     if not pred_tokens or not gold_tokens:
49         return 0.0
50     common = pred_tokens & gold_tokens
51     precision = len(common) / len(pred_tokens)
52     recall = len(common) / len(gold_tokens)

```

```

47     return 2 * precision * recall / (precision + recall) if (
precision + recall) > 0 else 0.0
48
49
50 def _cosine_similarity(embedder, text1: str, text2: str) -> float:
51     """Cosine similarity between two text embeddings."""
52     import numpy as np
53     e1 = embedder.embed_query(text1)
54     e2 = embedder.embed_query(text2)
55     v1, v2 = np.array(e1), np.array(e2)
56     denom = np.linalg.norm(v1) * np.linalg.norm(v2)
57     return float(np.dot(v1, v2) / denom) if denom > 0 else 0.0
58
59
60 def _effective_type(fixture_type: str, ground_truth: dict) -> str:
61     """Resolve 'realworld' to its concrete sub-type if specified
in ground_truth."""
62     if fixture_type == "realworld":
63         return ground_truth.get("realworld_subtype", "realworld")
64     return fixture_type
65
66
67 def _extract_yaml_block(text: str) -> str:
68     """Extract YAML block from markdown fenced code or plain text.
"""
69     if "```yaml" in text:
70         parts = text.split("```yaml", 1)
71         if len(parts) > 1:
72             yaml_lines = parts[1].split("```")[0].strip().
splitlines()
73             return "\n".join(yaml_lines).strip()
74     if "```" in text:
75         parts = text.split("```", 1)
76         if len(parts) > 1:
77             yaml_lines = parts[1].split("```")[0].strip().
splitlines()
78             return "\n".join(yaml_lines).strip()
79     return text.strip()
80
81
82 def compute_ansq(
83     answer: str,
84     ground_truth: dict,
85     fixture_type: str,
86     embedder=None,

```

```

87     ablation_mode: str | None = None,
88 ) -> dict:
89     """
90     Answer Quality metrics.
91
92     Sub-metrics:
93     syntactic_validity - (yaml_gen only) apakah YAML yang
94     digenerate dapat diparse?
95     schema_compliance - (yaml_gen only) apakah YAML lolos
96     kubernetes-validate?
97     answer_relevance - cosine similarity vs ground truth
98     answer
99     faithfulness - fraksi expected nodes yang disebut
100     dalam jawaban
101     layer3_compliance - (yaml_gen, ablation only) Neo4j
102     required-field check
103     """
104     scores = {}
105     fixture_type = _effective_type(fixture_type, ground_truth)
106
107     if fixture_type == "yaml_gen":
108         yaml_candidate = _extract_yaml_block(answer)
109         try:
110             import yaml
111             yaml.safe_load(yaml_candidate)
112             scores["syntactic_validity"] = 1.0
113         except Exception:
114             scores["syntactic_validity"] = 0.0
115     else:
116         scores["syntactic_validity"] = None
117
118     if fixture_type == "yaml_gen":
119         yaml_candidate = _extract_yaml_block(answer)
120         try:
121             import yaml
122             import kubernetes_validate
123             data = yaml.safe_load(yaml_candidate)
124             if isinstance(data, dict):
125                 kubernetes_validate.validate(data, "1.29", strict=
False)
126
127             scores["schema_compliance"] = 1.0
128         else:
129             scores["schema_compliance"] = 0.0
130         except ImportError:
131             scores["schema_compliance"] = None

```

```

126         except Exception:
127             scores["schema_compliance"] = 0.0
128         else:
129             scores["schema_compliance"] = None
130
131         gt_answer = ground_truth.get("answer", "")
132         if embedder is not None and gt_answer:
133             scores["answer_relevance"] = _cosine_similarity(embedder,
134             answer, gt_answer)
135         else:
136             scores["answer_relevance"] = _token_f1(answer, gt_answer)
137
138         # Faithfulness: fraksi expected nodes yang dirujuk dalam
139         # jawaban.
140         # Gunakan key_nodes (node kunci yang dikurasi) jika tersedia,
141         # fallback ke relevant_nodes.
142         # Menangani bentuk jamak (Pod → pods, Namespace → namespaces)
143         # dalam jawaban bahasa Indonesia.
144         def _node_matches(node: str, answer_lower: str) -> bool:
145             nl = node.lower()
146             return nl in answer_lower or nl + "s" in answer_lower or
147             nl + "es" in answer_lower
148
149         faith_source = ground_truth.get("key_nodes") or ground_truth.
150         get("relevant_nodes", [])
151         gt_nodes = [n.split(".")[0] for n in faith_source]
152         hit = sum(1 for n in gt_nodes if _node_matches(n, answer.lower
153         ()))
154         scores["faithfulness"] = hit / len(gt_nodes) if gt_nodes else
155         1.0
156
157         # Layer 3 compliance - hanya untuk ablation study (bukan
158         # production run atau A5)
159         if fixture_type == "yaml_gen" and ablation_mode is not None
160         and ablation_mode != "no_yaml_layer3":
161             yaml_candidate = _extract_yaml_block(answer)
162             try:
163                 import yaml as _yaml
164                 _data = _yaml.safe_load(yaml_candidate)
165                 if isinstance(_data, dict):
166                     from src.validation.yaml_validator import
167                     YAMLValidator
168                     _kind = _data.get("kind", "")
169                     _vresult = YAMLValidator().validate(yaml_candidate
170                     , _kind)

```

```

159         scores["layer3_compliance"] = None if _vresult["
syntax_errors"] \
160             else (1.0 if not _vresult["missing_fields"]
else 0.0)
161     else:
162         scores["layer3_compliance"] = None
163     except Exception:
164         scores["layer3_compliance"] = None
165 else:
166     scores["layer3_compliance"] = None
167
168 applicable = [v for v in scores.values() if v is not None]
169 scores["ansq_score"] = sum(applicable) / len(applicable) if
applicable else 0.0
170 return scores
171
172
173 def compute_retq(reasoning_path: list, ground_truth: dict) -> dict
:
174     """
175     Retrieval Quality metrics.
176
177     Sub-metrics (semua berkontribusi setara ke retq_score):
178     precision_at_k - fraksi retrieved nodes yang relevan
179     recall_at_k    - fraksi relevant nodes yang berhasil
diambil
180     f1_at_k        - rata-rata harmonik precision dan recall
181     graph_coverage - fraksi expected path SOURCE-nodes yang
cocok
182     ndcg_at_k      - kualitas peringkat: relevant node di awal
lebih baik
183     edge_coverage  - fraksi expected edges dalam reasoning_path
184     """
185     _RELATION_RE = re.compile(r"-\[([^\]]+)\]->?")
186
187     def _node_tokens(step: str) -> list:
188         cleaned = _RELATION_RE.sub(" ", step)
189         return [t for t in cleaned.split() if t]
190
191     expected_nodes = set(n.split(".")[1] for n in ground_truth.
get("relevant_nodes", []))
192     retrieved_nodes, seen_nodes = [], set()
193     for step in reasoning_path:
194         for tok in _node_tokens(step):
195             if tok not in seen_nodes:

```

```

196         seen_nodes.add(tok)
197         retrieved_nodes.append(tok)
198     retrieved_set = set(retrieved_nodes)
199
200     intersection = retrieved_set & expected_nodes
201     precision = len(intersection) / len(retrieved_set) if
retrieved_set else 0.0
202     recall    = len(intersection) / len(expected_nodes) if
expected_nodes else 1.0
203     f1 = 2 * precision * recall / (precision + recall) if (
precision + recall) > 0 else 0.0
204
205     expected_path = ground_truth.get("expected_path", [])
206     graph_coverage = (
207         len([p for p in expected_path
208             if any(p.split(" -")[0] in step for step in
reasoning_path)])
209         / len(expected_path) if expected_path else 1.0
210     )
211
212     k = len(retrieved_nodes)
213     dcg = sum(
214         (1.0 / math.log2(i + 2))
215         for i, node in enumerate(retrieved_nodes)
216         if node in expected_nodes
217     )
218     ideal_hits = min(len(expected_nodes), k) if k > 0 else 0
219     idcg = sum(1.0 / math.log2(i + 2) for i in range(ideal_hits))
220     ndcg_at_k = dcg / idcg if idcg > 0 else (1.0 if not
expected_nodes else 0.0)
221
222     if expected_path:
223         matched_edges = sum(
224             1 for ep in expected_path
225             if any(ep in step or step in ep for step in
reasoning_path)
226         )
227         edge_coverage = matched_edges / len(expected_path)
228     else:
229         edge_coverage = 1.0
230
231     retq_score = (precision + recall + f1 + graph_coverage +
ndcg_at_k + edge_coverage) / 6
232
233     return {

```

```

234     "precision_at_k": precision,
235     "recall_at_k": recall,
236     "f1_at_k": f1,
237     "graph_coverage": graph_coverage,
238     "ndcg_at_k": ndcg_at_k,
239     "edge_coverage": edge_coverage,
240     "retq_score": retq_score,
241 }
242
243
244 def compute_reaq(
245     reasoning_path: list,
246     answer: str,
247     ground_truth: dict,
248     fixture_type: str,
249     graph_context: str = "",
250     fixture_scope: str = "",
251     question: str = "",
252     k8s_vocabulary: set = None,
253     cgg_mode: bool = False,
254 ) -> dict:
255     """
256     Reasoning Quality metrics.
257
258     Sub-metrics:
259         hop_accuracy          - berapa expected path hops yang benar-
260         benar dilalui
261         multi_hop_success     - apakah pertanyaan multi-hop
262         menghasilkan traversal?
263         scope_accuracy        - hanya dievaluasi ketika pertanyaan
264         menyebutkan scope
265
266         grounding_score       - ATAU resource adalah Cluster-scoped
267         1 - hallucination_rate.
268         Mode normal: dicocokkan terhadap
269         vocab K8s global (Neo4j).
270         CGG mode: dicocokkan terhadap
271         graph_context query ini saja.
272     """
273     fixture_type = _effective_type(fixture_type, ground_truth)
274     expected_path = ground_truth.get("expected_path", [])
275     multi_hop = ground_truth.get("multi_hop", False)
276
277     if expected_path and multi_hop:
278         matched = sum(
279             1 for ep in expected_path

```

```

274         if any(ep.split(" -")[0] in step for step in
reasoning_path)
275     )
276     hop_accuracy = matched / len(expected_path)
277 else:
278     hop_accuracy = 1.0 if reasoning_path else 0.0
279
280 multi_hop_success = 1.0 if (multi_hop and reasoning_path) or (
not multi_hop) else 0.0
281
282 # Scope accuracy - kondisional: hanya dievaluasi jika
pertanyaan menyebut scope
283 # atau resource adalah Cluster-scoped. Namespaced resource
tanpa pertanyaan scope
284 # diberi skor 1.0 (tidak dihukum atas default Namespaced yang
benar).
285 question_lower = question.lower()
286 scope_relevant = (
287     any(kw in question_lower for kw in _SCOPE_Q_KEYWORDS)
288     or fixture_scope == "Cluster"
289 )
290
291 if not scope_relevant:
292     scope_accuracy = 1.0
293 else:
294     _NAMESPACEED_POSITIVE = ["namespaced", "dalam namespace", "
di namespace",
295                             "namespace-scoped", "namespace
tertentu"]
296     _NAMESPACEED_NEGATIVE = ["cluster-scoped", "cluster-wide",
"seluruh cluster",
297                             "tidak terikat namespace", "non-
namespaced"]
298     _CLUSTER_POSITIVE = ["cluster-scoped", "cluster-wide",
"seluruh cluster",
299                             "tidak terikat namespace", "non-
namespaced", "lintas namespace"]
300     _CLUSTER_NEGATIVE = ["namespaced", "dalam namespace", "
namespace-scoped"]
301
302     answer_lower = answer.lower()
303     if fixture_scope == "Namespaced":
304         correct_hit = any(kw in answer_lower for kw in
_NAMESPACEED_POSITIVE)

```

```

305         wrong_hit = any(kw in answer_lower for kw in
_NAMESPACED_NEGATIVE)
306         scope_accuracy = 1.0 if (correct_hit and not wrong_hit
) else (0.0 if wrong_hit else 0.5)
307         elif fixture_scope == "Cluster":
308             correct_hit = any(kw in answer_lower for kw in
_CLUSTER_POSITIVE)
309             wrong_hit = any(kw in answer_lower for kw in
_CLUSTER_NEGATIVE)
310             scope_accuracy = 1.0 if (correct_hit and not wrong_hit
) else (0.0 if wrong_hit else 0.5)
311         else:
312             scope_accuracy = 1.0
313
314     if cgg_mode:
315         from src.validation.cgg_validator import
cgg_grounding_score
316         grounding_score, hallucination_rate = cgg_grounding_score(
answer, graph_context)
317     else:
318         answer_terms = set(t.lower() for t in _K8S_TERM_RE.findall
(answer))
319         if answer_terms:
320             if k8s_vocabulary:
321                 grounded = sum(1 for t in answer_terms if t in
k8s_vocabulary)
322             else:
323                 ctx_lower = graph_context.lower()
324                 grounded = sum(1 for t in answer_terms if t in
ctx_lower)
325                 hallucination_rate = 1.0 - grounded / len(answer_terms
)
326         else:
327             hallucination_rate = 0.0
328             grounding_score = 1.0 - hallucination_rate
329
330     reaq_score = (hop_accuracy + multi_hop_success +
scope_accuracy + grounding_score) / 4
331
332     return {
333         "hop_accuracy": hop_accuracy,
334         "multi_hop_success": multi_hop_success,
335         "scope_accuracy": scope_accuracy,
336         "hallucination_rate": hallucination_rate,
337         "grounding_score": grounding_score,

```

```
338     "req_score": req_score,
339 }
```

Listing A.7 scripts/evaluate.py — Fungsi Metrik Evaluasi (AnsQ, RetQ, ReaQ)

LAMPIRAN B

FIXTURE EVALUASI PAKAR

Lampiran ini menyajikan 10 *fixture* evaluasi yang digunakan dalam validasi pakar (*expert evaluation*) sebagaimana diuraikan pada Subbab ???. Setiap *fixture* mewakili satu pertanyaan beserta *ground truth* yang mencakup *relevant nodes* dan *expected path* di *knowledge graph*. *Field* answer dan context dikecualikan dari tampilan ini untuk mempertahankan objektivitas penilaian. Tabel B.1 merangkum distribusi 10 *fixture* tersebut.

Tabel B.1 Ringkasan 10 Fixture Evaluasi Pakar

No	ID	Tipe	Resource Utama	Scope
1	deployment_basic	konseptual	Deployment	Namespaced
2	service_types	konseptual	Service	Namespaced
3	scale_existing_deployment	lanjutan	Deployment	Namespaced
4	ingress_service_pod	relasional	Ingress	Namespaced
5	hpa_deployment_pod	relasional	HorizontalPodAutoscaler	Namespaced
6	cronjob_backup	generasi YAML	CronJob	Namespaced
7	networkpolicy_deny_all	generasi YAML	NetworkPolicy	Namespaced
8	statefulset_with_pvc	generasi YAML	StatefulSet	Namespaced
9	pi_want_to_reject_...	skenario nyata	Namespace	Cluster
10	kubectl_find_pods_...	perintah	Pod	Namespaced

B.1 Fixture B.1 — deployment_basic (Konseptual)

Fixture ini menguji pemahaman sistem terhadap mekanisme *RollingUpdate* Deployment: bagaimana Pod diperbarui saat *image* berubah dan apakah terjadi *downtime*.

```
1 {
2   "id": "deployment_basic",
3   "type": "conceptual",
4   "question": "Apa yang terjadi pada Pod yang sedang berjalan
               ketika saya update image Deployment ke versi baru? Apakah ada
               downtime?",
```

```

5  "resource": "io.k8s.api.apps.v1.Deployment",
6  "scope": "Namespaced",
7  "multi_hop": true,
8  "ground_truth": {
9    "relevant_nodes": [
10     "io.k8s.api.apps.v1.Deployment",
11     "io.k8s.api.apps.v1.DeploymentCondition",
12     "io.k8s.api.apps.v1.DeploymentSpec",
13     "io.k8s.api.apps.v1.DeploymentStatus",
14     "io.k8s.api.apps.v1.DeploymentStrategy",
15     "io.k8s.api.apps.v1.RollingUpdateDeployment",
16     "io.k8s.api.core.v1.PodSpec",
17     "io.k8s.api.core.v1.PodTemplateSpec",
18     "io.k8s.apimachinery.pkg.apis.meta.v1.LabelSelector"
19   ],
20   "expected_path": [
21     "Deployment -[CONTAINS_POD_TEMPLATE]-> PodTemplateSpec",
22     "Deployment -[EXTENDS]-> DeploymentSpec",
23     "Deployment -[HAS_PROPERTY]-> DeploymentSpec",
24     "Deployment -[HAS_PROPERTY]-> DeploymentStatus",
25     "DeploymentSpec -[HAS_PROPERTY]-> DeploymentStrategy",
26     "DeploymentSpec -[HAS_PROPERTY]-> LabelSelector",
27     "DeploymentSpec -[HAS_PROPERTY]-> PodTemplateSpec",
28     "DeploymentStatus -[HAS_PROPERTY]-> DeploymentCondition",
29     "DeploymentStrategy -[HAS_PROPERTY]->
30     RollingUpdateDeployment",
31     "PodTemplateSpec -[HAS_PROPERTY]-> PodSpec"
32   ]
33 }

```

Listing B.1 fixture-deployment-basic.json — Konseptual: Pembaruan Image Deployment

B.2 Fixture B.2 — service_types (Konseptual)

Fixture ini menguji kemampuan sistem membandingkan dua mekanisme *expose* aplikasi: Service tipe LoadBalancer versus Ingress, beserta pertimbangan *trade-off*-nya.

```

1  {
2    "id": "service_types",
3    "type": "conceptual",
4    "question": "Kapan sebaiknya menggunakan Service tipe
5    LoadBalancer dibanding Ingress untuk expose aplikasi ke
6    internet? Apa perbedaan dan trade-off-nya?",

```

```

5  "resource": "io.k8s.api.core.v1.Service",
6  "scope": "Namespaced",
7  "multi_hop": true,
8  "ground_truth": {
9    "relevant_nodes": [
10     "io.k8s.api.core.v1.LoadBalancerStatus",
11     "io.k8s.api.core.v1.Pod",
12     "io.k8s.api.core.v1.PodSpec",
13     "io.k8s.api.core.v1.PodStatus",
14     "io.k8s.api.core.v1.Service",
15     "io.k8s.api.core.v1.ServicePort",
16     "io.k8s.api.core.v1.ServiceSpec",
17     "io.k8s.api.core.v1.ServiceStatus",
18     "io.k8s.api.core.v1.SessionAffinityConfig",
19     "io.k8s.api.networking.v1.Ingress",
20     "io.k8s.api.networking.v1.IngressRule",
21     "io.k8s.api.networking.v1.IngressSpec",
22     "io.k8s.apimachinery.pkg.apis.meta.v1.Condition"
23   ],
24   "expected_path": [
25     "Ingress -[HAS_PROPERTY]-> IngressSpec",
26     "IngressSpec -[HAS_PROPERTY]-> IngressRule",
27     "Pod -[EXTENDS]-> PodSpec",
28     "Pod -[HAS_PROPERTY]-> PodSpec",
29     "Pod -[HAS_PROPERTY]-> PodStatus",
30     "Service -[HAS_PROPERTY]-> ServiceSpec",
31     "Service -[HAS_PROPERTY]-> ServiceStatus",
32     "Service -[SELECTS_POD]-> Pod",
33     "ServiceSpec -[HAS_PROPERTY]-> ServicePort",
34     "ServiceSpec -[HAS_PROPERTY]-> SessionAffinityConfig",
35     "ServiceStatus -[HAS_PROPERTY]-> Condition",
36     "ServiceStatus -[HAS_PROPERTY]-> LoadBalancerStatus"
37   ]
38 }
39 }

```

Listing B.2 fixture-service-types.json — Konseptual: Service LoadBalancer vs. Ingress

B.3 Fixture B.3 — scale_existing_deployment (Lanjutan)

Fixture bertipe *followup* ini mengukur kemampuan sistem memanfaatkan konteks percakapan sebelumnya (pembuatan Deployment nginx) untuk menjawab pertanyaan lanjutan tentang perubahan jumlah replika.

```

1 {

```

```

2  "id": "scale_existing_deployment",
3  "type": "followup",
4  "context_question": "Buat YAML Deployment nginx dengan 3 replika",
5  "question": "Sekarang ubah jumlah replikanya menjadi 5",
6  "resource": "io.k8s.api.apps.v1.Deployment",
7  "scope": "Namespaced",
8  "multi_hop": false,
9  "ground_truth": {
10   "relevant_nodes": [
11     "io.k8s.api.apps.v1.Deployment",
12     "io.k8s.api.apps.v1.DeploymentCondition",
13     "io.k8s.api.apps.v1.DeploymentSpec",
14     "io.k8s.api.apps.v1.DeploymentStatus",
15     "io.k8s.api.apps.v1.DeploymentStrategy",
16     "io.k8s.api.core.v1.PodSpec",
17     "io.k8s.api.core.v1.PodTemplateSpec",
18     "io.k8s.apimachinery.pkg.apis.meta.v1.LabelSelector"
19   ],
20   "expected_path": [
21     "Deployment -[CONTAINS_POD_TEMPLATE]-> PodTemplateSpec",
22     "Deployment -[EXTENDS]-> DeploymentSpec",
23     "Deployment -[HAS_PROPERTY]-> DeploymentSpec",
24     "Deployment -[HAS_PROPERTY]-> DeploymentStatus",
25     "DeploymentSpec -[HAS_PROPERTY]-> DeploymentStrategy",
26     "DeploymentSpec -[HAS_PROPERTY]-> LabelSelector",
27     "DeploymentSpec -[HAS_PROPERTY]-> PodTemplateSpec",
28     "DeploymentStatus -[HAS_PROPERTY]-> DeploymentCondition",
29     "PodTemplateSpec -[HAS_PROPERTY]-> PodSpec"
30   ],
31   "expected_yaml_keys": ["spec.replicas"]
32 }
33 }

```

Listing B.3 fixture-scale-existing-deployment.json — Lanjutan:
Skalakan Replika Deployment

B.4 Fixture B.4 — ingress_service_pod (Relasional)

Fixture ini menguji penelusuran relasi multi-hop: alur traffic dari Ingress melalui Service hingga ke Pod, dan identifikasi titik kegagalan pada error 502.

```

1  {
2    "id": "ingress_service_pod",
3    "type": "relationship",

```

```

4  "question": "Saya punya Ingress dengan host app.example.com yang
    diarahkan ke Service frontend-svc port 80. Pod saya statusnya
    Running tapi request dari luar masih dapat response 502 Bad
    Gateway. Bagaimana alur traffic dari Ingress sampai ke Pod dan
    di mana biasanya masalah terjadi?",
5  "resource": "io.k8s.api.networking.v1.Ingress",
6  "scope": "Namespaced",
7  "multi_hop": true,
8  "ground_truth": {
9    "relevant_nodes": [
10     "io.k8s.api.core.v1.LoadBalancerStatus",
11     "io.k8s.api.core.v1.Pod",
12     "io.k8s.api.core.v1.PodSpec",
13     "io.k8s.api.core.v1.PodStatus",
14     "io.k8s.api.core.v1.Service",
15     "io.k8s.api.core.v1.ServicePort",
16     "io.k8s.api.core.v1.ServiceSpec",
17     "io.k8s.api.core.v1.ServiceStatus",
18     "io.k8s.api.core.v1.SessionAffinityConfig",
19     "io.k8s.api.core.v1.TypedLocalObjectReference",
20     "io.k8s.api.networking.v1.HTTPIngressPath",
21     "io.k8s.api.networking.v1.HTTPIngressRuleValue",
22     "io.k8s.api.networking.v1.Ingress",
23     "io.k8s.api.networking.v1.IngressBackend",
24     "io.k8s.api.networking.v1.IngressLoadBalancerIngress",
25     "io.k8s.api.networking.v1.IngressLoadBalancerStatus",
26     "io.k8s.api.networking.v1.IngressRule",
27     "io.k8s.api.networking.v1.IngressServiceBackend",
28     "io.k8s.api.networking.v1.IngressSpec",
29     "io.k8s.api.networking.v1.IngressStatus",
30     "io.k8s.api.networking.v1.IngressTLS",
31     "io.k8s.apimachinery.pkg.apis.meta.v1.Condition"
32   ],
33   "expected_path": [
34     "HTTPIngressPath -[HAS_PROPERTY]-> IngressBackend",
35     "HTTPIngressRuleValue -[HAS_PROPERTY]-> HTTPIngressPath",
36     "Ingress -[HAS_PROPERTY]-> IngressSpec",
37     "Ingress -[HAS_PROPERTY]-> IngressStatus",
38     "Ingress -[ROUTES_TO_SERVICE]-> Service",
39     "IngressBackend -[HAS_PROPERTY]-> IngressServiceBackend",
40     "IngressBackend -[HAS_PROPERTY]-> TypedLocalObjectReference"
41   ],
42   "IngressLoadBalancerStatus -[HAS_PROPERTY]->
    IngressLoadBalancerIngress",
    "IngressRule -[HAS_PROPERTY]-> HTTPIngressRuleValue",

```

```

43     "IngressSpec -[HAS_PROPERTY]-> IngressBackend",
44     "IngressSpec -[HAS_PROPERTY]-> IngressRule",
45     "IngressSpec -[HAS_PROPERTY]-> IngressTLS",
46     "IngressStatus -[HAS_PROPERTY]-> IngressLoadBalancerStatus",
47     "Pod -[EXTENDS]-> PodSpec",
48     "Pod -[HAS_PROPERTY]-> PodSpec",
49     "Pod -[HAS_PROPERTY]-> PodStatus",
50     "Service -[HAS_PROPERTY]-> ServiceSpec",
51     "Service -[HAS_PROPERTY]-> ServiceStatus",
52     "Service -[SELECTS_POD]-> Pod",
53     "ServiceSpec -[HAS_PROPERTY]-> ServicePort",
54     "ServiceSpec -[HAS_PROPERTY]-> SessionAffinityConfig",
55     "ServiceStatus -[HAS_PROPERTY]-> Condition",
56     "ServiceStatus -[HAS_PROPERTY]-> LoadBalancerStatus"
57 ],
58 "key_nodes": [
59     "io.k8s.api.networking.v1.Ingress",
60     "io.k8s.api.networking.v1.IngressSpec",
61     "io.k8s.api.networking.v1.IngressRule",
62     "io.k8s.api.core.v1.Service"
63 ]
64 }
65 }

```

Listing B.4 fixture-ingress-service-pod.json — Relasional: Alur Traffic
Ingress→Service→Pod

B.5 Fixture B.5 — hpa_deployment_pod (Relasional)

Fixture ini menguji kemampuan sistem menjelaskan siklus kerja HPA: alur dari pembacaan metrik CPU hingga pembaruan spec.replicas di Deployment, dan diagnosis ketika HPA tidak berfungsi.

```

1 {
2   "id": "hpa_deployment_pod",
3   "type": "relationship",
4   "question": "Saya sudah set HPA dengan
   targetCPUUtilizationPercentage 50% untuk Deployment saya, tapi
   meskipun CPU usage sudah di atas 80% selama beberapa menit,
   jumlah Pod tidak bertambah. Bagaimana HPA seharusnya mengatur
   jumlah Pod dan apa yang mungkin menyebabkan HPA tidak bekerja?",
5   "resource": "io.k8s.api.autoscaling.v2.HorizontalPodAutoscaler",
6   "scope": "Namespaced",
7   "multi_hop": true,
8   "ground_truth": {

```

```

9      "relevant_nodes": [
10          "io.k8s.api.apps.v1.Deployment",
11          "io.k8s.api.apps.v1.ReplicaSet",
12          "io.k8s.api.apps.v1.StatefulSet",
13          "io.k8s.api.autoscaling.v2.CrossVersionObjectReference",
14          "io.k8s.api.autoscaling.v2.HorizontalPodAutoscaler",
15          "io.k8s.api.autoscaling.v2.HorizontalPodAutoscalerBehavior",
16          "io.k8s.api.autoscaling.v2.HorizontalPodAutoscalerCondition"
17      ],
18      "io.k8s.api.autoscaling.v2.HorizontalPodAutoscalerSpec",
19      "io.k8s.api.autoscaling.v2.HorizontalPodAutoscalerStatus",
20      "io.k8s.api.autoscaling.v2.MetricSpec",
21      "io.k8s.api.autoscaling.v2.MetricStatus",
22      "io.k8s.api.autoscaling.v2.ResourceMetricSource",
23      "io.k8s.api.core.v1.PodSpec",
24      "io.k8s.api.core.v1.PodTemplateSpec"
25  ],
26  "expected_path": [
27      "Deployment -[CONTAINS_POD_TEMPLATE]-> PodTemplateSpec",
28      "Deployment -[EXTENDS]-> DeploymentSpec",
29      "Deployment -[HAS_PROPERTY]-> DeploymentSpec",
30      "Deployment -[HAS_PROPERTY]-> DeploymentStatus",
31      "HorizontalPodAutoscaler -[HAS_PROPERTY]->
HorizontalPodAutoscalerSpec",
32      "HorizontalPodAutoscaler -[HAS_PROPERTY]->
HorizontalPodAutoscalerStatus",
33      "HorizontalPodAutoscaler -[SCALES_RESOURCE]-> Deployment",
34      "HorizontalPodAutoscaler -[SCALES_RESOURCE]-> ReplicaSet",
35      "HorizontalPodAutoscaler -[SCALES_RESOURCE]-> StatefulSet",
36      "HorizontalPodAutoscalerBehavior -[HAS_PROPERTY]->
HPAScalingRules",
37      "HorizontalPodAutoscalerSpec -[HAS_PROPERTY]->
CrossVersionObjectReference",
38      "HorizontalPodAutoscalerSpec -[HAS_PROPERTY]->
HorizontalPodAutoscalerBehavior",
39      "HorizontalPodAutoscalerSpec -[HAS_PROPERTY]-> MetricSpec",
40      "HorizontalPodAutoscalerStatus -[HAS_PROPERTY]->
HorizontalPodAutoscalerCondition",
41      "HorizontalPodAutoscalerStatus -[HAS_PROPERTY]->
MetricStatus",
42      "MetricSpec -[HAS_PROPERTY]-> ContainerResourceMetricSource"
43  ],
44      "MetricSpec -[HAS_PROPERTY]-> ResourceMetricSource",
45      "PodTemplateSpec -[HAS_PROPERTY]-> PodSpec"
46  ],

```

```

45     "key_nodes": [
46         "io.k8s.api.autoscaling.v2.HorizontalPodAutoscaler",
47         "io.k8s.api.autoscaling.v2.HorizontalPodAutoscalerSpec",
48         "io.k8s.api.autoscaling.v2.CrossVersionObjectReference",
49         "io.k8s.api.apps.v1.Deployment"
50     ]
51 }
52 }

```

Listing B.5 fixture-hpa-deployment-pod.json — Relasional: HPA dan Penskalaan Otomatis

B.6 Fixture B.6 — cronjob_backup (Generasi YAML)

Fixture generasi YAML ini mengukur kemampuan sistem menghasilkan manifes CronJob yang valid dengan jadwal *cron* yang tepat dan konfigurasi *job template* yang benar.

```

1 {
2     "id": "cronjob_backup",
3     "type": "yaml_gen",
4     "question": "Buatkan YAML CronJob untuk menjalankan backup
5         database setiap hari jam 2 pagi",
6     "resource": "io.k8s.api.batch.v1.CronJob",
7     "scope": "Namespaced",
8     "multi_hop": true,
9     "ground_truth": {
10         "relevant_nodes": [
11             "io.k8s.api.batch.v1.CronJob",
12             "io.k8s.api.batch.v1.CronJobSpec",
13             "io.k8s.api.batch.v1.CronJobStatus",
14             "io.k8s.api.batch.v1.JobSpec",
15             "io.k8s.api.batch.v1.JobTemplateSpec",
16             "io.k8s.api.batch.v1.PodFailurePolicy",
17             "io.k8s.api.batch.v1.SuccessPolicy",
18             "io.k8s.api.core.v1.ObjectReference",
19             "io.k8s.api.core.v1.PodTemplateSpec",
20             "io.k8s.apimachinery.pkg.apis.meta.v1.LabelSelector"
21         ],
22         "expected_path": [
23             "CronJob -[CONTAINS_JOB_TEMPLATE]-> JobTemplateSpec",
24             "CronJob -[HAS_PROPERTY]-> CronJobSpec",
25             "CronJob -[HAS_PROPERTY]-> CronJobStatus",
26             "CronJobSpec -[HAS_PROPERTY]-> JobTemplateSpec",
27             "CronJobStatus -[HAS_PROPERTY]-> ObjectReference",
28             "JobSpec -[HAS_PROPERTY]-> LabelSelector",

```

```

28     "JobSpec -[HAS_PROPERTY]-> PodFailurePolicy",
29     "JobSpec -[HAS_PROPERTY]-> PodTemplateSpec",
30     "JobSpec -[HAS_PROPERTY]-> SuccessPolicy",
31     "JobTemplateSpec -[HAS_PROPERTY]-> JobSpec"
32 ],
33 "required_fields": ["apiVersion", "kind", "metadata", "spec"],
34 "expected_yaml_keys": [
35     "spec.schedule",
36     "spec.jobTemplate",
37     "spec.jobTemplate.spec.template"
38 ]
39 }
40 }

```

Listing B.6 fixture-cronjob-backup.json — Generasi YAML: CronJob Backup Database

B.7 Fixture B.7 — networkpolicy_deny_all (Generasi YAML)

Fixture ini menguji generasi NetworkPolicy dengan *empty podSelector* untuk memblokir semua *ingress traffic* ke namespace—pola keamanan yang umum tetapi memerlukan pemahaman semantik podSelector: {}.

```

1 {
2   "id": "networkpolicy_deny_all",
3   "type": "yaml_gen",
4   "question": "Buatkan YAML NetworkPolicy untuk memblokir semua
5     traffic masuk ke namespace production",
6   "resource": "io.k8s.api.networking.v1.NetworkPolicy",
7   "scope": "Namespaced",
8   "multi_hop": false,
9   "ground_truth": {
10     "relevant_nodes": [
11       "io.k8s.api.networking.v1.NetworkPolicy",
12       "io.k8s.api.networking.v1.NetworkPolicyEgressRule",
13       "io.k8s.api.networking.v1.NetworkPolicyIngressRule",
14       "io.k8s.api.networking.v1.NetworkPolicyPeer",
15       "io.k8s.api.networking.v1.NetworkPolicyPort",
16       "io.k8s.api.networking.v1.NetworkPolicySpec",
17       "io.k8s.apimachinery.pkg.apis.meta.v1.LabelSelector",
18       "io.k8s.apimachinery.pkg.apis.meta.v1.
19     LabelSelectorRequirement"
20     ],
21     "expected_path": [
22       "LabelSelector -[HAS_PROPERTY]-> LabelSelectorRequirement",
23       "NetworkPolicy -[HAS_PROPERTY]-> NetworkPolicySpec",

```

```

22     "NetworkPolicyEgressRule -[HAS_PROPERTY]-> NetworkPolicyPeer
23     ",
24     "NetworkPolicyEgressRule -[HAS_PROPERTY]-> NetworkPolicyPort
25     ",
26     "NetworkPolicyIngressRule -[HAS_PROPERTY]->
27     NetworkPolicyPeer",
28     "NetworkPolicyIngressRule -[HAS_PROPERTY]->
29     NetworkPolicyPort",
30     "NetworkPolicySpec -[HAS_PROPERTY]-> LabelSelector",
31     "NetworkPolicySpec -[HAS_PROPERTY]-> NetworkPolicyEgressRule
32     ",
33     "NetworkPolicySpec -[HAS_PROPERTY]->
34     NetworkPolicyIngressRule"
35 ],
36 "required_fields": ["apiVersion", "kind", "metadata", "spec"],
37 "expected_yaml_keys": ["spec.podSelector", "spec.policyTypes"]
38 }
39 }

```

Listing B.7 fixture-networkpolicy-deny-all.json — Generasi YAML:
NetworkPolicy Deny-All

B.8 Fixture B.8 — statefulset_with_pvc (Generasi YAML)

Fixture ini menguji generasi StatefulSet dengan volumeClaimTemplates—field yang spesifik untuk StatefulSet dan tidak ada di Deployment—untuk menguji kemampuan sistem membedakan kedua *workload* tersebut.

```

1 {
2   "id": "statefulset_with_pvc",
3   "type": "yaml_gen",
4   "question": "Buat YAML StatefulSet untuk database MySQL dengan
5   PersistentVolumeClaim 10Gi",
6   "resource": "io.k8s.api.apps.v1.StatefulSet",
7   "scope": "Namespaced",
8   "multi_hop": true,
9   "ground_truth": {
10     "relevant_nodes": [
11       "io.k8s.api.apps.v1.StatefulSet",
12       "io.k8s.api.apps.v1.StatefulSetSpec",
13       "io.k8s.api.apps.v1.StatefulSetStatus",
14       "io.k8s.api.apps.v1.StatefulSetUpdateStrategy",
15       "io.k8s.api.core.v1.Container",
16       "io.k8s.api.core.v1.PersistentVolumeClaim",
17       "io.k8s.api.core.v1.PersistentVolumeClaimSpec",
18       "io.k8s.api.core.v1.PodSpec",

```

```

18     "io.k8s.api.core.v1.PodTemplateSpec",
19     "io.k8s.api.storage.v1.StorageClass",
20     "io.k8s.apimachinery.pkg.apis.meta.v1.LabelSelector"
21 ],
22 "expected_path": [
23     "LabelSelector -[HAS_PROPERTY]-> LabelSelectorRequirement",
24     "PersistentVolumeClaim -[HAS_PROPERTY]->
25 PersistentVolumeClaimSpec",
26     "PersistentVolumeClaim -[HAS_PROPERTY]->
27 PersistentVolumeClaimStatus",
28     "PersistentVolumeClaim -[USES_STORAGE_CLASS]-> StorageClass"
29 ,
30     "PersistentVolumeClaimSpec -[HAS_PROPERTY]->
31 VolumeResourceRequirements",
32     "PodSpec -[HAS_CONTAINER]-> Container",
33     "PodSpec -[HAS_PROPERTY]-> Container",
34     "PodSpec -[MOUNTS_VOLUME]-> Volume",
35     "PodTemplateSpec -[HAS_PROPERTY]-> PodSpec",
36     "StatefulSet -[CLAIMS_VOLUME]-> PersistentVolumeClaim",
37     "StatefulSet -[CONTAINS_POD_TEMPLATE]-> PodTemplateSpec",
38     "StatefulSet -[HAS_PROPERTY]-> StatefulSetSpec",
39     "StatefulSet -[HAS_PROPERTY]-> StatefulSetStatus",
40     "StatefulSetSpec -[HAS_PROPERTY]-> LabelSelector",
41     "StatefulSetSpec -[HAS_PROPERTY]-> PersistentVolumeClaim",
42     "StatefulSetSpec -[HAS_PROPERTY]-> PodTemplateSpec",
43     "StatefulSetSpec -[HAS_PROPERTY]-> StatefulSetUpdateStrategy"
44 ,
45     "StatefulSetUpdateStrategy -[HAS_PROPERTY]->
46 RollingUpdateStatefulSetStrategy"
47 ],
48 "required_fields": ["apiVersion", "kind", "metadata", "spec"],
49 "expected_yaml_keys": [
50     "apiVersion",
51     "kind",
52     "metadata",
53     "spec",
54     "spec.volumeClaimTemplates"
55 ],
56 "key_nodes": [
57     "io.k8s.api.apps.v1.StatefulSet",
58     "io.k8s.api.apps.v1.StatefulSetSpec",
59     "io.k8s.api.core.v1.PersistentVolumeClaim"
60 ]
61 }

```

56 }

Listing B.8 fixture-statefulset-with-pvc.json — Generasi YAML:
StatefulSet dengan PVC

B.9 Fixture B.9 — pi_want_to_reject_all_docker (Skenario Nyata)

Fixture bertipe *realworld* ini berasal dari pertanyaan Stack Overflow. Pertanyaan tentang kebijakan pembatasan *docker registry* menguji kemampuan sistem mengarahkan pengguna ke konsep *Admission Controller* yang tidak memiliki representasi langsung sebagai node di KG.

```
1 {
2   "id": "pi_want_to_reject_all_docker",
3   "type": "realworld",
4   "question": "I want to reject all docker registries except my
5               own one. I'm looking for some kind of policies for docker
6               registries and their images. For example my registry name is
7               registry.my.company.com. How can I enforce this at the cluster
8               level?",
9   "resource": "io.k8s.api.core.v1.Namespace",
10  "scope": "Cluster",
11  "multi_hop": true,
12  "ground_truth": {
13    "relevant_nodes": [
14      "io.k8s.api.core.v1.Namespace",
15      "io.k8s.api.core.v1.NamespaceCondition",
16      "io.k8s.api.core.v1.NamespaceSpec",
17      "io.k8s.api.core.v1.NamespaceStatus"
18    ],
19    "expected_path": [
20      "Namespace -[HAS_PROPERTY]-> NamespaceSpec",
21      "Namespace -[HAS_PROPERTY]-> NamespaceStatus",
22      "NamespaceStatus -[HAS_PROPERTY]-> NamespaceCondition"
23    ],
24    "key_nodes": [
25      "io.k8s.api.core.v1.Namespace"
26    ]
27  }
28 }
```

Listing B.9 fixture-pi-reject-docker-registry.json — Skenario Nyata:
Kebijakan Registry Docker

B.10 Fixture B.10 — kubectl_find_pods_with_env (Perintah)

Fixture bertipe *command* ini menguji kemampuan sistem memberikan perintah `kubectl` yang tepat untuk mencari *environment variable* di seluruh Pod—pertanyaan operasional yang membutuhkan pemahaman struktur `Container.env` dan `envFrom`.

```
1 {
2   "id": "kubectl_find_pods_with_env",
3   "type": "command",
4   "question": "Bagaimana cara mencari tahu di Pod mana saja
5     environment variable DATABASE_URL digunakan di seluruh
6     namespace cluster Kubernetes saya?",
7   "resource": "io.k8s.api.core.v1.Pod",
8   "scope": "Namespaced",
9   "multi_hop": false,
10  "ground_truth": {
11    "relevant_nodes": [
12      "io.k8s.api.core.v1.ConfigMap",
13      "io.k8s.api.core.v1.Container",
14      "io.k8s.api.core.v1.EnvFromSource",
15      "io.k8s.api.core.v1.EnvVar",
16      "io.k8s.api.core.v1.Pod",
17      "io.k8s.api.core.v1.PodSpec",
18      "io.k8s.api.core.v1.PodStatus",
19      "io.k8s.api.core.v1.Secret",
20      "io.k8s.api.core.v1.ServiceAccount",
21      "io.k8s.api.core.v1.Volume",
22      "io.k8s.apimachinery.pkg.apis.meta.v1.LabelSelector"
23    ],
24    "expected_path": [
25      "Container -[HAS_PROPERTY]-> ContainerPort",
26      "Container -[HAS_PROPERTY]-> EnvFromSource",
27      "Container -[HAS_PROPERTY]-> EnvVar",
28      "Container -[HAS_PROPERTY]-> VolumeMount",
29      "Container -[LOADS_CONFIGMAP]-> ConfigMap",
30      "Pod -[EXTENDS]-> PodSpec",
31      "Pod -[HAS_PROPERTY]-> PodSpec",
32      "Pod -[HAS_PROPERTY]-> PodStatus",
33      "PodSpec -[HAS_CONTAINER]-> Container",
34      "PodSpec -[HAS_PROPERTY]-> Container",
35      "PodSpec -[HAS_PROPERTY]-> Volume",
36      "PodSpec -[MOUNTS_VOLUME]-> Volume",
37      "PodSpec -[USES_SECRET]-> Secret",
38      "PodSpec -[USES_SERVICE_ACCOUNT]-> ServiceAccount",
39      "PodStatus -[HAS_PROPERTY]-> ContainerStatus"
40    ]
41  }
```

```
39 }  
40 }
```

Listing B.10 `fixture-kubectl-find-pods-with-env.json` — Perintah:
Pencarian Environment Variable

LAMPIRAN C

HASIL EVALUASI KUANTITATIF

Lampiran ini menyajikan nilai metrik lengkap untuk seluruh 97 *fixture* evaluasi pada sistem *GraphRAG* Kubernetes. Tabel C.1 menampilkan metrik *Answer Quality* (AnsQ); Tabel C.2 menampilkan metrik *Retrieval Quality* (RetQ); dan Tabel C.3 menampilkan metrik *Reasoning Quality* (ReaQ). Kolom **Sint.** dan **Skema** pada Tabel C.1 hanya relevan untuk *fixture* bertipe `yaml`; nilai “—” menunjukkan bahwa metrik tersebut tidak diukur untuk tipe *fixture* yang bersangkutan.

Singkatan tipe *fixture*: *cmd* = perintah, *cncpt* = konseptual, *flwlp* = lanjutan, *plan* = perencanaan, *rlwld* = skenario nyata, *rel* = relasional, *trbl* = *troubleshooting*, *yaml* = generasi YAML.

C.1 Metrik *Answer Quality* (AnsQ)

Tabel C.1 Nilai Metrik AnsQ per Fixture (97 Fixture)

ID Fixture	Tipe	Sint.	Skema	Relevansi	Faithf.	AnsQ
kubect1_export_namespaces	cmd	—	—	0.4174	0.0000	0.2087
kubect1_find_pods_with_label	cmd	—	—	0.7674	0.0116	0.3895
kubect1_force_delete_pod	cmd	—	—	0.7962	0.0116	0.4039
configmap_usage	cncpt	—	—	0.7678	0.3333	0.5506
daemonset_purpose	cncpt	—	—	0.7732	0.3333	0.5533
deployment_basic	cncpt	—	—	0.8018	0.5000	0.6509
hpa_target	cncpt	—	—	0.5205	0.3333	0.4269
ingress_purpose	cncpt	—	—	0.6785	0.2500	0.4642
job_cronjob	cncpt	—	—	0.8016	0.4000	0.6008
namespace_quota	cncpt	—	—	0.6737	0.0000	0.3368
persistent_volume_concept	cncpt	—	—	0.6664	0.6000	0.6332
pod_spec	cncpt	—	—	0.6904	1.0000	0.8452
required_fields_container	cmd	—	—	0.6077	1.0000	0.8038

ID Fixture	Type	Sint.	Skema	Relevansi	Faithf.	AnsQ
scope_accuracy_node	cncpt	—	—	0.7184	0.6667	0.6925
secret_types	cncpt	—	—	0.8506	0.6667	0.7587
service_types	cncpt	—	—	0.7992	0.5000	0.6496
statefulset_storage	cncpt	—	—	0.7939	0.5000	0.6470
storageclass_concept	cncpt	—	—	0.8760	1.0000	0.9380
add_env_from_configmap	flwp	—	—	0.5690	0.1667	0.3678
add_env_from_secret	flwp	—	—	0.5823	0.3333	0.4578
add_hpa_to_deployment	flwp	—	—	0.7081	0.2083	0.4582
add_liveness_probe	flwp	—	—	0.6307	0.2727	0.4517
add_pvc_to_statefulset	flwp	—	—	0.6511	0.6667	0.6589
add_readiness_probe	flwp	—	—	0.6041	1.0000	0.8021
add_resource_limits	flwp	—	—	0.5564	0.6667	0.6115
add_resource_limits_deployment	flwp	—	—	0.5515	0.2000	0.3758
change_service_type	flwp	—	—	0.6098	0.5000	0.5549
expose_with_ingress	flwp	—	—	0.7803	0.3333	0.5568
scale_existing_deployment	flwp	—	—	0.6792	0.5000	0.5896
update_image_version	flwp	—	—	0.6225	0.2222	0.4224
plan_autoscale_ha	plan	—	—	0.6911	0.0909	0.3910
plan_cronjob_batch	plan	—	—	0.6626	0.2308	0.4467
plan_namespace_isolation	plan	—	—	0.8077	0.1818	0.4948
plan_redis_persistent	plan	—	—	0.8006	0.1136	0.4571
plan_webapp_with_db	plan	—	—	0.7631	0.2059	0.4845
configmap_volume_mount	twld	1.00	1.00	0.5487	0.7500	0.8247
deployment_vs_statefulset	twld	—	—	0.8647	0.5000	0.6824
hpa_custom_metrics_yaml	twld	1.00	0.00	0.6137	0.2500	0.4659
pbased_on_a_hrefhttps://stackoverflow.com/	twld	—	—	0.4141	0.0000	0.2071
pcopied_from_here_a_hrefhttps://medium.com/	twld	—	—	0.3424	0.0000	0.1712
pheres_a_simplified_version_of_kubernetes	twld	—	—	0.6634	1.0000	0.8317
pi_am_quite_new_to_tech	twld	—	—	0.6609	0.0000	0.3304
pi_am_wondering_if_it_is_possible_to_run_kubernetes_on_raspberry_pi	twld	—	—	0.6937	1.0000	0.8468
pi_cant_find_documentation_on_how_to_run_kubernetes_on_raspberry_pi	twld	—	—	0.4701	0.0000	0.2350
pi_have_a_command_to_run_kubernetes_on_raspberry_pi	twld	—	—	0.4113	0.0000	0.2057
pi_have_a_google_kubernetes_engine_instance	twld	—	—	0.5042	0.0000	0.2521

ID Fixture	Type	Sint.	Skema	Relevansi	Faithf.	AnsQ
pi_have_a_tekton_codepipeline	codepipeline	—	—	0.4149	0.0000	0.2075
pi_want_to_pass_a_certificate	certificate	—	—	0.3096	1.0000	0.6548
pi_want_to_reject_all_requests	request	—	—	0.4162	0.0000	0.2081
pid_like_to_confirm_information	information	—	—	0.6205	1.0000	0.8102
pim_trying_to_use_a_container	container	—	—	0.4446	1.0000	0.7223
pim_using_argocd_and_docker	docker	1.00	0.00	0.3478	0.0000	0.3369
pkubectl_provides_a_nodeway	nodeway	—	—	0.4358	1.0000	0.7179
precodespec_containers_image	image	—	—	0.2639	0.0000	0.1320
pthis_eks_cluster_has_priv	priv	—	—	0.4659	0.0000	0.2330
pusing_kubernetes_v1beta1	beta1	—	—	0.4032	0.0000	0.2016
pwhen_using_istio_with_kuber	kuber	—	—	0.3748	0.0000	0.1874
pwith_kubectli_know_docker	docker	—	—	0.4535	0.0000	0.2267
serviceaccount_pod_binding	binding	0.00	0.00	0.5001	0.2500	0.1875
anyof_intorstring	rel	—	—	0.6908	0.3333	0.5120
cronjob_job_pod	rel	—	—	0.8024	1.0000	0.9012
deep_deployment_container_re	re	—	—	0.8095	0.4000	0.6047
deep_statefulset_container_p	p	—	—	0.8304	0.8000	0.8152
deployment_pod_relation	rel	—	—	0.7320	0.3333	0.5327
extends_hpa_metric	rel	—	—	0.7317	0.6667	0.6992
hpa_deployment_pod	rel	—	—	0.8366	0.2500	0.5433
ingress_service_pod	rel	—	—	0.7994	0.5000	0.6497
namespace_quota_relation	rel	—	—	0.8742	0.6667	0.7704
networkpolicy_pod_selector	selector	—	—	0.8324	0.3333	0.5828
oneof_volume_source	rel	—	—	0.7064	0.2000	0.4532
pod_namespace_relation	rel	—	—	0.8215	1.0000	0.9107
pvc_storageclass	rel	—	—	0.7676	0.6667	0.7171
rbac_binding	rel	—	—	0.8573	1.0000	0.9287
secret_usage	rel	—	—	0.6880	0.6000	0.6440
service_selector	rel	—	—	0.7074	1.0000	0.8537
serviceaccount_token_binding	binding	—	—	0.8703	0.3333	0.6018
statefulset_pvc_pv	rel	—	—	0.8091	0.6667	0.7379
crashloopbackoff_oomkiller	killer	—	—	0.8216	0.0115	0.4165
deployment_rollout_strategy	strategy	—	—	0.6984	0.1034	0.4009

ID Fixture	Tipe	Sint.	Skema	Relevansi	Faithf.	AnsQ
imagepullbackoff_registry_se...	trbl	—	—	0.7773	0.0349	0.4061
pod_pending_no_resources	trbl	—	—	0.7605	0.0233	0.3919
service_no_endpoints	trbl	—	—	0.7841	0.0769	0.4305
clusterrole_read_pods	yaml	1.00	1.00	0.7305	0.5000	0.8076
clusterrolebinding_yaml	yaml	1.00	1.00	0.8816	1.0000	0.9704
configmap_env	yaml	1.00	1.00	0.6302	1.0000	0.9075
cronjob_backup	yaml	1.00	0.00	0.6563	0.2500	0.4766
daemonset_fluentd	yaml	1.00	1.00	0.6661	0.3333	0.7499
deployment_3_replicas	yaml	1.00	1.00	0.7579	0.3333	0.7728
deployment_liveness_probe	yaml	1.00	1.00	0.6142	1.0000	0.9035
ingress_rules	yaml	1.00	1.00	0.8142	0.3333	0.7869
networkpolicy_deny_all	yaml	1.00	1.00	0.7089	0.5000	0.8022
pod_configmap_volume	yaml	1.00	1.00	0.7760	0.6000	0.8440
pvc_dynamic	yaml	1.00	1.00	0.8274	0.5000	0.8319
rolebinding_dev	yaml	1.00	1.00	0.8807	1.0000	0.9702
service_nodeport	yaml	1.00	1.00	0.7001	0.3333	0.7584
statefulset_with_pvc	yaml	1.00	1.00	0.6332	0.6667	0.8250
yaml_layer3_required_fields	yaml	1.00	1.00	0.6163	0.5000	0.7791

C.2 Metrik *Retrieval Quality* (RetQ)

Tabel C.2 Nilai Metrik RetQ per Fixture (97 Fixture)

ID Fixture	Tipe	P@k	R@k	NDCG@k	EdgeCov	RetQ
kubecttl_export_namespaces	cmd	1.0000	0.5714	1.0000	1.0000	0.8831
kubecttl_find_pods_with_label	cmd	1.0000	0.3256	1.0000	0.2883	0.6842
kubecttl_force_delete_pod	cmd	1.0000	0.3256	1.0000	0.2883	0.6842
configmap_usage	cncpt	0.0000	0.0000	0.0000	0.0000	0.0000
daemonset_purpose	cncpt	1.0000	1.0000	1.0000	1.0000	1.0000
deployment_basic	cncpt	1.0000	0.8889	1.0000	0.9000	0.9550
hpa_target	cncpt	1.0000	1.0000	1.0000	1.0000	1.0000
ingress_purpose	cncpt	0.2653	0.9286	0.9508	0.8571	0.7238
job_cronjob	cncpt	1.0000	0.8333	1.0000	1.0000	0.9571
namespace_quota	cncpt	0.3846	1.0000	0.6181	1.0000	0.7597

ID Fixture	Type	P@k	R@k	NDCG@k	EdgeCov	RetQ
persistent_volume_concept	cncpt	0.1628	1.0000	0.4252	1.0000	0.6447
pod_spec	cncpt	0.7143	0.2740	0.6836	0.2258	0.5490
required_fields_containers	cncpt	0.1429	0.1333	0.1773	0.0000	0.2041
scope_accuracy_node	cncpt	1.0000	0.9375	1.0000	1.0000	0.9842
secret_types	cncpt	0.0000	0.0000	0.0000	1.0000	0.3333
service_types	cncpt	1.0000	0.7692	1.0000	0.8333	0.8842
statefulset_storage	cncpt	1.0000	1.0000	1.0000	1.0000	1.0000
storageclass_concept	cncpt	1.0000	0.7500	1.0000	1.0000	0.9345
add_env_from_configmap	flwp	1.0000	0.6667	1.0000	0.7500	0.8278
add_env_from_secret	flwp	1.0000	0.8000	1.0000	1.0000	0.9481
add_hpa_to_deployment	flwp	0.3175	1.0000	0.9969	1.0000	0.7994
add_liveness_probe	flwp	1.0000	0.7273	1.0000	0.8182	0.8676
add_pvc_to_statefulset	flwp	1.0000	1.0000	1.0000	1.0000	1.0000
add_readiness_probe	flwp	1.0000	0.8000	1.0000	1.0000	0.9481
add_resource_limits	flwp	1.0000	0.8000	1.0000	1.0000	0.9481
add_resource_limits_deployment	flwp	1.0000	0.8000	1.0000	0.9000	0.9148
change_service_type	flwp	1.0000	1.0000	1.0000	1.0000	1.0000
expose_with_ingress	flwp	0.2245	1.0000	1.0000	1.0000	0.7652
scale_existing_deployment	flwp	1.0000	1.0000	1.0000	1.0000	1.0000
update_image_version	flwp	1.0000	0.8889	1.0000	1.0000	0.9717
plan_autoscale_ha	plan	0.3827	0.9394	0.9595	0.9706	0.7993
plan_cronjob_batch	plan	0.2157	0.8462	0.8833	0.9091	0.6997
plan_namespace_isolation	plan	0.5000	0.8182	0.5028	0.9091	0.7100
plan_redis_persistent	plan	0.9762	0.9318	0.9839	0.9787	0.9707
plan_webapp_with_db	plan	0.5345	0.9118	0.9385	0.9429	0.8241
configmap_volume_mount	twld	0.0357	0.2500	0.0903	0.3333	0.2953
deployment_vs_statefulset	twld	0.0000	0.2581	1.0000	0.2647	0.6457
hpa_custom_metrics_yaml	twld	1.0000	1.0000	1.0000	1.0000	1.0000
pbased_on_a_hrefhttps://stackoverflow.com/questions/41391227/how-to-use-kubernetes-api-to-get-the-current-version-of-a-deployment	twld	1.0000	0.3256	1.0000	0.2883	0.6842
pcopied_from_here_a_hrefhttps://stackoverflow.com/questions/41391227/how-to-use-kubernetes-api-to-get-the-current-version-of-a-deployment	twld	0.0000	0.0000	0.0000	0.0000	0.0000
pheres_a_simplified_version_of_the_kubernetes_api	twld	1.0000	1.0000	1.0000	1.0000	1.0000
pi_am_quite_new_to_kubernetes	twld	0.0000	0.0000	0.0000	0.0000	0.0000
pi_am_wondering_if_it_is_possible_to_get_the_current_version_of_a_deployment	twld	0.0000	0.0000	0.0000	1.0000	0.3333

ID Fixture	Type	P@k	R@k	NDCG@k	EdgeCov	RetQ
pi_cant_find_documentation	division	0.0000	0.0000	0.0000	0.0000	0.0000
pi_have_a_command_to_run	rule	1.0000	0.5116	1.0000	0.5495	0.7897
pi_have_a_google_kubernetes	includes	0.0000	0.0000	0.0000	0.0000	0.0000
pi_have_a_tekton_pipeline	pipeline	0.1429	0.0714	0.0979	0.0000	0.0798
pi_want_to_pass_a_certificate	certificate	0.0000	0.0000	0.0000	1.0000	0.3333
pi_want_to_reject_all_requests	reject	0.0000	0.0000	0.0000	0.0000	0.0000
pid_like_to_confirm_information	info	0.0000	0.0000	0.0000	0.0000	0.1667
pim_trying_to_use_a_container	container	0.0000	1.0000	1.0000	1.0000	1.0000
pim_using_argocd_and_docker	docker	0.0000	0.0000	0.0000	0.0000	0.0000
pkubectl_provides_a_nice_way	niceway	0.0000	0.0000	0.0000	1.0000	0.3333
precodespec_containers	spec	0.0000	0.5116	1.0000	0.5495	0.7897
pthis_eks_cluster_has_priv	priv	0.0000	0.0000	0.0000	0.0000	0.0000
pusing_kubernetes_v1beta1	beta	0.0000	0.0000	0.0000	1.0000	0.3333
pwhen_using_istio_with_kuber	kuber	0.0000	0.0000	0.0000	1.0000	0.3333
pwith_kubectli_i_know_dan	dan	0.0000	0.0000	0.0000	0.0000	0.0000
serviceaccount_pod_binding	binding	0.1071	0.5000	0.3489	0.2000	0.3221
anyof_intorstring	rel	0.0690	0.6667	0.6364	1.0000	0.5828
cronjob_job_pod	rel	1.0000	0.6000	1.0000	0.6000	0.8250
deep_deployment_container	container	0.0000	0.2759	1.0000	0.2727	0.6585
deep_statefulset_container	container	0.0000	0.3333	1.0000	0.3191	0.6885
deployment_pod_relation	rel	1.0000	1.0000	1.0000	1.0000	1.0000
extends_hpa_metric	rel	1.0000	0.4524	1.0000	0.4200	0.7492
hpa_deployment_pod	rel	1.0000	0.4524	1.0000	0.4200	0.7492
ingress_service_pod	rel	1.0000	0.9545	1.0000	0.9130	0.9668
namespace_quota_relation	rel	1.0000	0.6667	1.0000	0.6000	0.7778
networkpolicy_pod_selector	selector	1.0000	0.6250	1.0000	0.4444	0.8064
oneof_volume_source	rel	0.0682	0.0652	0.0686	0.0169	0.1493
pod_namespace_relation	rel	1.0000	0.3218	1.0000	0.2883	0.6828
pvc_storageclass	rel	1.0000	1.0000	1.0000	1.0000	1.0000
rbac_binding	rel	1.0000	1.0000	1.0000	1.0000	1.0000
secret_usage	rel	0.1071	0.6000	0.4212	0.2500	0.3434
service_selector	rel	1.0000	0.2632	1.0000	0.2381	0.6530
serviceaccount_token_binding	binding	0.0000	0.6000	1.0000	0.5000	0.7250

ID Fixture	Type	P@k	R@k	NDCG@k	EdgeCov	RetQ
statefulset_pvc_pv	rel	1.0000	0.3415	1.0000	0.3261	0.6961
crashloopbackoff_oomkilled	trbl	1.0000	0.3218	1.0000	0.2857	0.6809
deployment_rollout_stuck	trbl	1.0000	0.2759	1.0000	0.2812	0.6649
imagepullbackoff_registry_pulls	trbl	1.0000	0.3256	1.0000	0.2883	0.6842
pod_pending_no_resources	trbl	1.0000	0.3256	1.0000	0.2883	0.6842
service_no_endpoints	trbl	1.0000	0.2564	1.0000	0.2326	0.6456
clusterrole_read_pods	yaml	0.1020	1.0000	1.0000	1.0000	0.7145
clusterrolebinding_yamls	yaml	0.9091	1.0000	1.0000	1.0000	0.9769
configmap_env	yaml	0.0000	0.0000	0.0000	1.0000	0.3333
cronjob_backup	yaml	0.2703	1.0000	1.0000	1.0000	0.7826
daemonset_fluentd	yaml	1.0000	1.0000	1.0000	1.0000	1.0000
deployment_3_replicas	yaml	1.0000	1.0000	1.0000	1.0000	1.0000
deployment_liveness_probe	yaml	1.0000	0.9667	1.0000	1.0000	0.9916
ingress_rules	yaml	0.4286	1.0000	1.0000	1.0000	0.8381
networkpolicy_deny_all	yaml	1.0000	1.0000	1.0000	1.0000	1.0000
pod_configmap_volume	yaml	1.0000	0.5116	1.0000	0.5495	0.7897
pvc_dynamic	yaml	1.0000	1.0000	1.0000	1.0000	1.0000
rolebinding_dev	yaml	1.0000	1.0000	1.0000	1.0000	1.0000
service_nodeport	yaml	1.0000	1.0000	1.0000	1.0000	1.0000
statefulset_with_pvc	yaml	0.9762	1.0000	1.0000	1.0000	0.9940
yaml_layer3_required_fields	yaml	1.0000	1.0000	1.0000	1.0000	1.0000

C.3 Metrik Reasoning Quality (ReaQ)

Tabel C.3 Nilai Metrik ReaQ per Fixture (97 Fixture)

ID Fixture	Type	HopAcc	Multi-Hop	Scope	Grounding	ReaQ
kubect1_export_namespaces	cmd	1.0000	1.0000	0.5000	1.0000	0.8750
kubect1_find_pods_with_label	cmd	1.0000	1.0000	1.0000	0.4000	0.8500
kubect1_force_delete_pod	cmd	1.0000	1.0000	1.0000	0.5000	0.8750
configmap_usage	cncpt	1.0000	1.0000	1.0000	1.0000	1.0000
daemonset_purpose	cncpt	1.0000	1.0000	1.0000	1.0000	1.0000
deployment_basic	cncpt	1.0000	1.0000	1.0000	1.0000	1.0000
hpa_target	cncpt	1.0000	1.0000	1.0000	1.0000	1.0000

ID Fixture	Type	HopAcc	Multi-Hop	Scope	Grounding	ReaQ
ingress_purpose	cncpt	1.0000	1.0000	1.0000	0.7500	0.9375
job_cronjob	cncpt	1.0000	1.0000	1.0000	1.0000	1.0000
namespace_quota	cncpt	1.0000	1.0000	1.0000	0.6667	0.9167
persistent_volume_concept	cncpt	1.0000	1.0000	1.0000	0.5556	0.8889
pod_spec	cncpt	1.0000	1.0000	1.0000	1.0000	1.0000
required_fields_containers	cncpt	1.0000	1.0000	1.0000	1.0000	1.0000
scope_accuracy_node	cncpt	1.0000	1.0000	0.5000	1.0000	0.8750
secret_types	cncpt	0.0000	1.0000	1.0000	1.0000	0.7500
service_types	cncpt	1.0000	1.0000	1.0000	0.7500	0.9375
statefulset_storage	cncpt	1.0000	1.0000	1.0000	0.7500	0.9375
storageclass_concept	cncpt	1.0000	1.0000	0.5000	0.6000	0.7750
add_env_from_configmap	flwp	1.0000	1.0000	1.0000	0.6000	0.9000
add_env_from_secret	flwp	1.0000	1.0000	1.0000	0.6000	0.9000
add_hpa_to_deployment	flwp	1.0000	1.0000	1.0000	0.6667	0.9167
add_liveness_probe	flwp	1.0000	1.0000	1.0000	0.6667	0.9167
add_pvc_to_statefulset	flwp	1.0000	1.0000	1.0000	0.5714	0.8929
add_readiness_probe	flwp	1.0000	1.0000	1.0000	0.6000	0.9000
add_resource_limits	flwp	1.0000	1.0000	1.0000	0.7143	0.9286
add_resource_limits_deployment	flwp	1.0000	1.0000	1.0000	0.6667	0.9167
change_service_type	flwp	1.0000	1.0000	1.0000	0.5000	0.8750
expose_with_ingress	flwp	1.0000	1.0000	1.0000	0.7500	0.9375
scale_existing_deployment	flwp	1.0000	1.0000	1.0000	0.6667	0.9167
update_image_version	flwp	1.0000	1.0000	1.0000	0.6667	0.9167
plan_autoscale_ha	plan	1.0000	1.0000	1.0000	0.5000	0.8750
plan_cronjob_batch	plan	1.0000	1.0000	1.0000	0.5000	0.8750
plan_namespace_isolation	plan	1.0000	1.0000	1.0000	0.8333	0.9583
plan_redis_persistent	plan	1.0000	1.0000	1.0000	0.5714	0.8929
plan_webapp_with_db	plan	1.0000	1.0000	1.0000	0.6364	0.9091
configmap_volume_mount	fld	1.0000	1.0000	1.0000	0.7500	0.9375
deployment_vs_statefulset	fld	1.0000	1.0000	1.0000	0.7500	0.9375
hpa_custom_metrics_yaml	fld	1.0000	1.0000	1.0000	0.5000	0.8750
pbased_on_a_hrefhttps://stackov	fld	1.0000	1.0000	1.0000	1.0000	1.0000
pcopied_from_here_a_hrefhttps://	fld	1.0000	1.0000	0.5000	1.0000	0.8750

ID Fixture	Type	HopAcc	Multi-Hop	Scope	Grounding	ReaQ
pheres_a_simplified_vndn	rel	1.0000	1.0000	1.0000	0.5000	0.8750
pi_am_quite_new_to_textuodp	rel	0.0000	1.0000	1.0000	0.5000	0.6250
pi_am_wondering_if_itrlsd	rel	0.0000	1.0000	1.0000	1.0000	0.7500
pi_cant_find_documentatvndn	rel	1.0000	1.0000	0.5000	1.0000	0.8750
pi_have_a_command_to_rvnd	rel	1.0000	1.0000	1.0000	1.0000	1.0000
pi_have_a_google_kubemndes	rel	0.0000	1.0000	0.5000	1.0000	0.6250
pi_have_a_tekton_codepnd	rel	1.0000	1.0000	1.0000	1.0000	1.0000
pi_want_to_pass_a_certrvdcate	rel	0.0000	1.0000	1.0000	0.7500	0.9375
pi_want_to_reject_allrlvddker	rel	0.0000	1.0000	0.5000	1.0000	0.6250
pid_like_to_confirm_infvdmatl	rel	1.0000	1.0000	1.0000	0.6000	0.9000
pim_trying_to_use_a_crvvdciner	rel	0.0000	1.0000	1.0000	0.6250	0.9062
pim_using_argocd_and_rvndant	rel	0.0000	1.0000	0.5000	1.0000	0.6250
pkubectl_provides_a_mvndway	rel	0.0000	1.0000	1.0000	0.3333	0.5833
precodespec_containersvndagel	rel	1.0000	1.0000	1.0000	0.5000	0.8750
pthis_eks_cluster_hasrvldprv	rel	0.0000	1.0000	1.0000	1.0000	0.7500
pusing_kubernetes_v1rvvldn_m	rel	0.0000	1.0000	1.0000	1.0000	0.7500
pwhen_using_istio_witrvldber	rel	1.0000	1.0000	1.0000	1.0000	1.0000
pwith_kubectl_i_know_rvndan	rel	1.0000	1.0000	0.5000	1.0000	0.8750
serviceaccount_pod_brvlding	rel	1.0000	1.0000	1.0000	1.0000	1.0000
anyof_intorstring	rel	1.0000	1.0000	1.0000	1.0000	1.0000
cronjob_job_pod	rel	1.0000	1.0000	1.0000	1.0000	1.0000
deep_deployment_containervrel	rel	1.0000	1.0000	1.0000	1.0000	1.0000
deep_statefulset_containervpl	rel	1.0000	1.0000	1.0000	1.0000	1.0000
deployment_pod_relativrel	rel	1.0000	1.0000	1.0000	1.0000	1.0000
extends_hpa_metric	rel	1.0000	1.0000	1.0000	1.0000	1.0000
hpa_deployment_pod	rel	1.0000	1.0000	1.0000	0.5000	0.8750
ingress_service_pod	rel	1.0000	1.0000	1.0000	0.7500	0.9375
namespace_quota_relativrel	rel	1.0000	1.0000	0.0000	1.0000	0.7500
networkpolicy_pod_selectrvr	rel	1.0000	1.0000	1.0000	1.0000	1.0000
oneof_volume_source	rel	1.0000	1.0000	1.0000	1.0000	1.0000
pod_namespace_relativrel	rel	1.0000	1.0000	1.0000	1.0000	1.0000
pvc_storageclass	rel	1.0000	1.0000	1.0000	1.0000	1.0000
rbac_binding	rel	1.0000	1.0000	1.0000	1.0000	1.0000

ID Fixture	Type	HopAcc	Multi-Hop	Scope	Grounding	ReaQ
secret_usage	rel	1.0000	1.0000	1.0000	1.0000	1.0000
service_selector	rel	1.0000	1.0000	1.0000	1.0000	1.0000
serviceaccount_token_binding	rel	1.0000	1.0000	1.0000	1.0000	1.0000
statefulset_pvc_pv	rel	1.0000	1.0000	1.0000	1.0000	1.0000
crashloopbackoff_oomkilled	trbl	1.0000	1.0000	1.0000	0.3333	0.8333
deployment_rollout_stuck	trbl	1.0000	1.0000	1.0000	0.8000	0.9500
imagepullbackoff_registry_selector	trbl	1.0000	1.0000	1.0000	0.5000	0.8750
pod_pending_no_resources	trbl	1.0000	1.0000	1.0000	0.5000	0.8750
service_no_endpoints	trbl	1.0000	1.0000	1.0000	0.6000	0.9000
clusterrole_read_pods	yaml	1.0000	1.0000	0.5000	0.6250	0.7812
clusterrolebinding_yaml	yaml	1.0000	1.0000	0.5000	0.6667	0.7917
configmap_env	yaml	0.0000	1.0000	1.0000	0.6667	0.6667
cronjob_backup	yaml	1.0000	1.0000	1.0000	0.5000	0.8750
daemonset_fluentd	yaml	1.0000	1.0000	1.0000	0.5000	0.8750
deployment_3_replicas	yaml	1.0000	1.0000	1.0000	0.6667	0.9167
deployment_liveness_probe	yaml	1.0000	1.0000	1.0000	0.5000	0.8750
ingress_rules	yaml	1.0000	1.0000	1.0000	0.7500	0.9375
networkpolicy_deny_all	yaml	1.0000	1.0000	1.0000	0.8000	0.9500
pod_configmap_volume	yaml	1.0000	1.0000	1.0000	0.6667	0.9167
pvc_dynamic	yaml	1.0000	1.0000	1.0000	0.4000	0.8500
rolebinding_dev	yaml	1.0000	1.0000	1.0000	0.6667	0.9167
service_nodeport	yaml	1.0000	1.0000	1.0000	0.5000	0.8750
statefulset_with_pvc	yaml	1.0000	1.0000	1.0000	0.4286	0.8571
yaml_layer3_required_fields	yaml	1.0000	1.0000	1.0000	0.6667	0.9167